

Attention is still what you need: Another Round of Exploring Shoup’s GGM*

Taiyu Wang[†] Cong Zhang[†] Hong-Sheng Zhou[‡] Xin Wang[§] Pengfei Chen[†]
Wenli Wang[†] Kui Ren[†] Chun Chen[†]

October 16, 2025

Abstract

The generic group model (GGM) is fundamental for evaluating the feasibility and limitations of group-based cryptosystems. Two prominent versions of the GGM exist in the literature: Shoup’s GGM and Maurer’s GGM. Zhandry (CRYPTO 2022) points out inherent limitations in Maurer’s GGM by demonstrating that several textbook cryptographic primitives, which are provably secure in Shoup’s GGM, cannot be proven secure in Maurer’s model.

In this work, we further investigate Shoup’s GGM and identify novel limitations that have been previously overlooked. Specifically, to prevent generic algorithms from generating valid group elements without querying the oracle, the model typically employs sufficiently large encoding lengths. This leads to sparse encodings, a setting referred to as the sparse generic group model (sparse GGM). We emphasize that this sparseness introduces several constraints:

- **Groups with AE and Black-Box Separation:** Shoup’s GGM is typically instantiated with elliptic curve groups, which admit admissible encodings (AE)—functions mapping from $\mathbb{Z}_p$ to elliptic curve points. We establish a black-box separation, showing that the sparse GGM fails to capture cryptographic groups that are both (1) computational Diffie-Hellman (CDH) secure and (2) compatible with admissible encodings.
- **Comparison with EC-GGM:** We examine the relationship between the sparse GGM and the Elliptic Curve Generic Group Model (EC-GGM) introduced by Groth and Shoup (EUROCRYPT 2022), which inherently yields CDH-secure groups with admissible encodings. Within the framework of indistinguishability, we prove that EC-GGM is strictly stronger than sparse GGM.
- **Dense Groups and Black-Box Separation:** We revisit groups with dense encodings and establish a black-box separation between CDH-secure dense groups and the sparse GGM.
- **Extension to Bilinear Settings:** Our results naturally extend to the sparse Generic Bilinear Group Model (GBM), demonstrating that the aforementioned constraints still hold.

In conclusion, our findings indicate that both feasibility and impossibility results in Shoup’s GGM should be reinterpreted in a fine-grained manner, encouraging further exploration of cryptographic constructions and black-box separations in EC-GGM or dense GGM.

*The work was mainly supported by National Key Research and Development Program of China, Grant No. 2023YFB3106000.

[†]The State Key Laboratory of Blockchain and Data Security, Zhejiang University, and Hangzhou High-Tech Zone (Binjiang) Blockchain and Data Security Research Institute, congresearch@zju.edu.cn

[‡]Virginia Commonwealth University, hszhou@vcu.edu

[§]Digital Technologies, Ant Group, wx352699@antgroup.com

Contents

1	Introduction	1
1.1	Our results	2
1.2	Interpretation	3
1.3	Technical overview	4
2	Preliminaries	9
2.1	Groups with Admissible Encodings	9
2.2	Dense Groups	11
2.3	Idealized Models	11
2.4	Indifferentiability	12
3	Sparse GGM vs. groups with nice admissible encodings	13
3.1	Non-interactive key exchange with nice AE	14
3.2	$\text{ae-}\mathcal{P}^{\text{NIKE}}$ does not exist in the sparse GGM	14
A	Sparse GGM vs. dense groups	24
A.1	Non-interactive key exchange with dense public-key space	24
A.2	$\text{d-}\mathcal{P}^{\text{NIKE}}$ does not exist in the sparse GGM	25
B	Sparse GGM vs. EC-GGM	30
B.1	EC-GGM statistically implies sparse GGM	30
B.2	Sparse GGM does not computationally imply EC-GGM	38
C	Groups with admissible encodings in EC-GGM	45
D	Sparse GGM vs. Dense GGM	46
D.1	Dense groups in dense GGM	46
D.2	Sparse GGM does not computationally imply dense GGM	48
E	Impossibility of groups in sparse GBM	55
E.1	Sparse GBM vs. groups with admissible encodings	55
E.2	Sparse GBM vs. dense groups	62
F	Discrete logarithm is hard in EC-GGM	68
G	Extracting new label is hard in sparse GGM/GBM	69
G.1	Analysis in sparse GGM	69
G.2	Analysis in sparse GBM	71

1 Introduction

Since the seminal work of Diffie and Hellman [DH76], group-based cryptosystems have become a cornerstone of modern cryptography, enabling secure key exchange, public key encryption, digital signatures, and many more. Building on this foundation, various types of cryptographic groups have been proposed, including multiplicative groups [DH76, ElG85], elliptic curve (EC) groups [Mil86, Kob87], and pairing-friendly groups [FR94, BGOS07], all of which play crucial roles in constructing cryptographic schemes and protocols.

In practical group-based cryptosystems, the **length** of group encodings plays a crucial role in efficiency, particularly affecting communication complexity. At equivalent security levels, groups with more compact encodings are typically preferred for implementation. NIST SP 800-186 [CMR⁺23] lists several recommended curves with 128-bit security, including Curve 25519, an elliptic curve defined over a 255-bit prime field. With standard point compression, a group element on Curve 25519 can be encoded in 256 bits (255 bits plus 1 for sign). Because EC groups generally offer more compact encodings at the same security level, they are widely favored for practical cryptographic constructions.

Importantly, EC groups always have **admissible encodings**; see [BF03, Ica09, BCI⁺10, LPS23]. Admissible encodings are broadly defined as efficiently computable functions that map from $\mathbb{Z}_p$ ¹ to group elements, satisfying two key properties: regularity (having constant preimage sizes) and preimage-computability (enabling efficient computation of all preimages for a given element in the range). These encodings enable the *oblivious sampling* of group elements without revealing their discrete logarithms, a critical feature for many group-based cryptosystems [BF03, BBB⁺18, MR19].

However, a fundamental limitation arises when attempting to prove the security of group-based cryptosystems — the inability to establish the unconditional hardness of the underlying computational assumptions (e.g., discrete logarithm, computational or decisional Diffie-Hellman) for any specific group. Over the past few decades, researchers have explored various approaches to establishing lower bounds on computational hardness, with the most prevalent method being the generic group model (GGM), where algorithms are limited to performing only generic group operations.

Roughly speaking, there are two widely accepted versions of the generic group model: Maurer’s GGM [Mau05] and Shoup’s GGM [Sho97]. Maurer’s GGM is modeled as a stateful system where algorithms make queries by referencing two group elements encountered during the computation (e.g., the 5th and 9th elements). In contrast, Shoup’s GGM is modeled as a random injection from the additive group $\mathbb{Z}_N$ to *sufficiently long* strings, where algorithms specify queries by providing two previously encountered strings from the computation.

Although both formulations appear in the literature, Maurer’s GGM is generally viewed as more restrictive. In particular, digital-signature schemes are impossible in Maurer’s GGM [DHH⁺21], and foundational constructions such as the Blum-Micali pseudorandom generator and the Goldreich-Goldwasser-Micali pseudorandom function cannot be realized in this model [Zha22]. For these reasons, we focus exclusively on Shoup’s GGM.

In Shoup’s GGM, the length of group encodings also plays a significant role. Numerous applications (e.g., [Gro16, Zha22, HMQS23, LZ24]) within this model rely on the requirement that algorithms cannot produce valid group elements without querying the oracle. To uphold this property, the generic group model must employ sparse encodings, making the sparse GGM a critical component of Shoup’s framework. However, sparse GGM does not support oblivious sampling, which implies that ***admissible encodings are effectively absent in sparse GGM!***

Attention is still needed! A potential gap arises between sparse variant of Shoup’s GGM and practical cryptographic groups, such as elliptic curve groups or other dense groups. Therefore, we plan to conduct further exploration of Shoup’s GGM, leading to the following research question:

Can sparse GGM effectively and accurately model elliptic curve groups or other dense cryptographic groups?

¹Here, p denotes the prime modulus used in elliptic curve groups, where the curve is typically defined by the equation $y^2 = x^3 + Ax + B \bmod p$.

1.1 Our results

In this work, we answer the question above negatively. Specifically, let G be a cryptographic group such that: (1) computational Diffie-Hellman (CDH) is assumed to hold with respect to $\mathbb{G}$; (2) the group encodings are dense or the group is associated with *nice* admissible encodings. Then $\mathbb{G}$ does not exist in the sparse GGM unconditionally.

In order to understand our results better, we start with a brief explanation of several important concepts including (1) sparse generic group models (GGMs), (2) dense groups, and (3) groups with nice admissible encodings. Let λ be the security parameter,

- *Sparse and dense GGMs.* We say GGM $\mathcal{G}$ is *sparse* if the GGM represents a random injection from $\mathbb{Z}_N$ to S , where N is a λ -bit prime, $S = \{0, 1\}^m$, and the difference $m - \lambda$ satisfying $m - \lambda \geq \omega(\log \lambda)$. Conversely, we say the GGM $\mathcal{G}$ is *dense* if the difference $m - \lambda \leq \Theta(\log \lambda)$.
- *Sparse and dense group encodings.* Similarly, we say a cryptographic group G of order N is *sparse* if the length of its group encodings, denoted by m , satisfies $m - \lambda \geq \omega(\log \lambda)$, and we say the group is *dense* if $m - \lambda \leq \Theta(\log \lambda)$.
- *Groups with nice admissible encodings.* We define a group $\mathbb{G}$ of order N to have a *nice admissible encoding* with respect to a constant ϱ and a polynomial poly_{AE} if there exists an efficiently computable function $\text{AE} : \mathbb{Z}_p \rightarrow \mathbb{G}$ satisfying the following conditions: (1) the preimage of any group element under AE is efficiently computable; (2) the encoding is ϱ -regular, meaning the size of any preimage set is at most ϱ ; (3) the domain size p is sufficiently large, i.e., $\frac{p}{N} \geq \text{poly}_{\text{AE}}$. As shown in [Ica09], all elliptic curve groups satisfy these conditions with $\varrho = 4$, and are therefore considered groups with nice admissible encodings.

1.1.1 Impossibility results in sparse GGM/GBM

Impossibility of groups with admissible encodings in a sparse GGM (Section 3). As previously discussed, the sparse GGM does not provide an admissible encoding. We argue that this limitation is an inherent drawback of the sparse GGM and proceed to establish a black-box separation between CDH-secure groups with admissible encodings and the sparse GGM.

Theorem 1.1 (Informal). *For any constant ϱ , CDH-secure groups with admissible encodings with respect to ϱ do not exist in the sparse generic group model.*

Impossibility of dense groups in a sparse GGM (Appendix A). We now turn our attention to the dense groups. Intuitively, the encodings in sparse GGM cannot be compressed in a generic manner and we demonstrate that this limitation represents another inherent drawback of the sparse GGM and proceed to establish a black-box separation between CDH-secure dense groups and the sparse GGM.

Theorem 1.2 (Informal). *CDH-secure dense groups do not exist in the sparse generic group model.*

Remark 1.1. In [JZW⁺24], Ji et al. investigate the relationship between CDH-secure groups with varying lengths of group encodings and demonstrate that shorter CDH-secure groups are separated from longer GGMs. However, we highlight that their analysis heavily depends on the assumption that both the groups and the GGM share the same security parameter—a condition we believe is not fully justified. In contrast, our results establish the black-box separation between dense groups and sparse GGM, resolving the open problem posed in [JZW⁺24].

Impossibility of groups with admissible encodings or dense groups in a sparse GBM (Appendix E). We further argue that the inherent difficulty in constructing groups with admissible encodings or dense groups persists as long as the idealized model remains sparse, even when the model is extended to the generic bilinear group model (GBM). The GBM $\mathcal{B}$ models two generic groups, i.e., the source generic group and the target generic group, and we say $\mathcal{B}$ is sparse if both the source generic group and the target generic group are sparse. We then establish black-box separations between CDH-secure groups with admissible encodings (CDH-secure dense groups) and the sparse GBM.

Theorem 1.3 (Informal). *For any constant ϱ , CDH-secure groups with admissible encodings with respect to ϱ does not exist in the sparse generic bilinear group model.*

Theorem 1.4 (Informal). *CDH-secure dense groups do not exist in the sparse generic bilinear group model.*

1.1.2 Exploring the relationship between EC-GGM and sparse GGM

Groth and Shoup [GS22] introduce a variant of the GGM, known as the *elliptic curve generic group model* (EC-GGM). Let E be the set of all points on an elliptic curve group of order N . The EC-GGM models a random injection from $\mathbb{Z}_N$ to E , while preserving certain structural properties inherent to elliptic curves—for instance, the preservation of which points share the same x -coordinate. It follows that CDH-secure groups with nice admissible encodings exist relative to the EC-GGM.

Theorem 1.5 (Informal). *CDH-secure groups with nice admissible encodings exist in the elliptic curve generic group model.*

Relationship between EC-GGM and sparse GGM (Appendix B). We next study the relationship between the EC-GGM and the sparse GGM in the framework of indistinguishability. Following Zhang and Zhandry’s analysis [ZZ23], we explore the relationship against computationally bounded adversary and prove that:

Theorem 1.6 (Informal). *In the framework of indistinguishability, EC-GGM is strictly stronger than sparse GGM.*

1.1.3 Exploring the relations between the sparse and the dense GGMs

Revisiting the relation between dense and sparse GGMs (Appendix D). We next revisit the study of the relationship between the dense GGM and the sparse GGM in the framework of indistinguishability against computationally bounded adversaries, and prove that:

Theorem 1.7 (Informal). *In the framework of indistinguishability, the dense generic group model is strictly stronger than the sparse generic group model.*

Remark 1.2. In [JZW⁺24], Ji et al. also prove that, within the framework of indistinguishability, the dense GGM is strictly stronger than the sparse GGM. However, we stress that their analysis also highly depends on the assumption that both the dense GGM and the sparse GGM share the same security parameter, which we believe is not fully justified. In contrast, our separation between the dense GGM and the sparse GGM is unconditional.

Theorem 1.8 (Informal). *CDH-secure dense groups exist in the dense GGM.*

1.2 Interpretation

Shoup’s GGM serves as the foundation for the group-based cryptosystems. Our findings show that extreme caution must be taken when proving security or establishing black-box separations within Shoup’s GGM. We next elaborate it from two perspectives.

Instantiating the GGM with elliptic curve groups. In the generic group model, algorithms are required to be generic. However, not all algorithms for group-based assumptions, such as the discrete logarithm problem, adhere to this requirement—for example, the index calculus attack against $\mathbb{Z}_N^*$ [Adl79]. Fortunately, in the case of elliptic curve groups, the only known attacks are generic in nature. As a result, the generic group model is typically instantiated using elliptic curve groups. However, to the best of our knowledge, in many practical cryptographic constructions (e.g., the zk-SNARKs in [Gro16]), the generic group model is often treated as a sparse GGM, where adversaries are restricted from obtaining valid group elements without making explicit queries. Furthermore, as shown in [Ica09], all elliptic curve groups fall into the category of groups with nice admissible encodings.

Our results establish a black-box separation between CDH-secure groups with admissible encodings and the sparse GGM, highlighting a potential and previously overlooked gap when instantiating the GGM with elliptic curve groups.

Black-box separations within the GGM. The generic group model is used to demonstrate the limitations of certain group-based cryptosystems. Notably, most known separations [Zha22, HMQS23] are established within sparse GGM. Our findings highlight important limitations of the black-box impossibility of sparse GGM, as it excludes both CDH-secure groups with nice admissible encodings and CDH-secure dense groups. Consequently, the relativizing separation between these groups and identity-based encryption (IBE) remains unresolved. Since many practical groups used fall under the category of groups with nice admissible encodings, our results motivate the further study of the complexity of such groups.

Furthermore, our findings indicate that, when examined in a fine-grained manner, the relationship between the GGM and the GBM must be reinterpreted. Specifically, Zhang and Zhandry [ZZ23] demonstrate that the GBM is strictly stronger than the GGM. In contrast, our results reveal that EC-GGM (or dense GGM) and sparse GBM are, in fact, incomparable.

In conclusion, our results encourage further investigation into both the construction of practical schemes and the establishment of black-box separations within the EC-GGM (or dense GGM). Additionally, our findings offer a deeper understanding of the complexities in GGMs and GBMs.

1.3 Technical overview

We now provide an overview of the core intuition and novelty behind our techniques.

1.3.1 Impossibility of groups with AE in sparse GGM.

To demonstrate the impossibility of a cryptographic primitive $\mathcal{P}$ within an idealized model $\mathcal{O}$, we typically employ the black-box separation methodology. Specifically, for any instantiation $\Pi^{\mathcal{O}}$ of $\mathcal{P}$, we construct a computationally unbounded adversary $\mathcal{A}$ such that (1) $\mathcal{A}$ breaks the security of $\Pi^{\mathcal{O}}$; and (2) $\mathcal{A}$ is query-efficient. The seminal result of Impagliazzo and Rudich [IR89], tells that the key agreement primitive cannot exist in the random oracle model (ROM). Specifically, for any protocol in this model, there exists an adversary capable of winning in a key recovery attack (KRA), which breaks the fundamental security requirement for key agreement. Returning to our context, it is important to note that CDH-secure groups with admissible encodings inherently imply KRA-secure key agreement (e.g., the Diffie-Hellman key exchange protocol). Therefore, if we could establish that KRA-secure key agreement is impossible in sparse GGM, the analysis would be complete.

For clarity of elaboration, we focus our analysis on a particular case of the key agreement primitive, i.e., the non-interactive key exchange (NIKE). We begin by briefly reviewing the key concepts behind the impossibility of KRA-secure NIKE in the random oracle model [IR89, BKS11], and then proceed to integrate those insights into our analysis.

KRA-secure NIKE vs. ROM. Let $\Pi^{\mathcal{H}} := (\text{KGen}^{\mathcal{H}}, \text{SHK}^{\mathcal{H}})$ be a NIKE scheme in the random oracle model $\mathcal{H}$, where (1) $\Pi^{\mathcal{H}}$ achieves *perfect* correctness, and (2) both KGen and SHK make at most q queries. Consider Alice and Bob as two honest parties, each associated with public keys pk_1 and pk_2 , respectively. We then construct an adversary $\mathcal{A}$ that, given pk_1 and pk_2 , outputs the valid shared key with a good probability. Specifically, the adversary $\mathcal{A}$ initializes two empty sets $S_{\text{que-res}}$ and S_{key} , and then proceeds through $4q + 1$ iterations of the following phases:

- *Simulation.* Simulate a proper view for Alice. Specifically, this view contains a set of query/response pairs $\tilde{S}_{\mathcal{A}}$ along with a private key $\tilde{\text{sk}}_1$ such that: (1) $\tilde{S}_{\mathcal{A}}$ is consistent with $S_{\text{que-res}}$ ²; and (2) this view induces the correct public key for Alice: $\text{KGen}^{\tilde{S}_{\mathcal{A}} \cup S_{\text{que-res}}}(\tilde{\text{sk}}_1) = \text{pk}_1$. Next, compute the shared key based on the simulated view, i.e., $\text{shk} = \text{SHK}^{\tilde{S}_{\mathcal{A}} \cup S_{\text{que-res}}}(\tilde{\text{sk}}_1, \text{pk}_2)$, and insert shk into the set S_{key} .
- *Update.* Update all queries in $\tilde{S}_{\mathcal{A}} \setminus S_{\text{que-res}}$ by accessing the random oracle $\mathcal{H}$, and add the corresponding query/response pairs into the set $S_{\text{que-res}}$.

²Since $S_{\text{que-res}}$ is only a subset of $\mathcal{H}$, it is possible that $\tilde{S}_{\mathcal{A}}$ may be inconsistent with the random oracle $\mathcal{H}$.

Finally, $\mathcal{A}$ outputs the majority of the shared keys from the set S_{key} .

Next, we outline the key intuition behind why $\mathcal{A}$ is likely to win in the KRA-secure game with a good probability. Let S_{Bob} denote the set of query/response pairs made by Bob during the real execution of the key exchange protocol. For any iteration, there are two cases:

- Case 1 (**Bad**): $\exists(\text{que}_A, \text{res}_A) \in \tilde{S}_A, (\text{que}_B, \text{res}_B) \in S_{\text{Bob}}$ s.t. $\text{que}_A = \text{que}_B$ but $\text{res}_A \neq \text{res}_B$.
- Case 2 (**Good**): $\forall(\text{que}_A, \text{res}_A) \in \tilde{S}_A, (\text{que}_B, \text{res}_B) \in S_{\text{Bob}}$, we have that if $\text{que}_A = \text{que}_B$, then $\text{res}_A = \text{res}_B$.

Observe that case 1 can occur in at most $2q$ iterations, as $|S_{\text{Bob}}| \leq 2q$. Once this case takes place, the update phase will absorb at least one pair $(\text{que}_B, \text{res}_B) \in S_{\text{Bob}}$ into $S_{\text{que-res}}$. For case 2, we note that when it occurs, there exists an alternate oracle $\tilde{\mathcal{H}}$ that remains consistent with both $\tilde{S}_A$ and S_{Bob} . Due to perfect correctness, the shared key computed during that iteration is guaranteed to be valid. Furthermore, since case 2 occurs in at least $2q + 1$ iterations, this ensures that the majority of keys in S_{key} are valid.

However, in the case of GGM, the attack fails immediately, as the GGM itself implies the KRA-secure NIKE. Next, we explain the reasoning behind the failure of the attack.

Why the attack fails in GGM? In contrast to the ROM, the GGM features two types of queries: labeling queries (e.g., $x, \mathcal{G}(x)$) and addition queries (e.g., $\mathcal{G}(x), \mathcal{G}(y), \mathcal{G}(x + y)$). Hence, to guarantee the validity of the shared key, we should define S_{Bob} to include all group encodings present in the queries (both labeling and addition) along with their corresponding discrete logarithms (which Bob may not know). Consequently, for any iteration, there are *three* cases:

- Case 1 (**Bad**): $\exists(\text{que}_A, \text{res}_A) \in \tilde{S}_A, (\text{que}_B, \text{res}_B) \in S_{\text{Bob}}$ s.t. $\text{que}_A = \text{que}_B$ but $\text{res}_A \neq \text{res}_B$.
- Case 2 (**Bad**): $\exists(\text{que}_A, \text{res}_A) \in \tilde{S}_A, (\text{que}_B, \text{res}_B) \in S_{\text{Bob}}$ s.t. $\text{que}_A \neq \text{que}_B$ but $\text{res}_A = \text{res}_B$.
- Case 3 (**Good**): $\forall(\text{que}_A, \text{res}_A) \in \tilde{S}_A, (\text{que}_B, \text{res}_B) \in S_{\text{Bob}}$, we have that if $\text{que}_A = \text{que}_B$ then $\text{res}_A = \text{res}_B$, and vice versa.

Observe that case 1 and case 3 can be handled similarly as above. However, case 2 may always occur, meaning it is impossible to find a GGM instance that is consistent with both $\tilde{S}_A$ and S_{Bob} . This implies the aforementioned attack fails.

Upon further exploration, we observe that the occurrence of case 2 indicates that the adversary is able to generate a valid group element without knowing its discrete logarithm, i.e., without making labeling queries. Therefore, to advance the analysis, it is necessary to handle the NIKE and GGM with a more fine-grained approach, ensuring that no query-efficient adversary can generate a valid group element without knowing its discrete logarithm. Specifically, the analysis aims to identify a hard problem related to the fine-grained NIKE and GGM, and prove that if an adversary, given the public keys pk_1 and pk_2 , succeeds in outputting a valid group element without making queries, then the hard problem does not hold anymore.

Solution in [JZW⁺24] and its limitations. Recently, Ji et.al. [JZW⁺24] demonstrated that CDH-secure cryptographic groups with shorter group descriptions cannot exist in the generic group model with longer encodings. Specifically, they establish a *somewhat* black-box separation between KRA-secure NIKE schemes associated with shorter public keys and the GGMs with longer group encodings³. The key idea in their work is that, given the public keys pk_1 and pk_2 , the extracted group element str can fall into one of two cases: (1) str is related to both pk_1 and pk_2 ; or (2) str is related to only one of the public keys but independent of the other. In the former case, Ji et.al. explain that str is typically a frequent query, which can be easily handled by repeatedly running the key generation algorithm on sufficiently many random inputs. In the latter case, where str is only relevant to only one public key (e.g., pk_1), they observe that pk_1 is the sole carrier of information about str . However, since the length of pk_1 is significantly shorter than that of str , it lacks the necessary capacity to encode all the information required for the recovery of str . Consequently, no query-efficient algorithm can extract str with a non-negligible probability.

³The formal primitive can be trivially constructed via shorter CDH-secure groups.

Furthermore, the hard problem identified in [JZW⁺24] can be interpreted as follows: given any public key (which is shorter), no query-efficient algorithm can extract a valid group element (which is longer) except with negligible probability.

However, the hard problem in [JZW⁺24] faces an inherent limitation. Specifically, it holds only if the length of pk_1 is significantly shorter than that of str . This condition is guaranteed when both the KRA-secure NIKE and the GGM *share the same security parameter*. However, if NIKE and the GGM are associated with different security parameters, this condition may no longer hold. Exploring deeper, we observe that the reason for this limitation lies in the fact that “shorter” and “longer” are relative terms with respect to the security parameter. A public key considered “shorter” under a large security parameter may still carry enough information to extract a “longer” GGM encoding when the security parameter is small. Therefore, we argue that the hard problem in [JZW⁺24] is based on an unjustified assumption, and their analysis *deviates from the conventional black-box separation*⁴.

Our techniques. To establish a black-box separation between groups with nice admissible encodings and sparse GGMs, it is essential to identify a new and *unconditional* hard problem. We leverage the sparseness property of the GGM and define the hard problem as follows: for any query-efficient algorithm with access to the sparse GGM, which only receives the security parameter as input, it cannot output a new and valid group element except with negligible probability⁵. By saying that a group element str is new, we mean that: (1) str has been used as an input in some addition queries made by the algorithm, and (2) str has not appeared as the output of any prior queries. As discussed in [Zha22], the hardness of this problem holds unconditionally. Next we elaborate on the high-level strategy for integrating this hard problem into the black-box separation.

Roughly speaking, we show that if the adversary $\mathcal{A}$, given inputs pk_1 and pk_2 , is able to generate a new group element with a good probability, then we can construct an extractor $\mathcal{E}$ which, given only the security parameter as input, can also output a new group element with a good probability.

The first technical challenge of our analysis is determining how the extractor $\mathcal{E}$ generates the two public keys. A natural approach would be:

Step 1: $\mathcal{E}$ randomly selects two private keys (sk_1 and sk_2), computes the corresponding public keys ($pk_i = \text{KGen}^G(sk_i)$), and runs $\mathcal{A}^G(pk_1, pk_2)$.

Step 2: Whenever $\mathcal{A}$ issues an addition query using a new group element (denoted as str) as input, $\mathcal{E}$ outputs str .

Unfortunately, this approach fails immediately. Given the fact that the GGM is sparse, any new group element str generated by $\mathcal{A}$ should be related to either pk_1 or pk_2 . More precisely, str is likely to appear during the execution of $\text{KGen}^G(sk_1)$ or $\text{KGen}^G(sk_2)$ with high probability. As a result, if following the aforementioned approach, then the extractor would fail to output a new group element, even if $\mathcal{A}$ does so.

To overcome this challenge, we must adopt an alternative method for generating the public keys, leveraging admissible encodings as part of the solution. Specifically, we configure the NIKE as a KRA-secure NIKE with nice admissible encodings, meaning there exists an admissible encoding that maps from $\mathbb{Z}_p$ to the public keys, where p is sufficiently large. Clearly, CDH-secure groups with nice admissible encodings implies such a NIKE. We then construct the extractor $\mathcal{E}$ as follows:

Step 1: $\mathcal{E}$ randomly selects two seeds ($seed_1$ and $seed_2$), computes the corresponding public keys via admissible encodings ($pk_i = \text{AE}^G(seed_i)$), and runs $\mathcal{A}^G(pk_1, pk_2)$.

Step 2: Whenever $\mathcal{A}$ issues an addition query using a new group element (denoted as str) as input, $\mathcal{E}$ outputs str .

⁴The conventional black-box separation framework establishes impossibility results unconditionally, whereas [JZW⁺24] demonstrates impossibility only under certain unjustified conditions.

⁵The probability is taken over the sampling of the GGM instances and the internal randomness of the algorithm.

At this point, the second technical challenge emerges. Specifically, let S_{AE} denote the set of the query/response pairs that arise during the execution of $\text{pk}_1 = \text{AE}^{\mathcal{G}}(\text{seed}_1)$ and $\text{pk}_2 = \text{AE}^{\mathcal{G}}(\text{seed}_2)$. Note that if the new group element generated by the adversary $\mathcal{A}$ consistently falls within the set S_{AE} , the extractor above will still fail to output a new group element.

To address this challenge, we refine the adversary's strategy. Specifically, we introduce a preprocessing phase for the adversary, define as follows:

- *Preprocessing.* Given the inputs pk_1 and pk_2 , the adversary $\mathcal{A}$ runs the inverse of the admissible encodings, computing $\text{seed}_i = \text{inv-AE}^{\mathcal{G}}(\text{pk}_i)$. Subsequently, $\mathcal{A}$ recalculates $\text{pk}_i = \text{AE}^{\mathcal{G}}(\text{seed}_i)$ and collects all associated query/response pairs into the set $S_{\text{que-res}}$.

The adversary then proceeds as follows: before entering the iteration phase, it first executes the preprocessing phase.

Next, we explain why the preprocessing phase is effective. It is important to note that, following the preprocessing phase, the adversary $\mathcal{A}$ has already gathered all the query/response pairs contained in S_{AE} . As a result, whenever $\mathcal{A}$ generates a new group element, it will also be new to $\mathcal{E}$.

The above outline is not comprehensive; for detailed technical explanations, please refer to Section 3.

The key idea of our analysis is that valid public keys can be generated through two distinct methods. In the context of NIKE with dense encodings, there are likewise two methods for generating public keys, one of which even does not require making any queries. Thus, our analysis can be naturally extended to establish a black-box separation between CDH-secure dense groups and sparse GGM. Furthermore, the hard problem above remains valid within the sparse generic bilinear group model (GBM), allowing all impossibility results to naturally extend to the sparse GBM as well.

1.3.2 EC-GGM vs. sparse GGM

To demonstrate that the elliptic curve generic group model (EC-GGM) is strictly stronger than the sparse generic group model (sparse GGM), we frame our goal within framework of indifferenciability. In particular, we establish that EC-GGM (denoted as $\text{ec-}\mathcal{G}$) statistically implies sparse GGM (denoted as $\mathcal{G}$); while the sparse GGM does not computationally imply EC-GGM.

EC-GGM statistically implies sparse GGM. We begin by explaining how $\text{ec-}\mathcal{G}$ implies $\mathcal{G}$ against statistical adversaries. Let $\text{ec-}\mathcal{G} = (\text{ec-}\mathcal{G}^{\text{L}}, \text{ec-}\mathcal{G}^{\text{A}})$ denote an EC-GGM corresponding to an elliptic curve defined over $\mathbb{Z}_p$, where each group element is represented as $(u, b) \in \mathbb{Z}_p \times \{0, 1\}$. According to the Hasse bound, the encoding of EC-GGM is inherently dense. To extend the encoding length to m bits for the sparse GGM, we introduce an additional oracle, the random permutation Perm over $\{0, 1\}^m$, along with its inverse, Perm^{Inv} . The labeling function is then defined as follows:

$$\text{L}^{\text{ec-}\mathcal{G}}(x) := \text{Perm}(\text{ec-}\mathcal{G}^{\text{L}}(x) || 0 \cdots 0).$$

The addition algorithm is constructed by applying the inverse oracle Perm^{Inv} . To demonstrate indifferenciability, we need to construct a simulator $\mathcal{S}$ that, with access to $\mathcal{G}$, can accurately simulate both $\text{ec-}\mathcal{G}$ and $(\text{Perm}, \text{Perm}^{\text{Inv}})$.

To simulate $\text{ec-}\mathcal{G}$, the simulator maintains a table to record the correspondence between the elliptic curve points $P := (u, b)$ and $\text{str} := \mathcal{G}(x)$. An important property of the EC-GGM is that for any $(u, b) \leftarrow \text{ec-}\mathcal{G}^{\text{L}}(x)$ and $(u', b') \leftarrow \text{ec-}\mathcal{G}^{\text{L}}(-x)$, it holds that $u = u'$ and $b = 1 - b'$. Consequently, whenever the simulator records the tuple (P, str) , where $P := (u, b)$ and $\text{str} := \mathcal{G}(x)$, it additionally records $(-P, \mathcal{G}(-x))$ with $-P := (u, 1 - b)$. Notably, $\mathcal{G}(-x)$ can be obtained by making additional queries to $\mathcal{G}$ on str , even without knowledge of x .

To simulate Perm , the simulator $\mathcal{S}$ first checks whether the query corresponds to an elliptic curve point and whether there exists a corresponding $\mathcal{G}(x)$. If both conditions are satisfied, $\mathcal{S}$ just responds with $\mathcal{G}$. If not, $\mathcal{S}$ responds with $\mathcal{G}(x)$ for a randomly sampled $x \in \mathbb{Z}_N$ or with a randomly chosen invalid string str , depending on whether the query corresponds to a valid point. The simulation of Perm^{Inv} follows a similar strategy.

Remark 1.3. Careful readers might argue that, within the elliptic curve generic group model, adversaries could perform certain non-generic operations, such as oblivious sampling. As a result, EC-GGM extends the adversaries' capabilities, necessitating a reinterpretation of the hardness of certain security assumptions (e.g., DLOG or CDH).

Fortunately, due to statistical indistinguishability, the hardness of the CDH assumption remains intact. Specifically, we demonstrate that if a statistical yet query-efficient adversary exists that can break CDH in EC-GGM, we can construct a differentiator capable of breaking indistinguishability. More precisely, the differentiator acts as the challenger in the CDH game, interacts with the adversary, and distinguishes between the real and ideal worlds by observing whether the adversary successfully wins the CDH game.

Next, we demonstrate that even in the context of computationally bounded adversaries, it is not possible to construct an indistinguishable EC-GGM within a sparse GGM.

Sparse GGM does not computationally imply EC-GGM. Suppose we have a proposed construction of an elliptic curve generic group model derived from a sparse GGM, denoted as $\Pi^{\mathcal{G}} := (\mathcal{L}^{\mathcal{G}}, \mathcal{A}^{\mathcal{G}})$. How can we demonstrate that a computationally bounded adversary is able to differentiate $\Pi^{\mathcal{G}}$ from $\text{ec-}\mathcal{G}$? A common approach is to identify a security game that remains secure in $\text{ec-}\mathcal{G}$ but can be easily broken in any construction of $\Pi^{\mathcal{G}}$.

Following the strategy outlined in [ZZ23], we define the security game as *discrete logarithm identification* (DLI), a variant of the discrete logarithm problem. Intuitively, the DLI game involves the following: given $\text{str} := \mathcal{L}(x)$, construct a probabilistic, efficient, and query-free circuit C such that $C(x)$ accepts with high probability, while $C(x')$ overwhelmingly rejects for all $x' \neq x$.

Next, we briefly recall the techniques from [ZZ23], where Zhang and Zhandry establish a separation between GGM and ROM against computationally bounded adversaries. Clearly, DLI is secure in GGM. To establish the separation, Zhang and Zhandry show that DLI can be easily broken in any group constructed from the ROM. In a nutshell, the adversary constructs the circuit C as follows: $C(\cdot)$ functions identically to $\mathcal{L}^{\mathcal{H}}(\cdot)$, but without access to $\mathcal{H}$, and accepts the input if the reconstructed result matches $\mathcal{L}^{\mathcal{H}}(x)$ (hardwired in C). The key idea in [ZZ23] is to *anticipate* the queries that C will make to the random oracle model. This enables the adversary to make all sensitive queries in advance and hardcode the corresponding query/response pairs into C , thereby creating an oracle-free circuit. Zhang and Zhandry demonstrate that, with high probability, all sensitive queries can be collected as follows:

1. randomly sample $r, r_1, \dots, r_n$ ⁶, execute $\mathcal{L}^{\mathcal{H}}(r), \mathcal{L}^{\mathcal{H}}(-r), \mathcal{L}^{\mathcal{H}}(r_1), \dots, \mathcal{L}^{\mathcal{H}}(r_n)$, and collect all the queries into $S_{\text{que-res}}$;
2. execute $\mathcal{L}^{\mathcal{H}}(x - r) \leftarrow \mathcal{A}^{\mathcal{H}}(\mathcal{L}^{\mathcal{H}}(x), \mathcal{L}^{\mathcal{H}}(-r))$ and $\mathcal{A}^{\mathcal{H}}(\mathcal{L}^{\mathcal{H}}(x - r), \mathcal{L}^{\mathcal{H}}(r))$, and collect all the queries into $S_{\text{que-res}}$.

Next, we outline our approach for the incorporating aforementioned technique into the analysis within sparse GGM. Specifically, given an input $\mathcal{L}^{\mathcal{G}}(x)$, the adversary $\mathcal{A}$ collects all the queries as described above, hardwires both the query/response pairs and $\mathcal{L}^{\mathcal{G}}(x)$ into the circuit C , which operates identically to $\mathcal{L}^{\mathcal{G}}(\cdot)$, and then outputs $C(\cdot)$. We then analyze the probability of $\mathcal{A}$ wins. Let Q_x be the set of all query/response pairs generated during the execution of $\mathcal{L}^{\mathcal{G}}(x)$ ⁷, for each pair $(\text{que}, \text{res}) \in Q_x$, there are four possible cases:

- Case 1: The label $\mathcal{L}^{\mathcal{G}}(x)$ does not depend on **res** at all;
- Case 2: The label $\mathcal{L}^{\mathcal{G}}(x)$ depends on **res**, but **res** does not appear in the response when computing $\mathcal{A}^{\mathcal{G}}(\mathcal{L}^{\mathcal{G}}(x - r), \mathcal{L}^{\mathcal{G}}(r))$;
- Case 3: The label $\mathcal{L}^{\mathcal{G}}(x)$ depends on **res**, which is obtained by making “labeling” query to $\mathcal{G}$ when computing $\mathcal{A}^{\mathcal{G}}(\mathcal{L}^{\mathcal{G}}(x - r), \mathcal{L}^{\mathcal{G}}(r))$;

⁶Here n is a sufficiently large integer.

⁷Without loss of generality, we assume that $\mathcal{L}^{\mathcal{G}}(\cdot)$ only makes labeling queries.

- Case 4: The label $L^G(x)$ depends on **res**, which is obtained by making “addition” query to $\mathcal{G}$ when computing $A^G(L^G(x-r), L^G(r))$;

Next, we present the analysis for handling each of the four cases.

For **(que, res)** in case 1 (same as in [ZZ23]), referred to as a **non-sensitive query**, since $L^G(x)$ does not depend on **res**, we can replace the response to **que** with a uniformly sampled string without affecting the final outcome.

For **(que, res)** in case 2 (same as in [ZZ23]), referred to as a **sensitive but frequent query**, since the computation of $A^G(L^G(x-r), L^G(r))$ does not obtain **res** by making queries, **res** must be extracted from the inputs, i.e., $L^G(x-r)$ and/or $L^G(r)$. This indicates that, with high probability, **(que, res)** $\in Q_x \cap (Q_{x-r} \cup Q_r)$. Moreover, since x , $x-r$ and r are pairwise independent, which means that $Q_x \cap Q_{x-r}$ and $Q_x \cap Q_r$ only contains “frequent” queries. Therefore, this query/response tuple can be collected by running $L^G(\cdot)$ on sufficiently many random inputs.

For **(que, res)** in case 3 (same as in [ZZ23]), referred to as a **sensitive labeling query**, we can collect all labeling queries made during the computation of $A^G(L^G(x-r), L^G(r))$ directly.

For **(que, res)** in case 4 (different from [ZZ23]), referred to as a **sensitive addition query**, where **que** = **(str₁, str₂)** consists of two valid group elements. Although **res** appears in this query, collecting this kind of query is not usually useful for our purpose. Specifically, when executing $L^G(x)$, the algorithm may only issue labeling queries at points $(x_1, \dots, x_q)$, whereas $S_{\text{que-res}}$ might only contain query/response pairs in the form of addition, such as $(\mathcal{G}(y_i), \mathcal{G}(z_i), \mathcal{G}(y_i + z_i))$, without explicitly knowing y_i or z_i . Consequently, during the reconstruction of $L^G(x)$, the algorithm may issue labeling queries at certain points, but $\mathcal{C}$ is unable to identify which tuple corresponds to the correct response, resulting in the reconstruction failure.

To overcome this technical challenge, we once again leverage the sparseness property of the GGM. Note that the occurrence of the query in case 4 indicates that the adversary is able to generate a new and valid group element without knowing its discrete logarithm. Thus, by applying the same analysis as outlined in Section 1.3.1, we can construct an extractor that, given only the security parameter as input, is able to produce a new and valid group element whenever the query in case 4 occurs. This demonstrates that the query in case 4 occurs with negligible probability.

The above outline is not comprehensive; for detailed technical explanations, please refer to Appendix B.

2 Preliminaries

Notations. In this paper, let $\lambda \in \mathbb{Z}$ denote the security parameter. We use $x||y$ to represent the concatenation of the strings x and y . For a finite set S , we denote a random sample s from S according to the uniform distribution as $s \xleftarrow{\$} S$, and the size of S as $|S|$. For a probabilistic algorithm Alg , we overload the notation and write $y \xleftarrow{\$} \text{Alg}(I)$ to denote that the variable y is obtained by running Alg on input I , where I may be a tuple $I = (I_1, \dots, I_n)$. If Alg is deterministic, we use the notation “ $\leftarrow$ ” instead of “ $\xleftarrow{\$}$ ”.

A positive function $\text{negl}(\cdot)$ is said to be negligible if, for all positive polynomial $\text{poly}(\cdot)$, there exists a constant $\lambda_0 > 0$ such that for all $\lambda > \lambda_0$, it holds that $\text{negl}(\lambda) < 1/\text{poly}(\lambda)$. We say that a function $\rho(\cdot)$ is noticeable in λ if its inverse, $1/\rho(\lambda)$, is polynomial in λ .

2.1 Groups with Admissible Encodings

In this work, we treat groups with admissible encodings as a cryptographic primitive. We first recall the formal definition of a primitive given by [RTV04].

Definition 2.1 (Cryptographic Primitive [RTV04]). *A primitive $\mathcal{P}$ is a pair $\langle \mathcal{F}, \mathcal{R} \rangle$, where $\mathcal{F}$ is a set of functions $f : \{0, 1\}^* \mapsto \{0, 1\}^*$ specifying correctness property, and $\mathcal{R}$ is a relation on pairs $\langle f, \mathcal{A} \rangle$, with $f \in \mathcal{F}$ and $\mathcal{A}$ an adversarial machine, specifying security property.*

- **Efficient implementation.** A function f is said to implement $\mathcal{P}$ or be an implementation of $\mathcal{P}$ if $f \in \mathcal{F}$. An efficient implementation of $\mathcal{P}$ is an implementation of $\mathcal{P}$ that is computable in polynomial time.
- **Secure implementation.** An adversarial machine $\mathcal{A}$ is said to $\mathcal{P}$ -break $f \in \mathcal{F}$ if $\langle f, \mathcal{A} \rangle \in \mathcal{R}$. A secure implementation of $\mathcal{P}$ is an implementation of $\mathcal{P}$ such that no PPT adversarial machine can $\mathcal{P}$ -break f .

We say that the **primitive $\mathcal{P}$ exists** if there is an efficient and secure implementation of $\mathcal{P}$.

To ensure correctness, we consider cryptographic groups that support an efficiently computable encoding from $\mathbb{Z}_p$ (for some integer p) into the group. Various formulations of such admissible encodings appear in the literature [BF03, BCI⁺10, LPS23]; in this work, we adopt the definition from [LPS23].

Definition 2.2 (Admissible Encodings). A function $\text{AE} : S \mapsto T$ between two finite sets is an admissible encoding with respect to ϱ if it satisfies the following conditions:

Computable: The function AE is computable in polynomial time.

Sampleable: Given any element t in the image of AE , one can efficiently compute its full preimage $\text{inv-AE}(t)$.

ϱ -regular: For any $t \in T$, the size of the preimage $|\text{inv-AE}(t)|$ is at most ϱ , where ϱ is a small constant.

Remark 2.1. The definition of admissible encodings in [BCI⁺10] differs slightly from that in Definition 2.2. Specifically, it requires that for uniformly distributed inputs $s \in S$, the output distribution $\text{AE}(s)$ is statistically close to uniform over T . However, most known admissible encodings—such as Icart’s encoding [Ica09, FT10]—do not produce uniformly distributed outputs. In this work, we adopt the definition from [LPS23], and note that the formulation in [BCI⁺10] can be seen as a special case of Definition 2.2 with $\varrho = 1$.

When working with admissible encodings for cryptographic groups, the target set T is typically identified with the set of group elements. A key application of admissible encodings is *oblivious sampling*, which requires that the encoding’s image covers a noticeable fraction of the set T . This, in turn, requires the domain to be sufficiently large. Specifically, we say an admissible encoding AE for a group $\mathbb{G}$ is *nice* if: (1) AE maps from $\mathbb{Z}_p$ to $\mathbb{G}$, and (2) the ratio $\frac{p}{|\mathbb{G}|}$ is not small.

As shown in [Ica09], all elliptic curve groups admit nice admissible encodings. We now formalize the notion of CDH-secure groups equipped with nice admissible encodings.

Definition 2.3 (CDH-Secure Group with Nice Admissible Encodings). The CDH-secure group with nice admissible encoding with respect to a constant ϱ and a polynomial poly_{AE} , denoted $\text{ae-}\mathcal{P}^{\text{CDH}}$, is defined as a pair $\langle \text{ae-}\mathcal{F}^{\text{CDH}}, \text{ae-}\mathcal{R}^{\text{CDH}} \rangle$, satisfying the following conditions:

1. The set $\text{ae-}\mathcal{F}^{\text{CDH}}$ consists of group-generation functions f that, on input a security parameter, f outputs the description of a finite cyclic group along with an additional function. Specifically, we write $(\mathbb{G}, g, N, \text{AE}) \leftarrow f(1^\lambda)$, where $\mathbb{G}$ is a cyclic group of λ -bit prime order N , g is a generator of $\mathbb{G}$, and $\text{AE} : \mathbb{Z}_p \mapsto \mathbb{G}$ is a ϱ -regular admissible encoding, with $p \geq N/\text{poly}_{\text{AE}}(\lambda)$.
2. The pair $\langle f, \mathcal{A} \rangle$ belongs to $\text{ae-}\mathcal{R}^{\text{CDH}}$ for a function $f \in \text{ae-}\mathcal{F}^{\text{CDH}}$ and a PPT adversarial machine $\mathcal{A}$ if there exists a polynomial $\text{poly}_{\mathcal{A}}(\cdot)$ such that

$$\Pr[\mathcal{A}(\mathbb{G}, g, N, \text{AE}, g^{x_1}, g^{x_2}) = g^{x_1 x_2}] > 1/\text{poly}_{\mathcal{A}}(\lambda)$$

for infinitely many values of λ , where $(\mathbb{G}, g, N, \text{AE}) \leftarrow f(1^\lambda)$ and x_1, x_2 are chosen uniformly from $\mathbb{Z}_N$.

We say that a CDH-secure group with nice admissible encoding $\text{ae-}\mathcal{P}^{\text{CDH}}$ exists if there is a function $f \in \text{ae-}\mathcal{F}^{\text{CDH}}$ such that no PPT adversarial machine $\mathcal{A}$ satisfies $\langle f, \mathcal{A} \rangle \in \text{ae-}\mathcal{R}^{\text{CDH}}$.

2.2 Dense Groups

We also consider cryptographic groups with dense encodings and formalize this primitive following the definition style in [RTV04].

Definition 2.4 (CDH-Secure Dense Group). *The CDH-secure dense group, denoted $\mathbf{d}\text{-}\mathcal{P}^{\text{CDH}}$, is defined as a pair $\langle \mathbf{d}\text{-}\mathcal{F}^{\text{CDH}}, \mathbf{d}\text{-}\mathcal{R}^{\text{CDH}} \rangle$, satisfying the following conditions:*

1. *The set $\mathbf{d}\text{-}\mathcal{F}^{\text{CDH}}$ consists of group-generation functions f that, on input of a security parameter λ , output the description of a finite cyclic group with a dense encoding. Specifically, we write $(\mathbb{G}, g, N, m) \leftarrow f(1^\lambda)$, where $\mathbb{G}$ is a cyclic group of λ -bit prime order N , g is a generator of $\mathbb{G}$, and m is the length of the group encoding. Here, m satisfies $m - \log N \leq \Theta(\log \lambda)$, meaning that each group element in $\mathbb{G}$ can be represented as an m -bit string.*
2. *The pair $\langle f, \mathcal{A} \rangle$ belongs to $\mathbf{d}\text{-}\mathcal{R}^{\text{CDH}}$ for a function $f \in \mathbf{d}\text{-}\mathcal{F}^{\text{CDH}}$ and a PPT adversarial machine $\mathcal{A}$ if there exists a polynomial $\text{poly}_{\mathcal{A}}(\cdot)$ such that*

$$\Pr[\mathcal{A}(\mathbb{G}, g, N, m, g^{x_1}, g^{x_2}) = g^{x_1 x_2}] > 1/\text{poly}_{\mathcal{A}}(\lambda)$$

for infinitely many values of λ , where $(\mathbb{G}, g, N, m) \leftarrow f(1^\lambda)$ and x_1, x_2 are chosen uniformly from $\mathbb{Z}_N$.

We say that a CDH-secure dense group $\mathbf{d}\text{-}\mathcal{P}^{\text{CDH}}$ exists if there is a function $f \in \mathbf{d}\text{-}\mathcal{F}^{\text{CDH}}$ such that no PPT adversarial machine $\mathcal{A}$ satisfies $\langle f, \mathcal{A} \rangle \in \mathbf{d}\text{-}\mathcal{R}^{\text{CDH}}$.

2.3 Idealized Models

In this subsection, we introduce the idealized models discussed in this paper, including the Generic Group Model (GGM) [Sho97], the Generic Bilinear Group Model (GBM) [BB04], and the Elliptic Curve Generic Group Model (EC-GGM) [GS22]. In each model, all entities, including the adversary and the challenger, have access to the corresponding oracle. Below, we specify the behavior of the oracle in each idealized model.

Definition 2.5 (Generic Group Model [Sho97]). *For a given security parameter $\lambda \in \mathbb{N}$, let $\mathcal{I}_{\mathbb{Z}_N, S}$ denote the set of all injections from $\mathbb{Z}_N$ to S , where N is a λ -bit prime number and $S = \{0, 1\}^m$. The generic group model $\mathcal{G}_{N, m}$ is an idealized model that samples a random injection σ from $\mathcal{I}_{\mathbb{Z}_N, S}$ and provides two oracles, $\mathcal{G}_{N, m}^{\text{label}}$ and $\mathcal{G}_{N, m}^{\text{add}}$. Concretely,*

- *The labeling oracle $\mathcal{G}_{N, m}^{\text{label}}$ takes as input $x \in \mathbb{Z}_N$ and returns $\sigma(x)$;*
- *The addition oracle $\mathcal{G}_{N, m}^{\text{add}}$ takes as input strings $\text{str}, \text{str}' \in S$ and behaves as follows: if there exist $x, x' \in \mathbb{Z}_N$ such that $\sigma(x) = \text{str}$ and $\sigma(x') = \text{str}'$, the oracle returns $\sigma(x + x')$; otherwise, it returns $\perp$.*

In this work, we further say that the generic group model $\mathcal{G}_{N, m}$ is *dense* if $m - \log N \leq \Theta(\log \lambda)$. Otherwise, we refer to it as *sparse*.

Definition 2.6 (Generic Bilinear Group Model [BB04]). *For a given security parameter $\lambda \in \mathbb{N}$, let $\mathcal{I}_{\mathbb{Z}_N, S_i}$ denote the set of all injections from $\mathbb{Z}_N$ to S_i , where N is a λ -bit prime number and $S_i = \{0, 1\}^{m_i}$ for $i \in \{1, 2, \mathbb{T}\}$. The generic bilinear group model $\mathcal{B}$ is an idealized model that samples random injections σ_1, σ_2 , and $\sigma_{\mathbb{T}}$ from $\mathcal{I}_{\mathbb{Z}_N, S_i}$, respectively, and provides seven oracles: $\mathcal{B}^{1\text{-label}}, \mathcal{B}^{1\text{-add}}, \mathcal{B}^{2\text{-label}}, \mathcal{B}^{2\text{-add}}, \mathcal{B}^{\mathbb{T}\text{-label}}, \mathcal{B}^{\mathbb{T}\text{-add}}$ and $\mathcal{B}^{\text{map}}$. Concretely,*

- *The labeling oracle $\mathcal{B}^{i\text{-label}}$, for $i \in \{1, 2, \mathbb{T}\}$, takes as input $x \in \mathbb{Z}_N$ and returns $\sigma_i(x)$;*
- *The addition oracle $\mathcal{B}^{i\text{-add}}$, for $i \in \{1, 2, \mathbb{T}\}$, takes as input strings $\text{str}, \text{str}' \in S_i$, and behaves as follows: if there exist $x, x' \in \mathbb{Z}_N$ such that $\sigma_i(x) = \text{str}$ and $\sigma_i(x') = \text{str}'$, the oracle returns $\sigma_{\mathbb{T}}(x + x')$; otherwise, it returns $\perp$.*

- The bilinear map oracle $\mathcal{B}^{\text{map}}$ takes as input strings $\text{str}_1 \in S_1$ and $\text{str}_2 \in S_2$, and behaves as follows: if there exist $x_1, x_2 \in \mathbb{Z}_N$ such that $\sigma_1(x_1) = \text{str}_1$ and $\sigma_2(x_2) = \text{str}_2$, the oracle returns $\sigma_T(x_1 \cdot x_2)$; otherwise, it returns $\perp$.

In this work, we consider only symmetric generic bilinear group models, meaning that $\mathcal{B}^{1\text{-label}} = \mathcal{B}^{2\text{-label}}$ and $\mathcal{B}^{1\text{-add}} = \mathcal{B}^{2\text{-add}}$. We make the functions N , m_S and m_T explicit⁸, and refer to the model as

$$\mathcal{B}_{N,m_S,m_T} = (\mathcal{B}_{N,m_S,m_T}^{S\text{-label}}, \mathcal{B}_{N,m_S,m_T}^{S\text{-add}}, \mathcal{B}_{N,m_S,m_T}^{T\text{-label}}, \mathcal{B}_{N,m_S,m_T}^{T\text{-add}}, \mathcal{B}_{N,m_S,m_T}^{\text{map}}).$$

We say that the model $\mathcal{B}_{N,m_S,m_T}$ is sparse if both $m_S - \log N \geq \omega(\log \lambda)$ and $m_T - \log N \geq \omega(\log \lambda)$.

Definition 2.7 (Elliptic Curve Generic Group Model [GS22]). Let $\lambda \in \mathbb{N}$ be the security parameter. Let E be the set of all points on the elliptic curve $y^2 = F(u)$ over $\mathbb{Z}_p$, where $|E| = N$ and N is a λ -bit prime number⁹. For any point $P = (u, b) \in \mathbb{Z}_p \times \{0, 1\}$ in E , we denote by $-P$ the point $(u, 1 - b) \in E$. Let $\mathcal{I}_{\mathbb{Z}_N, E}$ be the set of all injections σ from $\mathbb{Z}_N$ to E such that:

1. $\sigma(0) = \mathcal{O}$, where $\mathcal{O}$ is the point at infinity.
2. For all $x \in \mathbb{Z}_N$, it holds that $\sigma(-x) = -\sigma(x)$.

The elliptic curve generic group model $\text{ec-}\mathcal{G}_N$ is an idealized model that samples a random injection σ from $\mathcal{I}_{\mathbb{Z}_N, E}$ and provides two oracles, $\text{ec-}\mathcal{G}_N^{\text{label}}$ and $\text{ec-}\mathcal{G}_N^{\text{add}}$. Concretely,

- The labeling oracle $\text{ec-}\mathcal{G}_N^{\text{label}}$ takes as input $x \in \mathbb{Z}_N$ and returns $\sigma(x)$;
- The addition oracle $\text{ec-}\mathcal{G}_N^{\text{add}}$ takes as input two points $P, P' \in E$, where $\sigma(x) = P$ and $\sigma(x') = P'$, and returns $\sigma(x + x')$.

2.4 Indifferentiability

The framework of indifferentiability is proposed by Maurer, Renner, and Holenstein [MRH04], which formalizes a set of necessary and sufficient conditions for securely replacing one cryptosystem with another in an arbitrary environment. This framework is used to justify the structural soundness of various cryptographic primitives, including hash functions [CDMP05, DRS09], block ciphers [ABD⁺13, CHK⁺16, DSSL16, GWL23], domain extenders [CDMS10], authenticated encryption with associated data [BF18], and public-key cryptosystems [ZZ20]. It can also be used to study the relationship between idealized models [ZZ23]. In the following, we proceed to the definition of indifferentiability:

A cryptosystem Σ consists of a set of algorithms. Here, Σ is accessible via two interfaces $\Sigma.\text{hon}$ and $\Sigma.\text{adv}$, where $\Sigma.\text{hon}$ provides an honest interface through which the system can be accessed by all parties in a black-box manner, and $\Sigma.\text{adv}$ models the adversarial access to Σ .

Definition 2.8 (Indifferentiability [MRH04]). Let Σ_1 and Σ_2 be two cryptosystems and S be a simulator. The indifferentiability advantage of a differentiator $\mathcal{D}$ against (Σ_1, Σ_2) with respect to S is

$$\text{Adv}_{\Sigma_1, \Sigma_2, S, \mathcal{D}}^{\text{indif}}(1^\lambda) := \Pr[\text{Real}_{\Sigma_1, \mathcal{D}}] - \Pr[\text{Ideal}_{\Sigma_2, S, \mathcal{D}}],$$

where games $\text{Real}_{\Sigma_1, \mathcal{D}}$ and $\text{Ideal}_{\Sigma_2, S, \mathcal{D}}$ are defined in Fig. 1. We say Σ_1 is indifferentiable from Σ_2 , if there exists an efficient simulator S such that for any efficient differentiator $\mathcal{D}$, the advantage above is negligible. Moreover, we say Σ_1 is statistically indifferentiable from Σ_2 , if there exists an efficient simulator such that, for any unbounded differentiator $\mathcal{D}$, the advantage above is negligible.

⁸GBM is symmetric indicating that $m_1 = m_2$, and we redenote by m_S the encoding length of source group.

⁹For any $u \in \mathbb{Z}_p$ there are at most two points on the curve with u as their x-coordinate, namely, $(u, \pm y)$ for some y . To simplify, point compression is applied, and we denote the point (u, y) by the pair (u, b) , where $b = 0$ if y is even and $b = 1$ if y is odd.

Indifferentiability

$\text{Real}_{\Sigma_1, \mathcal{D}}:$ $b \leftarrow \mathcal{D}^{\text{HonestR}, \text{AdvR}}$ Return b .	$\text{HonestR}(X)$ Return $\Sigma_1.\text{hon}(X)$. $\text{AdvR}(X)$ Return $\Sigma_1.\text{adv}(X)$.
$\text{Ideal}_{\Sigma_2, \mathcal{S}, \mathcal{D}}:$ $b \leftarrow \mathcal{D}^{\text{HonestI}, \text{AdvI}}$ Return b .	$\text{HonestI}(X)$ Return $\Sigma_2.\text{hon}(X)$. $\text{AdvI}(X)$ Return $\mathcal{S}^{\Sigma_2.\text{adv}(\cdot)}(X)$.

Figure 1: Indifferentiability of Σ_1 and Σ_2 , where $\mathcal{S}$ is the simulator and $\mathcal{D}$ is the adversary.

Below, we also use the notations in [BF18] and consider the definition above to two systems with interfaces as:

$$\begin{aligned}
 (\Sigma_1.\text{hon}(X), \Sigma_1.\text{adv}(x)) &:= (\Pi^{\mathcal{F}_1}(X), \mathcal{F}_1(x)), \\
 (\Sigma_2.\text{hon}(X), \Sigma_2.\text{adv}(x)) &:= (\mathcal{F}_2(X), \mathcal{F}_2(x)),
 \end{aligned}$$

where $\mathcal{F}_1$ and $\mathcal{F}_2$ are two ideal objects sampled from their distributions and $\Pi^{\mathcal{F}_1}$ is a construction of $\mathcal{F}_2$ by calling $\mathcal{F}_1$. MRH prove the composition theorem for the framework of indifferentiability; for simplify, we give a game-based formalization from [RSS11].

Theorem 2.1 (Composition Theorem [MRH04]). *Let $\Sigma_1 := (\Pi^{\mathcal{F}_1}, \mathcal{F}_1)$ and $\Sigma_2 := (\mathcal{F}_2, \mathcal{F}_2)$ be two systems that Σ_1 is indifferentiable from Σ_2 with respect to a simulator $\mathcal{S}$, then Σ_1 is as secure as Σ_2 for any single-stage game. More concretely, let Game be a single-stage game, then for any adversary $\mathcal{A}$, there is an adversary $\mathcal{B}$ and a differentiator $\mathcal{D}$ such that*

$$\Pr[\text{Game}_{\Pi^{\mathcal{F}_1}, \mathcal{A}^{\mathcal{F}_1}}] \leq \Pr[\text{Game}_{\mathcal{F}_2, \mathcal{B}^{\mathcal{F}_2}}] + \text{Adv}_{\Sigma_1, \Sigma_2, \mathcal{S}, \mathcal{D}}^{\text{indif}}.$$

The proof of Thm. 2.1 is straightforward; due to space limit, we skip it here. Next, we give the formal definition of the separation between two idealized models in the framework of indifferentiability against computational adversaries.

Definition 2.9 (Computational Indifferentiable Separation [MRH04]). *Let Σ_1, Σ_2 be two idealized models, we say Σ_2 is computationally indifferentiable separated from Σ_1 if for any efficient algorithm Π and any efficient simulator $\mathcal{S}$, there exists an efficient differentiator $\mathcal{D}_{\Pi, \mathcal{S}}$ and a noticeable function ρ such that*

$$\text{Adv}_{\Pi^{\Sigma_1}, \Sigma_2, \mathcal{S}, \mathcal{D}_{\Pi, \mathcal{S}}}^{\text{indif}}(1^\lambda) := \left| \Pr[\text{Real}_{\Sigma_1, \mathcal{D}_{\Pi, \mathcal{S}}}] - \Pr[\text{Ideal}_{\Sigma_2, \mathcal{S}, \mathcal{D}_{\Pi, \mathcal{S}}}] \right| \geq \rho(\lambda).$$

Observe that, if an idealized model Σ_2 is computationally indifferentiable separated from another idealized model Σ_1 , it means that, we cannot build a scheme Π^{Σ_1} such that Π^{Σ_1} is indifferentiable from Σ_2 , even under arbitrarily strong computational assumptions.

3 Sparse GGM vs. groups with nice admissible encodings

In this section, we elaborate the main constraint of the sparse GGM. We demonstrate that any the cryptographic group with nice admissible encodings that is CDH-secure, denoted as $\text{ae-}\mathcal{P}^{\text{CDH}}$, cannot exist within the sparse GGM. Roughly speaking, to establish the black-box separation between $\text{ae-}\mathcal{P}^{\text{CDH}}$ and the sparse GGM, the standard approach is to build an adversary that, while computationally unbounded, is query-efficient and capable of breaking the CDH game with respect to any construction of $\text{ae-}\mathcal{P}^{\text{CDH}}$ in the sparse GGM.

For easier readability, our strategy follows the one in [IR89]. We introduces an intermediary primitive, i.e. non-interactive key exchange (NIKE) with nice admissible encoding that is secure against key-recovery attack (KRA-secure), denoted as $\text{ae-}\mathcal{P}^{\text{NIKE}}$, and prove that:

- $\text{ae-}\mathcal{P}^{\text{CDH}}$ implies $\text{ae-}\mathcal{P}^{\text{NIKE}}$;
- $\text{ae-}\mathcal{P}^{\text{NIKE}}$ does not exist in the sparse GGM.

3.1 Non-interactive key exchange with nice AE

In this section, we present the formal definition of the KRA-secure non-interactive key exchange with nice admissible encoding.

Definition 3.1 (NIKE with Nice AE). *A KRA-secure non-interactive key exchange protocol with nice admissible encoding, denoted $\text{ae-}\mathcal{P}^{\text{NIKE}}$, is defined as a pair $\langle \text{ae-}\mathcal{F}^{\text{NIKE}}, \text{ae-}\mathcal{R}^{\text{NIKE}} \rangle$:*

1. *The set $\text{ae-}\mathcal{F}^{\text{NIKE}}$ consists of functions f , each associated with a constant ϱ and a polynomial poly_{AE} in λ . For a given input security parameter, f outputs the description of a non-interactive key exchange protocol along with an additional function. Specifically, we write $(\text{KGen}, \text{SHK}, \text{AE}) \leftarrow f(1^\lambda)$, where algorithms are associated with $\mathcal{SK}, \mathcal{PK}$, and $\mathcal{K}$.*
 - *$\mathcal{SK}, \mathcal{PK}$, and $\mathcal{K}$ are the private-key space, public-key space and shared-key space, respectively, satisfying that $\mathcal{SK} := \mathbb{Z}_N$, where N is a λ -bit integer.*
 - *The public-key generation function $\text{KGen} : \mathcal{SK} \mapsto \mathcal{PK}$ is an injection, for generating a public key $\text{pk} \in \mathcal{PK}$ from a randomly chosen private key $\text{sk} \in \mathcal{SK}$.*
 - *The shared-key generation function $\text{SHK} : \mathcal{PK} \times \mathcal{SK} \mapsto \mathcal{K} \cup \{\perp\}$ for generating a shared key $\text{shk} \in \mathcal{K} \cup \{\perp\}$, where $\perp$ indicates a failed computation.*
 - *The encoding function $\text{AE} : \mathbb{Z}_p \mapsto \mathcal{PK}$ is a ϱ -regular admissible encoding from $\mathbb{Z}_p$ to the codomain of KGen , with $p \geq N/\text{poly}_{\text{AE}}(\lambda)$.*

Concretely, for randomly chosen $\text{sk} \xleftarrow{\$} \mathcal{SK}$ and $\text{sk}' \xleftarrow{\$} \mathcal{SK}$, compute $\text{pk} \leftarrow \text{KGen}(\text{sk})$ and $\text{pk}' \leftarrow \text{KGen}(\text{sk}')$. We write $\text{shk} \leftarrow \text{SHK}(\text{pk}', \text{sk})$ and $\text{shk}' \leftarrow \text{SHK}(\text{pk}, \text{sk}')$. The protocol is required to achieve perfect correctness, meaning that:

$$\Pr[\text{shk} = \perp \vee \text{shk}' = \perp \vee \text{shk} \neq \text{shk}'] = 0.$$

2. *For a function $f = (\text{KGen}, \text{SHK}, \text{AE}) \in \text{ae-}\mathcal{F}^{\text{NIKE}}$ and a PPT (adversarial) machine $\mathcal{A}$, we define $\langle f, \mathcal{A} \rangle \in \text{ae-}\mathcal{R}^{\text{NIKE}}$ if $\mathcal{A}$ can break the security property of f against key-recovery attack (KRA). Specifically, there exists a polynomial $\text{poly}_{\mathcal{A}}(\cdot)$ such that:*

$$\Pr[\mathcal{A}(\text{pk}, \text{pk}') = \text{SHK}(\text{pk}', \text{sk}) = \text{SHK}(\text{pk}, \text{sk}') \neq \perp] > 1/\text{poly}_{\mathcal{A}}(\lambda)$$

for infinitely many values of λ . Here, for randomly chosen $\text{sk} \xleftarrow{\$} \mathcal{SK}$ and $\text{sk}' \xleftarrow{\$} \mathcal{SK}$, compute $\text{pk} \leftarrow \text{KGen}(\text{sk})$ and $\text{pk}' \leftarrow \text{KGen}(\text{sk}')$, respectively.

We say $\text{ae-}\mathcal{P}^{\text{NIKE}}$ exists if there is a function $f \in \text{ae-}\mathcal{F}^{\text{NIKE}}$ such that no PPT adversarial machine $\mathcal{A}$ satisfies $\langle f, \mathcal{A} \rangle \in \text{ae-}\mathcal{R}^{\text{NIKE}}$.

It is apparent that $\text{ae-}\mathcal{P}^{\text{CDH}}$ implies $\text{ae-}\mathcal{P}^{\text{NIKE}}$ by defining $\text{KGen}(x) := g^x$ and $\text{SHK}(\text{pk}, \text{sk}') := \text{pk}^{\text{sk}'}$. Therefore, it is sufficient to prove that $\text{ae-}\mathcal{P}^{\text{NIKE}}$ does not exist in the sparse GGM.

3.2 $\text{ae-}\mathcal{P}^{\text{NIKE}}$ does not exist in the sparse GGM

In this section, we establish the black-box separation between $\text{ae-}\mathcal{P}^{\text{NIKE}}$ and the sparse GGM. Specifically, we construct an adversary that, while computationally unbounded, is query-efficient and capable of breaking the KRA-secure game with a noticeable probability.

Theorem 3.1. *Let $\mathcal{G}_{N,m}$ be a generic group model such that $m - \log N \geq \omega(\log \lambda)$. Let $\Pi^{\mathcal{G}_{N,m}} = (\text{KGen}^{\mathcal{G}_{N,m}}, \text{SHK}^{\mathcal{G}_{N,m}}, \text{AE}^{\mathcal{G}_{N,m}})$ be any non-interactive key exchange protocol with nice admissible encoding parameterized by a constant ϱ and a polynomial poly_{AE} . Then there exists an adversary $\mathcal{A}$ and two polynomials poly_1 and poly_2 such that:*

- *$\mathcal{A}$ makes poly_1 queries to $\mathcal{G}_{N,m}$;*

- $\mathcal{A}$ breaks the KRA-secure game with advantage $\frac{1}{\text{poly}_2}$.

Proof. To establish the proof, we present a formal description of the adversary $\mathcal{A}$, as illustrated in Figure 2. Here, we specify that any algorithm in $\Pi^{\mathcal{G}_{N,m}}$ makes at most q queries, where q is a polynomial. We then clarify some undefined notions: By $\{(que_1, res_1), \dots, (que_q, res_q)\} \xrightarrow{\text{query}} \text{KGen}^{\mathcal{G}_{N,m}}(x)$, we mean that when running $\text{KGen}^{\mathcal{G}_{N,m}}(x)$, the algorithm makes queries $(que_1, \dots, que_q)$ to the oracle $\mathcal{G}_{N,m}$, and obtains $(res_1, \dots, res_q)$ ¹⁰.

Trivial to note that the adversary $\mathcal{A}^{\mathcal{G}_{N,m}}$ is query-efficient. We next prove that the adversary $\mathcal{A}^{\mathcal{G}_{N,m}}$ can successfully guess the valid shared key with a noticeable probability. Let $S_{\text{Bob-L}}$ denote the set of all valid group encodings that appear when running the algorithms $\text{KGen}^{\mathcal{G}_{N,m}}(sk_2)$ and $\text{SHK}^{\mathcal{G}_{N,m}}(pk_1, sk_2)$; these encodings are either the responses of labeling/addition queries or the valid inputs of the addition queries. Clearly, $|S_{\text{Bob-L}}| \leq 6q$, since each algorithm makes at most q queries to $\mathcal{G}_{N,m}$. We then define:

$$S_{\text{Bob}} := \{(x, \text{str}) \mid \text{str} \in S_{\text{Bob-L}}, \mathcal{G}_{N,m}^{\text{label}}(x) = \text{str}\}.$$

Note that in each iteration, if the encoding pairs in $\tilde{S}_{\mathcal{A}}$ are consistent with S_{Bob} , the shared key computed in that iteration would be correct. Specifically, in such a context, there exists another instance of GGM that is consistent with both the simulation view of $\mathcal{A}$ and the real view of user Bob. The perfect correctness of $\text{ae-}\mathcal{P}^{\text{NIKE}}$ guarantees that the shared key computed in this iteration must be equal to the true key computed by Bob. However, without knowledge of sk_2 , $\tilde{S}_{\mathcal{A}}$ might be inconsistent with S_{Bob} with high probability. In fact, there are three events:

- Event 1: There exist $(x, \text{str}) \in \tilde{S}_{\mathcal{A}}$ and $(x', \text{str}') \in S_{\text{Bob}}$ such that $x = x'$ but $\text{str} \neq \text{str}'$.
- Event 2: There exist $(x, \text{str}) \in \tilde{S}_{\mathcal{A}}$ and $(x', \text{str}') \in S_{\text{Bob}}$ such that $x \neq x'$ but $\text{str} = \text{str}'$.
- Event 3: For any $(x, \text{str}) \in \tilde{S}_{\mathcal{A}}$ and $(x', \text{str}') \in S_{\text{Bob}}$, we have that if $x = x'$ then $\text{str} = \text{str}'$, and vice versa.

We immediately observe that Event 1 occurs at most $6q$ times, since the updating phase would eliminate at least one pair in S_{Bob} with each occurrence. Next we show that, with a noticeable probability, Event 3 would deduce the valid shared key. According to the adversary, depicted in Figure 2, we note that the set $\tilde{S}_{\mathcal{A}} \cup S_{\text{que-res}}$ responds to the labeling queries properly. For the addition queries, say $\text{que} = (\text{str}_1, \text{str}_2)$, there are two cases:

- Case 1: $\tilde{S}_{\mathcal{A}} \cup S_{\text{que-res}}$ covers (x_1, str_1) , (x_2, str_2) , and $(x_1 + x_2, \text{str}_3)$;
- Case 2: either str_1 or str_2 is not collected in $\tilde{S}_{\mathcal{A}} \cup S_{\text{que-res}}$.

For the former case, $\tilde{S}_{\mathcal{A}} \cup S_{\text{que-res}}$ responds to the addition query properly; for the latter case, $\tilde{S}_{\mathcal{A}} \cup S_{\text{que-res}}$ always responds with $\perp$, which means that if both str_1 and str_2 are valid group elements then the response is invalid. Therefore, the only bad case that prevents Event 3 from guessing a valid shared key is that the adversary $\mathcal{A}$ generates valid group elements str without making labeling queries (i.e., without knowing the corresponding discrete logarithm). Besides, we observe that, when Event 2 occurs, the adversary also generates valid group elements without making labeling queries. Hence, if the probability of such a bad case is small, then the adversary $\mathcal{A}^{\mathcal{G}_{N,m}}$ can break the KRA-secure game and output the valid shared key with a good probability.

According to the description of the adversary $\mathcal{A}^{\mathcal{G}_{N,m}}$, we immediately observe that, $\mathcal{A}^{\mathcal{G}_{N,m}}$ aborts if either pk_1 or pk_2 has no preimage with respect to $\text{AE}^{\mathcal{G}_{N,m}}$. To analyze the probability of such a bad event, we define a tuple $(\mathcal{G}_{N,m}, sk_1, sk_2)$ is invertible if the following conditions are satisfied:

1. $\text{inv-AE}^{\mathcal{G}_{N,m}}(\text{KGen}^{\mathcal{G}_{N,m}}(sk_1)) \neq \emptyset$;

¹⁰As explained above, we assume that the algorithms $\text{KGen}^{\mathcal{G}_{N,m}}$ and $\text{AE}^{\mathcal{G}_{N,m}}$ only make labeling queries.

Adversary $\mathcal{A}^{\mathcal{G}_{N,m}}$

$\mathcal{A}^{\mathcal{G}_{N,m}}(\text{pk}_1, \text{pk}_2)$:

$S_{\text{key}} \leftarrow \emptyset; S_{\text{que-res}} \leftarrow \emptyset;$

if $\text{inv-AE}^{\mathcal{G}_{N,m}}(\text{pk}_1) = \emptyset$ or $\text{inv-AE}^{\mathcal{G}_{N,m}}(\text{pk}_2) = \emptyset$: return $\perp$;

Preprocessing phase: //collect queries from admissible encoding

for every seed $\in \text{inv-AE}^{\mathcal{G}_{N,m}}(\text{pk}_1) \cup \text{inv-AE}^{\mathcal{G}_{N,m}}(\text{pk}_2)$:

$\{(\text{que}_1, \text{res}_1), \dots, (\text{que}_q, \text{res}_q)\} \xleftarrow{\text{query}} \text{AE}^{\mathcal{G}_{N,m}}(\text{seed});$ //queries from AE

$S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup \{(\text{que}_1, \text{res}_1), \dots, (\text{que}_q, \text{res}_q)\};$

for $i \in \{1, 2\}$: //queries from AE-inverse

$\{(\text{que}_1, \text{res}_1), \dots, (\text{que}_q, \text{res}_q)\} \xleftarrow{\text{query}} \text{inv-AE}^{\mathcal{G}_{N,m}}(\text{pk}_i);$

for $j = 1$ to q :

if que_j is a labeling query: $S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup \{(\text{que}_j, \text{res}_j)\};$

if $\text{que}_j = (\text{str}_1, \text{str}_2)$ is an addition query:

if $\exists (x_1, \text{str}_1), (x_2, \text{str}_2) \in S_{\text{que-res}}: S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup \{(x_1 + x_2, \text{res}_j)\};$

Iteration phase:

for $i = 1$ to $12q + 1$:

Simulation: search a proper view of Alice

search a private key $\tilde{\text{sk}}_1$ and a set of encoding pairs $\tilde{S}_{\mathcal{A}}$ satisfying the following properties:

1. $\tilde{S}_{\mathcal{A}}$ is consistent with $S_{\text{que-res}}$ and $|\tilde{S}_{\mathcal{A}}| \leq 12q$;
2. $\text{pk}_1 \leftarrow \text{KGen}^{S_{\text{que-res}} \cup \tilde{S}_{\mathcal{A}}}(\tilde{\text{sk}}_1)$;
3. $\tilde{S}_{\mathcal{A}}$ is sufficient for the algorithm $\tilde{\text{shk}}_i \leftarrow \text{SHK}^{S_{\text{que-res}} \cup \tilde{S}_{\mathcal{A}}}(\text{pk}_2, \tilde{\text{sk}}_1)$:
 - if $\text{que} = x$ is a labeling query, then x should be covered in $S_{\text{que-res}} \cup \tilde{S}_{\mathcal{A}}$;
 - if $\text{que} = (\text{str}_1, \text{str}_2)$ is an addition query and there are $(x_1, \text{str}_1), (x_2, \text{str}_2)$ in $S_{\text{que-res}} \cup \tilde{S}_{\mathcal{A}}$, then $(x_1 + x_2, \text{str}_3)$ should also be covered in $S_{\text{que-res}} \cup \tilde{S}_{\mathcal{A}}$;
 - if $\text{que} = (\text{str}_1, \text{str}_2)$ is an addition query but there is no (x_1, str_1) and/or (x_2, str_2) in $S_{\text{que-res}} \cup \tilde{S}_{\mathcal{A}}$, then respond with $\perp$.

$S_{\text{key}} \leftarrow S_{\text{key}} \cup \{\tilde{\text{shk}}_i\};$

Updating: replace the guessing labeling queries with valid encodings

for each $(x, \text{str}) \in \tilde{S}_{\mathcal{A}} \setminus S_{\text{que-res}}: S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup (x, \mathcal{G}_{N,m}^{\text{label}}(x));$

Final phase: //output the guessing shared key

return the majority value in S_{key} .

Figure 2: The description of $\mathcal{A}$ that breaks the KRA-secure game of $\text{ae-}\mathcal{P}^{\text{NIKE}}$ in $\mathcal{G}_{N,m}$.

2. $\text{inv-AE}^{\mathcal{G}_{N,m}}(\text{KGen}^{\mathcal{G}_{N,m}}(\text{sk}_2)) \neq \emptyset$.

Due to the fact that $\text{AE}^{\mathcal{G}_{N,m}}$ is a ϱ -regular admissible encoding from $\mathbb{Z}_p$ to the N valid public keys, there are at least $\left\lceil \frac{p}{\varrho} \right\rceil$ public keys with preimages under $\text{AE}^{\mathcal{G}_{N,m}}$. Each such public key corresponds to a unique private key. Therefore, we have that

$$\begin{aligned} \Pr[(\mathcal{G}_{N,m}, \text{sk}_1, \text{sk}_2) \text{ is invertible}] &\geq \frac{1}{\varrho^2} \cdot \left(\frac{p}{N}\right)^2 \\ &\geq \frac{1}{\varrho^2} \cdot \frac{1}{\text{poly}_{\text{AE}}(\lambda)^2}, \end{aligned}$$

where the probability is over the sampling of the instance of the GGM and private keys.

Moreover, we observe that once the randomness of the adversary is fixed, then the algorithm $\mathcal{A}^{\mathcal{G}_{N,m}}(\text{pk}_1, \text{pk}_2)$ is deterministic. Let r be a nonce, we say the adversary performs good with respect to the tuple $(\mathcal{G}_{N,m}, \text{sk}_1, \text{sk}_2, r)$, if the adversary $\mathcal{A}^{\mathcal{G}_{N,m}}(\text{pk}_1, \text{pk}_2)$, utilizing r as its source of randomness, is unable to generate a valid group element without knowing the discrete logarithm. Here, pk_i represents the public key generated from the private key sk_i .

Next, we introduce the concept of “good” for the tuple $(\mathcal{G}_{N,m}, \text{sk}_1, \text{sk}_2)$. More formally, we say a tuple $(\mathcal{G}_{N,m}, \text{sk}_1, \text{sk}_2)$ is good if

$$\Pr[\mathcal{A} \text{ performs good with respect to } (\mathcal{G}_{N,m}, \text{sk}_1, \text{sk}_2, r)] \geq \frac{1}{2},$$

where $\text{pk}_1 \leftarrow \text{KGen}^{\mathcal{G}_{N,m}}(\text{sk}_1)$ and $\text{pk}_2 \leftarrow \text{KGen}^{\mathcal{G}_{N,m}}(\text{sk}_2)$, and the probability is over the internal randomness of $\mathcal{A}$. Analogously, we say a tuple $(\mathcal{G}_{N,m}, \text{sk}_1, \text{sk}_2)$ is bad if

$$\Pr[\mathcal{A} \text{ performs good with respect to } (\mathcal{G}_{N,m}, \text{sk}_1, \text{sk}_2, r)] < \frac{1}{2}.$$

For clarity, we denote these three events in which the tuple $(\mathcal{G}_{N,m}, \text{sk}_1, \text{sk}_2)$ is invertible, good or bad as Invertible, Good and Bad, respectively. Due to the perfect correctness of $\Pi^{\mathcal{G}_{N,m}}$, it is apparent that the probability $\mathcal{A}$ wins the KRA-secure game, conditioned on $\text{Good} \wedge \text{Invertible}$, is $\geq \frac{1}{2}$. Thus, the probability of $\mathcal{A}$ wins can be bounded by

$$\begin{aligned} \Pr[\mathcal{A} \text{ wins}] &\geq \Pr[\mathcal{A}^{\mathcal{G}_{N,m}} \text{ wins} | \text{Good} \wedge \text{Invertible}] \cdot \Pr[\text{Good} \wedge \text{Invertible}] \\ &\geq \frac{1}{2} \cdot \Pr[\text{Good} \wedge \text{Invertible}] = \frac{1}{2} \cdot \Pr[\text{Invertible}] \cdot \Pr[\text{Good} | \text{Invertible}] \\ &\geq \frac{1}{2\varrho^2} \cdot \frac{1}{\text{poly}_{\text{AE}}(\lambda)^2} \cdot \Pr[\text{Good} | \text{Invertible}]. \end{aligned}$$

Therefore, it suffices to prove that $\Pr[\text{Good} | \text{Invertible}]$ is noticeable. To finalize the proof, we utilize the sparsity property of the GGM. As discussed in [Zha22], given access to a sparse GGM, any algorithm that receives only the security parameter as input cannot output a valid group element without knowledge of the corresponding discrete logarithm except with negligible probability¹¹. For ease of exposition, we refer to this event as the algorithm outputs a new group element. More precisely, we say an algorithm $\mathcal{E}$ outputs a new group element str , if (1) str has been used as an input to some addition queries made by $\mathcal{E}$, and (2) str has not appeared as the output of any previous queries (whether labeling or addition) made by $\mathcal{E}$.

We note that, if $\Pr[\text{Good} | \text{Invertible}]$ is small, then, conditioned on Invertible, the adversary $\mathcal{A}$ is likely to generate a new group element with a good probability. Consequently, our proof strategy proceeds in two steps:

¹¹We establish the concrete upper bound for this event in Lemma G.1; due to space limit, we put the proof in Appendix G.1.

Extractor $\mathcal{E}^{\mathcal{G}_{N,m}}(1^\lambda)$

$\text{seed}_1, \text{seed}_2 \xleftarrow{\$} \mathbb{Z}_p; \text{pk}_1 \leftarrow \text{AE}^{\mathcal{G}_{N,m}}(\text{seed}_1); \text{pk}_2 \leftarrow \text{AE}^{\mathcal{G}_{N,m}}(\text{seed}_2); S_{\text{que-res}} \leftarrow \emptyset;$

Preprocessing phase: //collect queries from admissible encoding

for every $\text{seed} \in \text{inv-AE}^{\mathcal{G}_{N,m}}(\text{pk}_1) \cup \text{inv-AE}^{\mathcal{G}_{N,m}}(\text{pk}_2)$:

$\{(\text{que}_1, \text{res}_1), \dots, (\text{que}_q, \text{res}_q)\} \xleftarrow{\text{query}} \text{AE}^{\mathcal{G}_{N,m}}(\text{seed});$ //queries from AE

$S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup \{(\text{que}_1, \text{res}_1), \dots, (\text{que}_q, \text{res}_q)\};$

for $i \in \{1, 2\}$: //queries from AE-inverse

$\{(\text{que}_1, \text{res}_1), \dots, (\text{que}_q, \text{res}_q)\} \xleftarrow{\text{query}} \text{inv-AE}^{\mathcal{G}_{N,m}}(\text{pk}_i);$

for $j = 1$ to q :

if que_j is a labeling query: $S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup \{(\text{que}_j, \text{res}_j)\};$

if $\text{que}_j = (\text{str}_1, \text{str}_2)$ is an addition query:

if $\exists (x_1, \text{str}_1), (x_2, \text{str}_2) \in S_{\text{que-res}}: S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup \{(x_1 + x_2, \text{res}_j)\};$

if $\text{str}_1 \notin S_{\text{que-res}}$ and $\mathcal{G}_{N,m}^{\text{add}}(\text{str}_1, \text{str}_1) \neq \perp$, return str_1 ;

if $\text{str}_2 \notin S_{\text{que-res}}$ and $\mathcal{G}_{N,m}^{\text{add}}(\text{str}_2, \text{str}_2) \neq \perp$, return str_2 ;

Iteration phase:

for $i = 1$ to $12q + 1$:

Simulation: search a proper view of Alice

search a private key sk_1 and a set of encoding pairs $\tilde{S}_{\mathcal{A}}$ satisfying the following properties:

1. $\tilde{S}_{\mathcal{A}}$ is consistent with $S_{\text{que-res}}$ and $|\tilde{S}_{\mathcal{A}}| \leq 12q$;

2. $\text{pk}_1 \leftarrow \text{KGen}^{S_{\text{que-res}} \cup \tilde{S}_{\mathcal{A}}}(\text{sk}_1)$;

3. $\tilde{S}_{\mathcal{A}}$ is sufficient for the algorithm $\widetilde{\text{shk}}_i \leftarrow \text{SHK}^{S_{\text{que-res}} \cup \tilde{S}_{\mathcal{A}}}(\text{pk}_2, \text{sk}_1)$:

- if $\text{que} = x$ is a labeling query, then x should be covered in $S_{\text{que-res}} \cup \tilde{S}_{\mathcal{A}}$;

- if $\text{que} = (\text{str}_1, \text{str}_2)$ is an addition query and there are $(x_1, \text{str}_1), (x_2, \text{str}_2)$ in $S_{\text{que-res}} \cup \tilde{S}_{\mathcal{A}}$, then $(x_1 + x_2, \text{str}_3)$ should also be covered in $S_{\text{que-res}} \cup \tilde{S}_{\mathcal{A}}$;

- if $\text{que} = (\text{str}_1, \text{str}_2)$ is an addition query but there is no (x_1, str_1) and/or (x_2, str_2) in $S_{\text{que-res}} \cup \tilde{S}_{\mathcal{A}}$, then respond with $\perp$.

For each $(x, \text{str}) \in \tilde{S}_{\mathcal{A}} \setminus S_{\text{que-res}}$: if $\mathcal{G}_{N,m}^{\text{add}}(\text{str}, \text{str}) \neq \perp$, return str ;

Re-run the algorithm $\widetilde{\text{shk}}_i \leftarrow \text{SHK}^{S_{\text{que-res}} \cup \tilde{S}_{\mathcal{A}}}(\text{pk}_2, \text{sk}_1)$ and $\text{que} = (\text{str}_1, \text{str}_2)$ is an addition query during the execution:

if $\text{str}_1 \notin S_{\text{que-res}}$ and $\mathcal{G}_{N,m}^{\text{add}}(\text{str}_1, \text{str}_1) \neq \perp$, return str_1 ;

if $\text{str}_2 \notin S_{\text{que-res}}$ and $\mathcal{G}_{N,m}^{\text{add}}(\text{str}_2, \text{str}_2) \neq \perp$, return str_2 ;

Updating: replace the guessing labeling queries with valid encodings

for each $(x, \text{str}) \in \tilde{S}_{\mathcal{A}} \setminus S_{\text{que-res}}$: $S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup (x, \mathcal{G}_{N,m}^{\text{label}}(x))$;

return $\perp$.

Figure 3: The description of $\mathcal{E}$ w.r.t. $\text{ae-}\mathcal{P}^{\text{NIKE}}$ that outputs a new group element of $\mathcal{G}_{N,m}$.

1. Construct an extractor $\mathcal{E}$ that takes only the security parameter as input;
2. Demonstrate that, if $\Pr[\text{Good}|\text{Invertible}]$ is small, then $\mathcal{E}$ outputs a new group element with a good probability.

We now present the formal description of the extractor $\mathcal{E}$, as depicted in Figure 3. The extractor $\mathcal{E}$ takes only the security parameter as input, randomly selects two seeds (i.e., seed_1 and seed_2), and computes the corresponding public keys (i.e., pk_1 and pk_2) using admissible encodings. Following this, the extractor proceeds in a manner closely resembling the adversary in the KRA-security game, as shown in Figure 2. Specifically, the extractor proceeds the preprocessing phase and the iteration phase. During the simulation process of each iteration phase, if a new and valid group element appears, either in $\tilde{S}_A$ or as an input of some addition queries, then $\mathcal{E}$ outputs this element (highlighted in red). Observe that the randomness of $\mathcal{E}$ is composed of three components: seed_1 , seed_2 , and r , where r represents the randomness used during the iteration phase. Furthermore, once the GGM instance and the two seeds are fixed, the extractor $\mathcal{E}$ proceeds identically to the adversary $\mathcal{A}$ in the KRA-security game, continuing until $\mathcal{E}$ successfully outputs a new and valid group element.

Next, we analyze the probability of the extractor wins, which occurs when $\mathcal{E}$ outputs a new and valid group element. Let $\mathcal{G}_{N,m}$ denote the GGM to which the extractor $\mathcal{E}$ has access. Let seed_1 and seed_2 denote the seeds sampled by the extractor. We define the seed pair, $(\text{seed}_1, \text{seed}_2)$, to be bad with respect to $\mathcal{G}_{N,m}$, if the tuple $(\mathcal{G}_{N,m}, \text{sk}_1, \text{sk}_2)$, deduced from $(\text{seed}_1, \text{seed}_2)$, is a Bad and Invertible tuple. More precisely, when we say a tuple $(\mathcal{G}_{N,m}, \text{sk}_1, \text{sk}_2)$ deduced from $(\text{seed}_1, \text{seed}_2)$ with respect to $\mathcal{G}_{N,m}$, we mean that

$$\text{KGen}^{\mathcal{G}_{N,m}}(\text{sk}_1) = \text{AE}^{\mathcal{G}_{N,m}}(\text{seed}_1) \text{ and } \text{KGen}^{\mathcal{G}_{N,m}}(\text{sk}_2) = \text{AE}^{\mathcal{G}_{N,m}}(\text{seed}_2).$$

For clarity, we refer to this event as BadSeed in the following analysis. According to the definition of Bad, we note that if $\mathcal{E}$ fortunately samples a bad seed pair, i.e., BadSeed occurs, then $\mathcal{E}$ wins with a good probability. Concretely,

$$\Pr[\mathcal{E}^{\mathcal{G}_{N,m}} \text{ wins} | \text{BadSeed}] \geq \frac{1}{2}.$$

Next, we analyze the probability of BadSeed with respect to any specific GGM instance. To facilitate the analysis, we introduce several concepts relevant to the generic group model. For any instance of the GGM, denoted as $\mathcal{G}_{N,m}$, we define $Q_{\mathcal{G}_{N,m}}$ as the set of all private keys such that for any private key $\text{sk} \in Q_{\mathcal{G}_{N,m}}$, the corresponding public key (denoted $\text{pk} = \text{KGen}^{\mathcal{G}_{N,m}}(\text{sk})$) has valid preimages under the admissible encodings. Moreover, due to that the admissible encodings is ϱ -regular, it is apparent that

$$\left\lceil \frac{p}{\varrho} \right\rceil \leq |Q_{\mathcal{G}_{N,m}}| \leq N.$$

We then categorize all instances of the GGM into two types: good GGM instances and bad GGM instances. Specifically, let $\mathcal{G}_{N,m}$ be a GGM instance, and $Q_{\mathcal{G}_{N,m}} := \{\text{sk}_1, \dots, \text{sk}_t\}$. Let (i, j) be a pair where $i, j \in [1, t]$ ¹². We classify $\mathcal{G}_{N,m}$ as a bad instance if there are more than $\frac{1}{2\varrho^2} \cdot t^2$ pairs such that each induces a Bad and Invertible tuple. More precisely, when we say a pair such as (i^*, j^*) induces a Bad and Invertible tuple, we mean that the tuple $(\mathcal{G}_{N,m}, \text{sk}_{i^*}, \text{sk}_{j^*})$ is bad and invertible. Analogously, we classify $\mathcal{G}_{N,m}$ as a good instance if there are at most $\frac{1}{2\varrho^2} \cdot t^2$ pairs, each of which induces a Bad and Invertible tuple. Therefore, for any good GGM instance, there are at least $(1 - \frac{1}{2\varrho^2}) \cdot t^2$ tuples that are both Good and Invertible.

Furthermore, for any GGM instance $\mathcal{G}_{N,m}$, if a tuple $(\mathcal{G}_{N,m}, \text{sk}_1, \text{sk}_2)$ is invertible, then there are at least one seed pair, such as $(\text{seed}_1, \text{seed}_2)$, from which $(\mathcal{G}_{N,m}, \text{sk}_1, \text{sk}_2)$ can be deduced from. Consequently, for any fixed GGM instance, we have that,

$$\text{Num of bad and invertible tuples} \leq \text{Num of bad seed pairs}.$$

¹²Here, i and j may be equal.

We are now prepared to establish the lower bound on the probability of the extractor $\mathcal{E}$ wins when accessing $\mathcal{G}_{N,m}$, conditioned on $\mathcal{G}_{N,m}$ being a bad GGM instance. Concretely, let t be the size of $Q_{\mathcal{G}_{N,m}}$, we have that,

$$\begin{aligned}
\Pr[\mathcal{E}^{\mathcal{G}_{N,m}} \text{ wins} | \mathcal{G}_{N,m} \text{ is bad}] &\geq \Pr[\mathcal{E}^{\mathcal{G}_{N,m}} \text{ wins} | \text{BadSeed}] \cdot \frac{\text{Num of bad seed pairs}}{\text{Num of all seed pairs}} \\
&\geq \frac{1}{2} \cdot \frac{\text{Num of bad and invertible tuples}}{\text{Num of all seed pairs}} \\
&\geq \frac{1}{2} \cdot \frac{\frac{1}{2\varrho^2} \cdot t^2}{p^2} \\
&\geq \frac{1}{2} \cdot \frac{\frac{1}{2\varrho^2} \cdot \left\lceil \frac{p}{\varrho} \right\rceil^2}{p^2} \\
&\geq \frac{1}{4\varrho^4}.
\end{aligned}$$

Next, we analyze the probability of $\mathcal{E}$ wins, where the probability is also considered over the uniform sampling of the GGM instance. Specifically,

$$\begin{aligned}
\Pr[\mathcal{E} \text{ wins}] &\geq \Pr[\mathcal{E}^{\mathcal{G}_{N,m}} \text{ wins} | \mathcal{G}_{N,m} \text{ is bad}] \cdot \frac{\text{Num of bad GGM instances}}{\text{Num of all GGM instances}} \\
&\geq \frac{1}{4\varrho^4} \cdot \frac{\text{Num of bad GGM instances}}{\text{Num of all GGM instances}}.
\end{aligned}$$

For clarity in exposition, we define TotalGGM as the set of all GGM instances, BadGGM as the subset of bad GGM instances, and GoodGGM as the subset of good GGM instances. Moreover, we define

$$\text{BadRatio} := \frac{|\text{BadGGM}|}{|\text{TotalGGM}|}; \text{GoodRatio} := \frac{|\text{GoodGGM}|}{|\text{TotalGGM}|}.$$

Thus, the probability of the extractor $\mathcal{E}$ wins is bounded by

$$\Pr[\mathcal{E} \text{ wins}] \geq \frac{1}{4\varrho^4} \cdot \text{BadRatio}.$$

According to the analysis in [Zha22] and Lemma G.1, we know that the probability of $\mathcal{E}$ outputs a new group element is negligible, indicating that BadRatio is negligible. Therefore, it suffices to prove that, if $\Pr[\text{Good} | \text{Invertible}]$ is not noticeable, then BadRatio is not negligible.

Claim. If $\Pr[\text{Good} | \text{Invertible}] \leq \frac{1}{4\varrho^2} \cdot \frac{1}{\text{poly}_{\text{AE}}(\lambda)^2}$, then $\text{BadRatio} \geq \frac{1}{2}$.

We proceed our analysis by contraposition. Specifically, we prove that if $\text{BadRatio} < \frac{1}{2}$, then $\Pr[\text{Good} | \text{Invertible}] > \frac{1}{4\varrho^2} \cdot \frac{1}{\text{poly}_{\text{AE}}(\lambda)^2}$. By definition, we have that

$$\begin{aligned}
\Pr[\text{Good} | \text{Invertible}] &= \frac{\Pr[\text{Good} \wedge \text{Invertible}]}{\Pr[\text{Invertible}]} \\
&= \frac{\sum_{\mathcal{G}_{N,m}} \text{Num of Good and Invertible tuples w.r.t. } \mathcal{G}_{N,m}}{\sum_{\mathcal{G}_{N,m}} \text{Num of Invertible tuples w.r.t. } \mathcal{G}_{N,m}},
\end{aligned}$$

where the summation $\sum_{\mathcal{G}_{N,m}}$ is taken over all GGM instances. Note that for any GGM instance $\mathcal{G}_{N,m}$, since the size of the private-key space is N , there are at most N^2 relevant tuples associated with $\mathcal{G}_{N,m}$. This implies that

$$\sum_{\mathcal{G}_{N,m}} \text{Num of Invertible tuples w.r.t. } \mathcal{G}_{N,m} \leq |\text{TotalGGM}| \cdot N^2.$$

Furthermore, by the definition of good GGM instance, for any good GGM instance $\mathcal{G}_{N,m}$, there are at least $(1 - \frac{1}{2\varrho^2}) \cdot t^2$ tuples that are both Good and Invertible tuples associated with $\mathcal{G}_{N,m}$. Therefore, we know that

$$\sum_{\mathcal{G}_{N,m}} \text{Num of Good and Invertible tuples w.r.t. } \mathcal{G}_{N,m} \geq |\text{GoodGGM}| \cdot (1 - \frac{1}{2\varrho^2}) \cdot t^2,$$

where $t = |Q_{\mathcal{G}_{N,m}}| \geq \frac{p}{\varrho}$. Thus, we have that

$$\begin{aligned} \Pr[\text{Good}|\text{Invertible}] &\geq \frac{|\text{GoodGGM}| \cdot (1 - \frac{1}{2\varrho^2}) \cdot t^2}{|\text{TotalGGM}| \cdot N^2} \\ &\geq (1 - \text{BadRatio}) \cdot \frac{(1 - \frac{1}{2\varrho^2}) \cdot \frac{p^2}{\varrho^2}}{N^2} \\ &\geq (1 - \text{BadRatio}) \cdot \frac{1}{2\varrho^2} \cdot \frac{1}{\text{poly}_{\text{AE}}(\lambda)^2}. \end{aligned}$$

Therefore, we conclude that, if $\text{BadRatio} < \frac{1}{2}$, then

$$\Pr[\text{Good}|\text{Invertible}] \geq \frac{1}{4\varrho^2} \cdot \frac{1}{\text{poly}_{\text{AE}}(\lambda)^2}.$$

Combining together, we establish the entire proof. □

□

Acknowledgment

Cong Zhang was supported by the National Key Research and Development Program of China (Grant No. 2023YFB3106000). This work was supported by the Open Research Fund of The State Key Laboratory of Blockchain and Data Security, Zhejiang University. This work was supported by Ant Group through CCF-Ant Research Fund (Grant No. CCF-AFSG RF20230308). Hong-Sheng Zhou was supported in part by NSF grant CNS-1801470 and a VCU Research Quest grant.

References

- [ABD⁺13] Elena Andreeva, Andrey Bogdanov, Yevgeniy Dodis, Bart Mennink, and John P. Steinberger. On the indifferntiability of key-alternating ciphers. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part I*, volume 8042 of *LNCS*, pages 531–550. Springer, Berlin, Heidelberg, August 2013.
- [Adl79] Leonard Adleman. A subexponential algorithm for the discrete logarithm problem with applications to cryptography. In *20th Annual Symposium on Foundations of Computer Science (SFCS 1979)*, pages 55–60. IEEE Computer Society, 1979.
- [BB04] Dan Boneh and Xavier Boyen. Short signatures without random oracles. In Christian Cachin and Jan Camenisch, editors, *EUROCRYPT 2004*, volume 3027 of *LNCS*, pages 56–73. Springer, Berlin, Heidelberg, May 2004.
- [BBB⁺18] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *2018 IEEE Symposium on Security and Privacy*, pages 315–334. IEEE Computer Society Press, May 2018.
- [BCI⁺10] Eric Brier, Jean-Sébastien Coron, Thomas Icart, David Madore, Hugues Randriam, and Mehdi Tibouchi. Efficient indifferntiable hashing into ordinary elliptic curves. In Tal Rabin, editor, *CRYPTO 2010*, volume 6223 of *LNCS*, pages 237–254. Springer, Berlin, Heidelberg, August 2010.

- [BF03] Dan Boneh and Matthew K. Franklin. Identity-based encryption from the weil pairing. *SIAM J. Comput.*, 32(3):586–615, 2003.
- [BF18] Manuel Barbosa and Pooya Farshim. Indifferentiable authenticated encryption. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part I*, volume 10991 of *LNCS*, pages 187–220. Springer, Cham, August 2018.
- [BGOS07] Paulo S. L. M. Barreto, Steven D. Galbraith, Colm O’heigeartaigh, and Michael Scott. Efficient pairing computation on supersingular abelian varieties. *DCC*, 42(3):239–271, 2007.
- [BKS⁺11] Zvika Brakerski, Jonathan Katz, Gil Segev, and Arkady Yerukhimovich. Limits on the power of zero-knowledge proofs in cryptographic constructions. In Yuval Ishai, editor, *TCC 2011*, volume 6597 of *LNCS*, pages 559–578. Springer, Berlin, Heidelberg, March 2011.
- [CDMP05] Jean-Sébastien Coron, Yevgeniy Dodis, Cécile Malinaud, and Prashant Puniya. Merkle-Damgård revisited: How to construct a hash function. In Victor Shoup, editor, *CRYPTO 2005*, volume 3621 of *LNCS*, pages 430–448. Springer, Berlin, Heidelberg, August 2005.
- [CDMS10] Jean-Sébastien Coron, Yevgeniy Dodis, Avradip Mandal, and Yannick Seurin. A domain extender for the ideal cipher. In Daniele Micciancio, editor, *TCC 2010*, volume 5978 of *LNCS*, pages 273–289. Springer, Berlin, Heidelberg, February 2010.
- [CHK⁺16] Jean-Sébastien Coron, Thomas Holenstein, Robin Künzler, Jacques Patarin, Yannick Seurin, and Stefano Tessaro. How to build an ideal cipher: The indistinguishability of the Feistel construction. *Journal of Cryptology*, 29(1):61–114, January 2016.
- [CMR⁺23] Lily Chen, Dustin Moody, Karen Randall, Andrew Regenscheid, and Angela Robinson. Recommendations for discrete logarithm-based cryptography: Elliptic curve domain parameters, 2023. <https://doi.org/10.6028/NIST.SP.800-186>.
- [DH76] Whitfield Diffie and Martin E. Hellman. New directions in cryptography. *IEEE Trans. Inf. Theory*, 22(6):644–654, 1976.
- [DHH⁺21] Nico Döttling, Dominik Hartmann, Dennis Hofheinz, Eike Kiltz, Sven Schäge, and Bogdan Ursu. On the impossibility of purely algebraic signatures. In Kobbi Nissim and Brent Waters, editors, *TCC 2021, Part III*, volume 13044 of *LNCS*, pages 317–349. Springer, Cham, November 2021.
- [DRS09] Yevgeniy Dodis, Thomas Ristenpart, and Thomas Shrimpton. Salvaging Merkle-Damgård for practical applications. In Antoine Joux, editor, *EUROCRYPT 2009*, volume 5479 of *LNCS*, pages 371–388. Springer, Berlin, Heidelberg, April 2009.
- [DSSL16] Yevgeniy Dodis, Martijn Stam, John P. Steinberger, and Tianren Liu. Indistinguishability of confusion-diffusion networks. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 679–704. Springer, Berlin, Heidelberg, May 2016.
- [ElG85] Taher ElGamal. A public key cryptosystem and a signature scheme based on discrete logarithms. *IEEE Transactions on Information Theory*, 31(4):469–472, 1985.
- [FR94] Gerhard Frey and Hans-Georg Rück. A remark concerning m -divisibility and the discrete logarithm in the divisor class group of curves. *Mathematics of computation*, 62(206):865–874, 1994.
- [FT10] Pierre-Alain Fouque and Mehdi Tibouchi. Estimating the size of the image of deterministic hash functions to elliptic curves. In Michel Abdalla and Paulo S. L. M. Barreto, editors, *LATINCRYPT 2010*, volume 6212 of *LNCS*, pages 81–91. Springer, Berlin, Heidelberg, August 2010.

- [Gro16] Jens Groth. On the size of pairing-based non-interactive arguments. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 305–326. Springer, Berlin, Heidelberg, May 2016.
- [GS22] Jens Groth and Victor Shoup. On the security of ECDSA with additive key derivation and presignatures. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part I*, volume 13275 of *LNCS*, pages 365–396. Springer, Cham, May / June 2022.
- [GWL23] Chun Guo, Lei Wang, and Dongdai Lin. Impossibility of indifferentiable iterated blockciphers from 3 or less primitive calls. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part IV*, volume 14007 of *LNCS*, pages 408–439. Springer, Cham, April 2023.
- [HMQS23] Mohammad Hajiabadi, Mohammad Mahmoody, Wei Qi, and Sara Sarfaraz. Lower bounds on assumptions behind registration-based encryption. In Guy N. Rothblum and Hoeteck Wee, editors, *TCC 2023, Part II*, volume 14370 of *LNCS*, pages 306–334. Springer, Cham, November / December 2023.
- [Ica09] Thomas Icart. How to hash into elliptic curves. In Shai Halevi, editor, *CRYPTO 2009*, volume 5677 of *LNCS*, pages 303–316. Springer, Berlin, Heidelberg, August 2009.
- [IR89] Russell Impagliazzo and Steven Rudich. Limits on the provable consequences of one-way permutations. In *21st ACM STOC*, pages 44–61. ACM Press, May 1989.
- [JZW⁺24] Keyu Ji, Cong Zhang, Taiyu Wang, Bingsheng Zhang, Hong-Sheng Zhou, Xin Wang, and Kui Ren. On the complexity of cryptographic groups and generic group models. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part VII*, volume 15490 of *LNCS*, pages 3–35. Springer, Singapore, December 2024.
- [Kob87] Neal Koblitz. Elliptic curve cryptosystems. *Mathematics of computation*, 48(177):203–209, 1987.
- [LPS23] Helger Lipmaa, Roberto Parisella, and Janno Siim. Algebraic group model with oblivious sampling. In Guy N. Rothblum and Hoeteck Wee, editors, *TCC 2023, Part IV*, volume 14372 of *LNCS*, pages 363–392. Springer, Cham, November / December 2023.
- [LZ24] George Lu and Mark Zhandry. Limits on the power of prime-order groups: Separating Q-type from static assumptions. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part V*, volume 14924 of *LNCS*, pages 46–74. Springer, Cham, August 2024.
- [Mau05] Ueli M. Maurer. Abstract models of computation in cryptography (invited paper). In Nigel P. Smart, editor, *10th IMA International Conference on Cryptography and Coding*, volume 3796 of *LNCS*, pages 1–12. Springer, Berlin, Heidelberg, December 2005.
- [Mil86] Victor S. Miller. Use of elliptic curves in cryptography. In Hugh C. Williams, editor, *CRYPTO’85*, volume 218 of *LNCS*, pages 417–426. Springer, Berlin, Heidelberg, August 1986.
- [MR19] Daniel Masny and Peter Rindal. Endemic oblivious transfer. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 309–326. ACM Press, November 2019.
- [MRH04] Ueli M. Maurer, Renato Renner, and Clemens Holenstein. Indifferentiability, impossibility results on reductions, and applications to the random oracle methodology. In Moni Naor, editor, *TCC 2004*, volume 2951 of *LNCS*, pages 21–39. Springer, Berlin, Heidelberg, February 2004.
- [RSS11] Thomas Ristenpart, Hovav Shacham, and Thomas Shrimpton. Careful with composition: Limitations of the indistinguishability framework. In Kenneth G. Paterson, editor, *EUROCRYPT 2011*, volume 6632 of *LNCS*, pages 487–506. Springer, Berlin, Heidelberg, May 2011.

- [RTV04] Omer Reingold, Luca Trevisan, and Salil P. Vadhan. Notions of reducibility between cryptographic primitives. In Moni Naor, editor, *TCC 2004*, volume 2951 of *LNCS*, pages 1–20. Springer, Berlin, Heidelberg, February 2004.
- [Sho97] Victor Shoup. Lower bounds for discrete logarithms and related problems. In Walter Fumy, editor, *EUROCRYPT’97*, volume 1233 of *LNCS*, pages 256–266. Springer, Berlin, Heidelberg, May 1997.
- [Zha22] Mark Zhandry. To label, or not to label (in generic groups). In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part III*, volume 13509 of *LNCS*, pages 66–96. Springer, Cham, August 2022.
- [ZZ20] Mark Zhandry and Cong Zhang. Indifferentiability for public key cryptosystems. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part I*, volume 12170 of *LNCS*, pages 63–93. Springer, Cham, August 2020.
- [ZZ23] Cong Zhang and Mark Zhandry. The relationship between idealized models under computationally bounded adversaries. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part VI*, volume 14443 of *LNCS*, pages 390–419. Springer, Singapore, December 2023.

A Sparse GGM vs. dense groups

In this section, we focus on cryptographic groups with dense group encodings, demonstrating that any such group that is CDH-secure, denoted as $\mathcal{d}\text{-}\mathcal{P}^{\text{CDH}}$, cannot exist within the sparse GGM. Following a similar strategy as in Section 3, we introduce an intermediary primitive, i.e. non-interactive key exchange (NIKE) with dense public-key space, which is secure against key-recovery attack (KRA-secure), denoted as $\mathcal{d}\text{-}\mathcal{P}^{\text{NIKE}}$, and prove that:

- $\mathcal{d}\text{-}\mathcal{P}^{\text{CDH}}$ implies $\mathcal{d}\text{-}\mathcal{P}^{\text{NIKE}}$;
- $\mathcal{d}\text{-}\mathcal{P}^{\text{NIKE}}$ does not exist in the sparse GGM.

A.1 Non-interactive key exchange with dense public-key space

In this section, we present the formal definition of the KRA-secure non-interactive key exchange with dense public-key space.

Definition A.1 (NIKE with Dense Public-Key Space). *A KRA-secure non-interactive key exchange protocol with dense public-key space, denoted as $\mathcal{d}\text{-}\mathcal{P}^{\text{NIKE}}$, consists of the pair $\langle \mathcal{d}\text{-}\mathcal{F}^{\text{NIKE}}, \mathcal{d}\text{-}\mathcal{R}^{\text{NIKE}} \rangle$:*

1. *The set $\mathcal{d}\text{-}\mathcal{F}^{\text{NIKE}}$ consists of functions f that, on input of a security parameter, output the description of a non-interactive key exchange protocol. Specifically, we write $(\text{KGen}, \text{SHK}) \leftarrow f(1^\lambda)$, where algorithms are associated with $\mathcal{SK}$, $\mathcal{PK}$, and $\mathcal{K}$.*
 - $\mathcal{SK}$, $\mathcal{PK}$, and $\mathcal{K}$ are the private-key space, public-key space and shared-key space, respectively, satisfying that $\mathcal{SK} := \mathbb{Z}_N$, where N is a λ -bit integer, and $\log |\mathcal{PK}| - \log |\mathcal{SK}| \leq \Theta(\log \lambda)$;
 - The public-key generation function $\text{KGen} : \mathcal{SK} \mapsto \mathcal{PK}$ is an injection, for generating a public key $\text{pk} \in \mathcal{PK}$ from a randomly chosen private key $\text{sk} \in \mathcal{SK}$.
 - The shared-key generation function $\text{SHK} : \mathcal{PK} \times \mathcal{SK} \mapsto \mathcal{K} \cup \{\perp\}$ for generating a shared key $\text{shk} \in \mathcal{K} \cup \{\perp\}$, where $\perp$ indicates a failed computation.

Concretely, for randomly chosen $sk \xleftarrow{\$} \mathcal{SK}$ and $sk' \xleftarrow{\$} \mathcal{SK}$, compute $pk \leftarrow \text{KGen}(sk)$ and $pk' \leftarrow \text{KGen}(sk')$. We write $shk \leftarrow \text{SHK}(pk', sk)$ and $shk' \leftarrow \text{SHK}(pk, sk')$. The protocol is required to achieve perfect correctness, meaning that:

$$\Pr [shk = \perp \vee shk' = \perp \vee shk \neq shk'] = 0.$$

2. For a function $f = (\text{KGen}, \text{SHK}) \in \mathcal{d}\text{-}\mathcal{F}^{\text{NIKE}}$ and a PPT (adversarial) machine $\mathcal{A}$, we define $\langle f, \mathcal{A} \rangle \in \mathcal{d}\text{-}\mathcal{R}^{\text{NIKE}}$ if $\mathcal{A}$ can break the security property of f against key-recovery attack (KRA).

We say $\mathcal{d}\text{-}\mathcal{P}^{\text{NIKE}}$ exists if there is a function $f \in \mathcal{d}\text{-}\mathcal{F}^{\text{NIKE}}$ such that no PPT adversarial machine $\mathcal{A}$ satisfies $\langle f, \mathcal{A} \rangle \in \mathcal{d}\text{-}\mathcal{R}^{\text{NIKE}}$.

It is apparent that $\mathcal{d}\text{-}\mathcal{P}^{\text{CDH}}$ implies $\mathcal{d}\text{-}\mathcal{P}^{\text{NIKE}}$ by defining $\text{KGen}(x) := g^x$ and $\text{SHK}(pk, sk') := pk^{sk'}$. Therefore, it is sufficient to prove that $\mathcal{d}\text{-}\mathcal{P}^{\text{NIKE}}$ does not exist in the sparse GGM.

A.2 $\mathcal{d}\text{-}\mathcal{P}^{\text{NIKE}}$ does not exist in the sparse GGM

In this section, we establish the black-box separation between $\mathcal{d}\text{-}\mathcal{P}^{\text{NIKE}}$ and the sparse GGM. Specifically, we construct an adversary that, while computationally unbounded, is query-efficient and capable of breaking the KRA-secure game with a noticeable probability.

Theorem A.1. Let $\mathcal{G}_{N,m}$ be a generic group model such that $m - \log N \geq \omega(\log \lambda)$. Let $\Pi^{\mathcal{G}_{N,m}} = (\text{KGen}^{\mathcal{G}_{N,m}}, \text{SHK}^{\mathcal{G}_{N,m}})$ be any non-interactive key exchange protocol with dense public-key space. Then there exists an adversary $\mathcal{A}$ and two polynomials poly_1 and poly_2 such that:

- $\mathcal{A}$ makes poly_1 queries to $\mathcal{G}_{N,m}$;
- $\mathcal{A}$ breaks the KRA-secure game with advantage $\frac{1}{\text{poly}_2}$.

Proof. To establish the proof, we present a formal description of the adversary $\mathcal{A}$, as illustrated in Figure 4. Here, we specify that any algorithm in $\Pi^{\mathcal{G}_{N,m}}$ makes at most q queries, where q is a polynomial.

Trivial to note that the adversary $\mathcal{A}^{\mathcal{G}_{N,m}}$ is query-efficient. We next prove that the adversary $\mathcal{A}^{\mathcal{G}_{N,m}}$ can successfully guess the valid shared key with a noticeable probability. Let $S_{\text{Bob-L}}$ denote the set of all valid group encodings that appear when running the algorithms $\text{KGen}^{\mathcal{G}_{N,m}}(sk_2)$ and $\text{SHK}^{\mathcal{G}_{N,m}}(pk_1, sk_2)$; these encodings are either the responses of labeling/addition queries or the valid inputs of the addition queries. Clearly, $|S_{\text{Bob-L}}| \leq 6q$, since each algorithm makes at most q queries to $\mathcal{G}_{N,m}$. We then define:

$$S_{\text{Bob}} := \{(x, \text{str}) \mid \text{str} \in S_{\text{Bob-L}}, \mathcal{G}_{N,m}^{\text{label}}(x) = \text{str}\}.$$

Note that in each iteration, if the encoding pairs in $\tilde{S}_{\mathcal{A}}$ are consistent with S_{Bob} , the shared key computed in that iteration would be correct. Specifically, in such a context, another GGM instance exists that is consistent with both the simulation view of $\mathcal{A}$ and the real view of Bob, and the perfect correctness of $\mathcal{d}\text{-}\mathcal{P}^{\text{NIKE}}$ ensures that the shared key computed in this iteration is correct. However, without knowledge of sk_2 , $\tilde{S}_{\mathcal{A}}$ might be inconsistent with S_{Bob} with high probability. Specifically, as analyzed in Section 3, there are essentially three bad events that prevent $\mathcal{A}$ from guessing the correct shared key:

- Event 1: There exist $(x, \text{str}) \in \tilde{S}_{\mathcal{A}}$ and $(x', \text{str}') \in S_{\text{Bob}}$ such that $x = x'$ but $\text{str} \neq \text{str}'$.
- Event 2: There exist $(x, \text{str}) \in \tilde{S}_{\mathcal{A}}$ and $(x', \text{str}') \in S_{\text{Bob}}$ such that $x \neq x'$ but $\text{str} = \text{str}'$.
- Event 3: When running $\text{SHK}^{S_{\text{que-res}} \cup \tilde{S}_{\mathcal{A}}}(\text{pk}_2, \tilde{sk}_1)$, there exists an addition query $\text{que} = (\text{str}_1, \text{str}_2)$ such that either str_1 or str_2 is not collected in $S_{\text{que-res}} \cup \tilde{S}_{\mathcal{A}}$, yet both are present in S_{Bob} .

Adversary $\mathcal{A}^{\mathcal{G}_{N,m}}$

$\mathcal{A}^{\mathcal{G}_{N,m}}(\text{pk}_1, \text{pk}_2)$:

$S_{\text{key}} \leftarrow \emptyset; S_{\text{que-res}} \leftarrow \emptyset;$

Iteration phase:

for $i = 1$ to $12q + 1$:

Simulation: search a proper view of Alice

search a private key $\tilde{\text{sk}}_1$ and a set of encoding pairs $\tilde{S}_{\mathcal{A}}$ satisfying the following properties:

1. $\tilde{S}_{\mathcal{A}}$ is consistent with $S_{\text{que-res}}$ and $|\tilde{S}_{\mathcal{A}}| \leq 12q$;
2. $\text{pk}_1 \leftarrow \text{KGen}^{S_{\text{que-res}} \cup \tilde{S}_{\mathcal{A}}}(\tilde{\text{sk}}_1)$;
3. $\tilde{S}_{\mathcal{A}}$ is sufficient for the algorithm $\widetilde{\text{shk}}_i \leftarrow \text{SHK}^{S_{\text{que-res}} \cup \tilde{S}_{\mathcal{A}}}(\text{pk}_2, \tilde{\text{sk}}_1)$:
 - if $\text{que} = x$ is a labeling query, then x should be covered in $S_{\text{que-res}} \cup \tilde{S}_{\mathcal{A}}$;
 - if $\text{que} = (\text{str}_1, \text{str}_2)$ is an addition query and there are $(x_1, \text{str}_1), (x_2, \text{str}_2)$ in $S_{\text{que-res}} \cup \tilde{S}_{\mathcal{A}}$, then $(x_1 + x_2, \text{str}_3)$ should also be covered in $S_{\text{que-res}} \cup \tilde{S}_{\mathcal{A}}$;
 - if $\text{que} = (\text{str}_1, \text{str}_2)$ is an addition query but there is no (x_1, str_1) and/or (x_2, str_2) in $S_{\text{que-res}} \cup \tilde{S}_{\mathcal{A}}$, then respond with $\perp$.

$S_{\text{key}} \leftarrow S_{\text{key}} \cup \{\widetilde{\text{shk}}_i\};$

Update: replace the guessing labeling queries with valid encodings

for each $(x, \text{str}) \in \tilde{S}_{\mathcal{A}} \setminus S_{\text{que-res}}$: $S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup (x, \mathcal{G}_{N,m}^{\text{label}}(x))$;

Final phase: //output the guessing shared key

return the majority value in S_{key} .

Figure 4: The description of $\mathcal{A}$ that breaks the KRA-secure game of $\text{d-}\mathcal{P}^{\text{NIKE}}$ in $\mathcal{G}_{N,m}$.

The updating phase of $\mathcal{A}$ causes Event 1 to occur at most $6q$ times. Meanwhile, the occurrence of either Event 2 or Event 3 results in a bad case that the adversary $\mathcal{A}$ generates valid group elements str without making labeling queries (i.e., without knowing the corresponding discrete logarithm). Therefore, if the probability of such a bad case is small, then the adversary $\mathcal{A}^{\mathcal{G}_{N,m}}$ can break the KRA-secure game and output the valid shared key with a good probability.

Consider choosing a GGM $\mathcal{G}_{N,m}$ along with two private keys sk_1 and sk_2 , and fixing them. We observe that once the randomness of the adversary is set, the algorithm $\mathcal{A}^{\mathcal{G}_{N,m}}(\text{pk}_1, \text{pk}_2)$ becomes deterministic. Let r be a nonce, we say the adversary performs good with respect to the tuple $(\mathcal{G}_{N,m}, \text{sk}_1, \text{sk}_2, r)$ if $\mathcal{A}^{\mathcal{G}_{N,m}}(\text{pk}_1, \text{pk}_2)$, using r as its source of randomness, cannot generate a valid group element without knowing the corresponding discrete logarithm. Here, pk_i represents the public key derived from the private key sk_i .

Next, we introduce the concept of “good” for the tuple $(\mathcal{G}_{N,m}, \text{sk}_1, \text{sk}_2)$. More formally, we say a tuple $(\mathcal{G}_{N,m}, \text{sk}_1, \text{sk}_2)$ is good if

$$\Pr[\mathcal{A} \text{ performs good with respect to } (\mathcal{G}_{N,m}, \text{sk}_1, \text{sk}_2, r)] \geq \frac{1}{2},$$

where $\text{pk}_1 \leftarrow \text{KGen}^{\mathcal{G}_{N,m}}(\text{sk}_1)$ and $\text{pk}_2 \leftarrow \text{KGen}^{\mathcal{G}_{N,m}}(\text{sk}_2)$, and the probability is over the internal randomness of $\mathcal{A}$. Analogously, we say a tuple $(\mathcal{G}_{N,m}, \text{sk}_1, \text{sk}_2)$ is bad if

$$\Pr[\mathcal{A} \text{ performs good with respect to } (\mathcal{G}_{N,m}, \text{sk}_1, \text{sk}_2, r)] < \frac{1}{2}.$$

For clarify, we denote these two events in which the tuple $(\mathcal{G}_{N,m}, \text{sk}_1, \text{sk}_2)$ is good or bad as Good and Bad, respectively. Given the perfect correctness of $\Pi^{\mathcal{G}_{N,m}}$, it follows that the probability of $\mathcal{A}$ winning the KRA-secure game, conditioned on Good, is $\geq \frac{1}{2}$. Thus,

$$\begin{aligned} \Pr[\mathcal{A} \text{ wins}] &\geq \Pr[\mathcal{A}^{\mathcal{G}_{N,m}} \text{ wins} | \text{Good}] \cdot \Pr[\text{Good}] \\ &\geq \frac{1}{2} \cdot \Pr[\text{Good}]. \end{aligned}$$

Therefore, it suffices to prove that $\Pr[\text{Good}]$ is noticeable. To finalize the proof, we adopt the same strategy as in Section 3, leveraging the sparsity property of the GGM. Specifically, note that if $\Pr[\text{Good}]$ is small, the adversary $\mathcal{A}$ is likely to generate a valid group element without knowing the corresponding discrete logarithm (which we refer to as a new group element). Consequently, our proof proceeds in two steps:

1. Construct an extractor $\mathcal{E}$ that takes only the security parameter as input;
2. Demonstrate that, if $\Pr[\text{Good}]$ is small, then $\mathcal{E}$ outputs a new group element with a good probability.

These two steps lead to a contradiction of Lemma 3.1. We now present the formal description of the extractor $\mathcal{E}$, as illustrated in Figure 5. The extractor $\mathcal{E}$ takes only the security parameter as input, and randomly selects two public keys (i.e., pk_1 and pk_2) from the public-key space. It then operates in a manner closely resembling the adversary in the KRA-security game, as shown in Figure 4. Specifically, the extractor proceeds the iteration phase. During the simulation process of each iteration phase, if a new and valid group element appears, either in $\tilde{S}_{\mathcal{A}}$ or as an input of some addition queries, then $\mathcal{E}$ outputs this element (highlighted in red). Observe that the randomness of $\mathcal{E}$ is composed of three components: pk_1 , pk_2 , and r , where r represents the randomness used during the iteration phase. Furthermore, once the GGM instance $\mathcal{G}_{N,m}$ and the two public keys pk_1 , pk_2 are fixed, and assuming both pk_1 and pk_2 are valid, the extractor $\mathcal{E}$ proceeds identically to the adversary $\mathcal{A}$ in the KRA-security game, continuing until $\mathcal{E}$ successfully outputs a new and valid group element. Here, we define a public key pk as valid if there exists a corresponding sk such that $\text{KGen}^{\mathcal{G}_{N,m}}(\text{sk}) = \text{pk}$.

Next, we analyze the probability of the extractor wins, which occurs when $\mathcal{E}$ outputs a new and valid group element. Let $\mathcal{G}_{N,m}$ denote the GGM to which the extractor $\mathcal{E}$ has access. Let pk_1 and pk_2 denote the public keys sampled by the extractor. We define a public-key pair, say $(\text{pk}_1, \text{pk}_2)$, to be bad with respect to $\mathcal{G}_{N,m}$ if there exists a tuple $(\mathcal{G}_{N,m}, \text{sk}_1, \text{sk}_2)$, that corresponds to $(\text{pk}_1, \text{pk}_2)$ and is a Bad tuple. More precisely, when we say a tuple $(\mathcal{G}_{N,m}, \text{sk}_1, \text{sk}_2)$ corresponds to $(\text{pk}_1, \text{pk}_2)$ w.r.t. $\mathcal{G}_{N,m}$, we mean that

$$\text{KGen}^{\mathcal{G}_{N,m}}(\text{sk}_1) = \text{pk}_1 \text{ and } \text{KGen}^{\mathcal{G}_{N,m}}(\text{sk}_2) = \text{pk}_2.$$

For simplicity, we refer to this event as BadPKs in the following analysis. According to the definition of Bad, we note that if $\mathcal{E}$ fortunately samples a bad public-key pair, i.e., BadPKs occurs, then $\mathcal{E}$ wins with a good probability. Concretely,

$$\Pr[\mathcal{E}^{\mathcal{G}_{N,m}} \text{ wins} | \text{BadPKs}] \geq \frac{1}{2}.$$

Extractor $\mathcal{E}^{\mathcal{G}_{N,m}}(1^\lambda)$

$\text{pk}_1 \xleftarrow{\$} \mathcal{PK}, \text{pk}_2 \xleftarrow{\$} \mathcal{PK};$

$S_{\text{que-res}} \leftarrow \emptyset;$

Iteration phase:

for $i = 1$ to $12q + 1$:

Simulation: search a proper view of Alice

search a private key $\tilde{\text{sk}}_1$ and a set of encoding pairs $\tilde{S}_{\mathcal{A}}$ satisfying the following properties:

1. $\tilde{S}_{\mathcal{A}}$ is consistent with $S_{\text{que-res}}$ and $|\tilde{S}_{\mathcal{A}}| \leq 12q$;
2. $\text{pk}_1 \leftarrow \text{KGen}^{S_{\text{que-res}} \cup \tilde{S}_{\mathcal{A}}}(\tilde{\text{sk}}_1)$;
3. $\tilde{S}_{\mathcal{A}}$ is sufficient for the algorithm $\tilde{\text{shk}}_i \leftarrow \text{SHK}^{S_{\text{que-res}} \cup \tilde{S}_{\mathcal{A}}}(\text{pk}_2, \tilde{\text{sk}}_1)$:
 - if $\text{que} = x$ is a labeling query, then x should be covered in $S_{\text{que-res}} \cup \tilde{S}_{\mathcal{A}}$;
 - if $\text{que} = (\text{str}_1, \text{str}_2)$ is an addition query and there are $(x_1, \text{str}_1), (x_2, \text{str}_2)$ in $S_{\text{que-res}} \cup \tilde{S}_{\mathcal{A}}$, then $(x_1 + x_2, \text{str}_3)$ should also be covered in $S_{\text{que-res}} \cup \tilde{S}_{\mathcal{A}}$;
 - if $\text{que} = (\text{str}_1, \text{str}_2)$ is an addition query but there is no (x_1, str_1) and/or (x_2, str_2) in $S_{\text{que-res}} \cup \tilde{S}_{\mathcal{A}}$, then respond with $\perp$.

For each $(x, \text{str}) \in \tilde{S}_{\mathcal{A}} \setminus S_{\text{que-res}}$: if $\mathcal{G}_{N,m}^{\text{add}}(\text{str}, \text{str}) \neq \perp$, return str ;

Re-run the algorithm $\tilde{\text{shk}}_i \leftarrow \text{SHK}^{S_{\text{que-res}} \cup \tilde{S}_{\mathcal{A}}}(\text{pk}_2, \tilde{\text{sk}}_1)$ and $\text{que} = (\text{str}_1, \text{str}_2)$ is an addition query during the execution:

if $\text{str}_1 \notin S_{\text{que-res}}$ and $\mathcal{G}_{N,m}^{\text{add}}(\text{str}_1, \text{str}_1) \neq \perp$, return str_1 ;

if $\text{str}_2 \notin S_{\text{que-res}}$ and $\mathcal{G}_{N,m}^{\text{add}}(\text{str}_2, \text{str}_2) \neq \perp$, return str_2 ;

Updating: replace the guessing labeling queries with valid encodings

for each $(x, \text{str}) \in \tilde{S}_{\mathcal{A}} \setminus S_{\text{que-res}}$: $S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup (x, \mathcal{G}_{N,m}^{\text{label}}(x))$;

return $\perp$.

Figure 5: The description of $\mathcal{E}$ w.r.t. $\text{d-}\mathcal{P}^{\text{NIKE}}$ that outputs a new group element of $\mathcal{G}_{N,m}$.

Next, we analyze the probability of BadPKs with respect to any specific GGM instance. To facility the analysis, we categorize all instances of the GGM into two types: good GGM instances and bad GGM instances. Specifically, let $\mathcal{G}_{N,m}$ be a GGM instance. Let $(\text{sk}_i, \text{sk}_j)$ be a private-key pair where $\text{sk}_i, \text{sk}_j \in \mathbb{Z}_N$ ¹³. We classify $\mathcal{G}_{N,m}$ as a bad instance if there are more than $\frac{1}{4} \cdot N^2$ private-key pairs such that each induces a Bad tuple. More precisely, when we say a private-key pair $(\text{sk}_i, \text{sk}_j)$ w.r.t. $\mathcal{G}_{N,m}$ induces a Bad tuple, we

¹³ sk_i and sk_j may be equal.

mean that the tuple $(\mathcal{G}_{N,m}, sk_i, sk_j)$ is bad. Analogously, we classify $\mathcal{G}_{N,m}$ as a good instance if there are at most $\frac{1}{4} \cdot N^2$ private-key pairs, each of which induces a Bad tuple. Therefore, for any good GGM instance, there are at least $(1 - \frac{1}{4}) \cdot N^2$ tuples that are Good.

Moreover, for any $\mathcal{G}_{N,m}$, there is exactly one tuple $(\mathcal{G}_{N,m}, sk_1, sk_2)$ corresponds to a particular (pk_1, pk_2) with respect to $\mathcal{G}_{N,m}$, where both pk_1 and pk_2 are valid. Consequently, for any fix GGM instance, we have that

$$\text{Num of Bad tuples} = \text{Num of BadPKs}.$$

We are now prepared to establish the lower bound on the probability of the extractor $\mathcal{E}$ wins when accessing $\mathcal{G}_{N,m}$, conditioned on $\mathcal{G}_{N,m}$ being a bad GGM instance. Concretely, we have that

$$\begin{aligned} \Pr[\mathcal{E}^{\mathcal{G}_{N,m}} \text{ wins} | \mathcal{G}_{N,m} \text{ is bad}] &\geq \Pr[\mathcal{E}^{\mathcal{G}_{N,m}} \text{ wins} | \text{BadPKs}] \cdot \frac{\text{Num of BadPKs}}{\text{Num of all pk pairs}} \\ &\geq \frac{1}{2} \cdot \frac{\text{Num of Bad tuples}}{\text{Num of all pk pairs}} \\ &\geq \frac{1}{2} \cdot \frac{\frac{1}{4} \cdot N^2}{|\mathcal{PK}|^2} \\ &= \frac{1}{8} \cdot \left(\frac{|\mathcal{SK}|}{|\mathcal{PK}|} \right)^2 = \frac{1}{8} \cdot \frac{1}{\Theta(\lambda)^2} \end{aligned}$$

Next, we analyze the probability of $\mathcal{E}$ wins, where the probability is also considered over the uniform sampling of the GGM instance. Specifically,

$$\begin{aligned} \Pr[\mathcal{E} \text{ wins}] &\geq \Pr[\mathcal{E}^{\mathcal{G}_{N,m}} \text{ wins} | \mathcal{G}_{N,m} \text{ is bad}] \cdot \frac{\text{Num of bad GGM instances}}{\text{Num of all GGM instances}} \\ &\geq \frac{1}{8} \cdot \frac{1}{\Theta(\lambda)^2} \cdot \frac{\text{Num of bad GGM instances}}{\text{Num of all GGM instances}}. \end{aligned}$$

For clarity in exposition, we define TotalGGM as the set of all GGM instances, BadGGM as the subset of bad GGM instances, and GoodGGM as the subset of good GGM instances. Moreover, we define

$$\text{BadRatio} := \frac{|\text{BadGGM}|}{|\text{TotalGGM}|}; \text{GoodRatio} := \frac{|\text{GoodGGM}|}{|\text{TotalGGM}|}.$$

Thus, the probability of the extractor $\mathcal{E}$ wins is bounded by

$$\Pr[\mathcal{E} \text{ wins}] \geq \frac{1}{8} \cdot \frac{1}{\Theta(\lambda)^2} \cdot \text{BadRatio}.$$

According to the analysis in [Zha22] and Lemma G.1, we know that the probability of $\mathcal{E}$ outputs a new group element is negligible, indicating that BadRatio is negligible. Therefore, it suffices to prove that, if $\Pr[\text{Good}]$ is not noticeable, then BadRatio is not negligible.

Claim. If $\Pr[\text{Good}] \leq \frac{1}{2}$, then $\text{BadRatio} \geq \frac{1}{4}$.

We proceed our analysis by contraposition. Specifically, we prove that if $\text{BadRatio} < \frac{1}{4}$, then $\Pr[\text{Good}] > \frac{1}{2}$. Note that for any GGM instance $\mathcal{G}_{N,m}$, since the size of the private-key space is N , there are at most N^2 relevant tuples associated with $\mathcal{G}_{N,m}$. Thus, we have that

$$\Pr[\text{Good}] = \frac{\sum_{\mathcal{G}_{N,m}} (\text{Num of Good w.r.t. } \mathcal{G}_{N,m})}{|\text{TotalGGM}| \cdot N^2}$$

where the summation $\sum_{\mathcal{G}_{N,m}}$ is taken over all GGM instances.

Furthermore, by the definition of good GGM instance, for any good GGM instance $\mathcal{G}_{N,m}$, there are at least $(1 - \frac{1}{4}) \cdot N^2 = \frac{3}{4} \cdot N^2$ tuples that are Good associated with $\mathcal{G}_{N,m}$. Therefore, we know that

$$\sum_{\mathcal{G}_{N,m}} (\text{Num of Good w.r.t. } \mathcal{G}_{N,m}) \geq |\text{GoodGGM}| \cdot \frac{3}{4} \cdot N^2,$$

Therefore, we have that

$$\begin{aligned} \Pr[\text{Good}] &\geq \frac{|\text{GoodGGM}| \cdot \frac{3}{4} \cdot N^2}{|\text{TotalGGM}| \cdot N^2} \\ &= (1 - \text{BadRatio}) \cdot \frac{3}{4}. \end{aligned}$$

Therefore, we conclude that, if $\text{BadRatio} < \frac{1}{4}$, then

$$\Pr[\text{Good}] \geq (1 - \frac{1}{4}) \cdot \frac{3}{4} = \frac{9}{16} > \frac{1}{2}.$$

Combining together, we establish the entire proof. □

□

B Sparse GGM vs. EC-GGM

In this section, we analyze the relationship between the Elliptic Curve Generic Group Model (EC-GGM) and the sparse GGM within the framework of indistinguishability. Specifically, we demonstrate that EC-GGM statistically implies sparse GGM, whereas sparse GGM does not computationally imply EC-GGM.

B.1 EC-GGM statistically implies sparse GGM

We describe how to build a sparse generic group from an EC-GGM equipped with an independent random permutation. Our construction consists of the following two building blocks:

- $\text{ec-}\mathcal{G}_N = (\text{ec-}\mathcal{G}_N^{\text{label}}, \text{ec-}\mathcal{G}_N^{\text{add}})$ is an elliptic curve generic group model over a point set E defined by the equation $y^2 = F(u)$ over $\mathbb{Z}_p$;
- Perm is a random permutation over $\{0, 1\}^m$, along with its inverse Perm^{Inv} .

For simplicity, we denote Orac as the tuple $(\text{ec-}\mathcal{G}_N, \text{Perm}, \text{Perm}^{\text{Inv}})$, and define $\Delta m := m - (\lceil \log p \rceil + 1)$. The construction $\Pi^{\text{Orac}} = (\text{L}^{\text{Orac}}, \text{A}^{\text{Orac}})$ is depicted in Figure 6. Correctness of the construction is straightforward to verify; the remaining task is to establish its indistinguishability from sparse GGM.

Theorem B.1. *Let $\mathcal{G}_{N,m}$ be a sparse generic group model. The scheme Π^{Orac} is indistinguishable from $\mathcal{G}_{N,m}$. More precisely, there exists a simulator $\mathcal{S}$ such that for all q -query differentiator $\mathcal{D}$, we have*

$$\text{Adv}_{\Pi^{\text{Orac}}, \mathcal{G}_{N,m}, \mathcal{S}, \mathcal{D}}^{\text{indif}} \leq \frac{36q^2}{N} + \frac{12q^2 + q}{2^\lambda} + \frac{q}{2^{\omega(\log \lambda)}}.$$

The simulator makes at most λq queries to $\mathcal{G}_{N,m}$.

Proof. According to the definition of indistinguishability, the differentiator interacts with two honest interfaces, namely $\mathcal{G}_{N,m}^{\text{label}}$ and $\mathcal{G}_{N,m}^{\text{add}}$, as well as four adversarial interfaces: $\text{ec-}\mathcal{G}_N^{\text{label}}$, $\text{ec-}\mathcal{G}_N^{\text{add}}$, Perm , and Perm^{Inv} . Our goal is to construct an efficient simulator $\mathcal{S}$ in the ideal world that faithfully simulates these four adversarial interfaces. For clarity of exposition, we introduce the notation $\text{str}^x := \mathcal{G}_{N,m}^{\text{add}}(\underbrace{\text{str}, \dots, \text{str}}_x)$ in the

generic group model, where str denotes a valid group element and $x \in \mathbb{Z}_N$. We now proceed to give a full description of the simulator $\mathcal{S}$.

Construction Π^{Orac}

$\underline{L}^{\text{Orac}}(x)$:

$P \leftarrow \text{ec-}\mathcal{G}_N^{\text{label}}(x)$;
 if $P = \mathcal{O}$, return $\text{Perm}(0^m)$;
 parse P into (u, b) ;
 $\text{str} \leftarrow \text{Perm}(u||b||0^{\Delta m})$;
 return str .

$\underline{A}^{\text{Orac}}(\text{str}_1, \text{str}_2)$:

for $i \in \{1, 2\}$:
 if $\text{Perm}^{\text{Inv}}(\text{str}_i) \neq u_i||b_i||0^{\Delta m}$: return $\perp$;
 if $u_i||b_i||0^{m-\Delta m} = \mathcal{O}$: $P_i := \mathcal{O}$;
 else $P_i := (u_i, b_i)$;
 if $P_1 \notin E$ or $P_2 \notin E$: return $\perp$;
 $P \leftarrow \text{ec-}\mathcal{G}_N^{\text{add}}(P_1, P_2)$;
 if $P = \mathcal{O}$: return $\text{Perm}(0^m)$;
 parse P into (u, b) , return $\text{Perm}(u||b||0^{\Delta m})$.

Figure 6: The construction Π^{Orac} in $\text{ec-}\mathcal{G}_N$ and Perm .

Simulator $\mathcal{S}$

$\underline{S.\text{ec-}\mathcal{G}_N^{\text{label}}}(x)$:

if $x = 0$, return $\mathcal{O}$;
 $\text{str} \leftarrow \mathcal{G}_{N,m}^{\text{label}}(x)$;
 if $\exists(x, P, \text{str}) \in T_{\text{label}}$: return P ;
 if $\exists(\diamond, P, \text{str}) \in T_{\text{label}}$: replace $\diamond$ with x , return P ; // $\diamond$ is a symbol.
 $P \xleftarrow{\$} E$; $T_{\text{label}} \leftarrow T_{\text{label}} \cup \{(x, P, \text{str}), (N - x, -P, \text{str}^{N-1})\}$;
 parse P into (u, b) ; $T_{\text{Perm}} \leftarrow T_{\text{Perm}} \cup \{(u||b||0^{\Delta m}, \text{str}), (u||1-b||0^{\Delta m}, \text{str}^{N-1})\}$;
 return P .

$\underline{S.\text{ec-}\mathcal{G}_N^{\text{add}}}(P_1, P_2)$:

if $\exists(P_1, P_2, P) \in T_{\text{add}}$ or $(P_2, P_1, P) \in T_{\text{add}}$: return P ;
 if $P_1 = \mathcal{O}$: return P_2 ;
 if $P_2 = \mathcal{O}$: return P_1 ;
 for $i \in \{1, 2\}$, if $\nexists(*, P_i, \text{str}_i) \in T_{\text{label}}$: // * refer to $x \in \mathbb{Z}_N$ or the symbol $\diamond$

$\text{str}_i \leftarrow \mathcal{G}_{N,m}^{\text{label}}(x_i)$ for $x_i \xleftarrow{\$} \mathbb{Z}_N$;
 $T_{\text{label}} \leftarrow T_{\text{label}} \cup \{(x_i, P_i, \text{str}_i), (N - x_i, -P_i, \text{str}_i^{N-1})\}$;
 parse P_i into (u_i, b_i) ;
 $T_{\text{Perm}} \leftarrow T_{\text{Perm}} \cup \{(u_i||b_i||0^{\Delta m}, \text{str}_i), (u_i||1-b_i||0^{\Delta m}, \text{str}_i^{N-1})\}$;

for $(*, P_1, \text{str}_1), (*, P_2, \text{str}_2) \in T_{\text{label}}$:

$\text{str} \leftarrow \mathcal{G}_{N,m}^{\text{add}}(\text{str}_1, \text{str}_2)$;
 if $\text{str} = \mathcal{G}_{N,m}^{\text{label}}(0)$: return $\mathcal{O}$;
 if $\exists(*, P, \text{str}) \in T_{\text{label}}$: $T_{\text{add}} \leftarrow T_{\text{add}} \cup \{(P_1, P_2, P)\}$; return P ;
 $P \xleftarrow{\$} E$; $T_{\text{label}} \leftarrow T_{\text{label}} \cup \{(\diamond, P, \text{str}), (\diamond, -P, \text{str}^{N-1})\}$;
 parse P into (u, b) ; $T_{\text{Perm}} \leftarrow T_{\text{Perm}} \cup \{(u||b||0^{\Delta m}, \text{str}), (u||1-b||0^{\Delta m}, \text{str}^{N-1})\}$;
 $T_{\text{add}} \leftarrow T_{\text{add}} \cup \{(P_1, P_2, P)\}$;
 return P .

$S.\text{Perm}(r)$:

```

if  $\exists(r, \text{str}) \in T_{\text{Perm}}$ , return  $\text{str}$ ;
if  $r = 0^m$ , return  $\mathcal{G}_{N,m}^{\text{label}}(0)$ ;
if  $r = u||b||0^{\Delta m}$  s.t.  $P := (u, b) \in E$ :
     $x \xleftarrow{\$} \mathbb{Z}_N$ ;  $\text{str} \leftarrow \mathcal{G}_{N,m}^{\text{label}}(x)$ ;
     $T_{\text{label}} \leftarrow T_{\text{label}} \cup \{(x, P, \text{str}), (N - x, -P, \text{str}^{N-1})\}$ ;
     $T_{\text{Perm}} \leftarrow T_{\text{Perm}} \cup \{(u||b||0^{\Delta m}, \text{str}), (u||1 - b||0^{\Delta m}, \text{str}^{N-1})\}$ ;
    return  $\text{str}$ ;
 $\text{str} \xleftarrow{\$} \{0, 1\}^m$ , if  $\mathcal{G}_{N,m}^{\text{add}}(\text{str}, \text{str}) \neq \perp$ : resample  $\text{str}$  (up to  $\lambda$  times);
 $T_{\text{Perm}} \leftarrow T_{\text{Perm}} \cup \{(r, \text{str})\}$ ;
return  $\text{str}$ .

```

$S.\text{Perm}^{\text{Inv}}(\text{str})$:

```

if  $\exists(r, \text{str}) \in T_{\text{Perm}}$ , return  $r$ ;
if  $\text{str} = \mathcal{G}_{N,m}^{\text{label}}(0)$ , return  $0^m$ ;
if  $\mathcal{G}_{N,m}^{\text{add}}(\text{str}, \text{str}) \neq \perp$ :
     $P \xleftarrow{\$} E$ ;  $T_{\text{label}} \leftarrow T_{\text{label}} \cup \{(\diamond, P, \text{str}), (\diamond, -P, \text{str}^{N-1})\}$ ;
    parse  $P$  into  $(u, b)$ ;  $T_{\text{Perm}} \leftarrow T_{\text{Perm}} \cup \{(u||b||0^{\Delta m}, \text{str}), (u||1 - b||0^{\Delta m}, \text{str}^{N-1})\}$ ;
    return  $u||b||0^{\Delta m}$ ;
 $r \xleftarrow{\$} \{0, 1\}^m$ ;  $T_{\text{Perm}} \leftarrow T_{\text{Perm}} \cup \{(r, \text{str})\}$ ;
return  $r$ .

```

We begin by noting that our simulator $\mathcal{S}$ issues at most λq queries to $\mathcal{G}_{N,m}$ and maintains three internal tables, each of size at most $6q$. This confirms that $\mathcal{S}$ is efficient. Next, we show that for any (even computationally unbounded) distinguisher $\mathcal{D}$, the views in the real world and ideal world with respect to $\mathcal{S}$ above are statistically close. Our approach proceeds via a sequence of hybrid games, in which each successive game introduces a small change in the way the system responds to oracle queries. The simulator $\mathcal{S}$ is ultimately defined by the system in the final game. Before describing the hybrid games, we specify the relevant parameters:

- There are six types of queries in each game: $(x; \mathcal{G}_{N,m}^{\text{label}})$, $(\text{str}_1, \text{str}_2; \mathcal{G}_{N,m}^{\text{add}})$, $(x; \text{ec-}\mathcal{G}_N^{\text{label}})$, $(P_1, P_2; \text{ec-}\mathcal{G}_N^{\text{add}})$, $(r; \text{Perm})$ and $(\text{str}; \text{Perm}^{\text{Inv}})$, where $x \in \mathbb{Z}_N$, $r, \text{str}, \text{str}_1, \text{str}_2 \in \{0, 1\}^m$ and $P_1, P_2 \in E$.
- The differentiator is restricted to at most q total queries, where q is an integer in $\text{poly}(\lambda)$.
- In the real world, the oracle responses are generated by the construction of sparse generic group $\tilde{\mathcal{G}}_{N,m} = (\tilde{\mathcal{G}}_{N,m}^{\text{label}}, \tilde{\mathcal{G}}_{N,m}^{\text{add}})$, the elliptic curve generic group $\text{ec-}\tilde{\mathcal{G}}_N = (\text{ec-}\tilde{\mathcal{G}}_N^{\text{label}}, \text{ec-}\tilde{\mathcal{G}}_N^{\text{add}})$, and a random permutation $(\widetilde{\text{Perm}}, \widetilde{\text{Perm}}^{\text{Inv}})$.
- In each game, the system's responses are denoted as $\mathcal{G}_{N,m}^{r\text{-label}}$, $\mathcal{G}_{N,m}^{r\text{-add}}$, $\text{ec-}\mathcal{G}_N^{r\text{-label}}$, $\text{ec-}\mathcal{G}_N^{r\text{-add}}$, Perm^r and $\text{Perm}^{r\text{-Inv}}$. For instance, $\text{Perm}^r(r)$ denotes the system's response to the query $\text{que} = (r; \text{Perm})$.

Denote by D_i the event that the differentiator outputs 1 in Game i . We now define the sequence of hybrid games.

Game 0. This game is identical to the real game, with the exception that the system maintains three tables, denoted as T_{label} , T_{add} and T_{Perm} . Specifically, the system responds to all queries in the same manner as in

the real world. For instance, $\text{ec-}\mathcal{G}_N^{\text{label}}(x) = \text{ec-}\tilde{\mathcal{G}}_N^{\text{label}}(x)$, $\text{ec-}\mathcal{G}_N^{\text{add}}(P_1, P_2) = \text{ec-}\tilde{\mathcal{G}}_N^{\text{add}}(P_1, P_2)$ and so on. The tables are maintained as follows:

- T_{label} : Initiated empty, and consists of tuples with the form (x, P, str) or $(\diamond, P, \text{str})$, where $\diamond$ represents an unknown discrete logarithm. When the differentiator makes a query $(x; \text{ec-}\mathcal{G}_N^{\text{label}})$ that does not yet exist in T_{label} , the system inserts $(x, \text{ec-}\tilde{\mathcal{G}}_N^{\text{label}}(x), \tilde{\mathcal{G}}_{N,m}^{\text{label}}(x))$ into T_{label} .
- T_{add} : Initiated empty, and consists of tuples with the form (P_1, P_2, P) . When the differentiator makes a query $(P_1, P_2; \text{ec-}\mathcal{G}_N^{\text{add}})$ that does not yet exist in T_{add} , the system inserts $(P_1, P_2, \text{ec-}\tilde{\mathcal{G}}_N^{\text{add}}(P_1, P_2))$ into T_{add} .
- T_{perm} : Initiated empty, and consists of tuples with the form (r, str) . When the differentiator makes a query $(r; \text{Perm})$ or $(\text{str}; \text{Perm}^{\text{Inv}})$ that does not yet exist in T_{perm} , the system inserts $(r, \widetilde{\text{Perm}}(r))$ or $(\widetilde{\text{Perm}}^{\text{Inv}}(\text{str}), \text{str})$ into T_{perm} , respectively.

It is important to note that all queries are answered by the real oracles, and the internal tables maintained by the system are entirely hidden from the differentiator. As a result, the differentiator's view in the real game is identical to that in Game 0. Therefore, we have

$$\Pr[\text{D}_{\text{Real}}] = \Pr[\text{D}_0]$$

Next, we introduce a new game in which the system responds to a subset of the queries using these tables. For clarity, we define a specific relationship between the queries and their corresponding tables. Specifically,

- If $\text{que} = (x; \text{ec-}\mathcal{G}_N^{\text{label}})$ is a labeling query, we say $\text{que} \in T_{\text{label}}$ if there exists a tuple $(t_1, t_2, t_3) \in T_{\text{label}}$ such that $t_1 = x$;
- If $\text{que} = (P_1, P_2; \text{ec-}\mathcal{G}_N^{\text{add}})$ is an addition query, we say $\text{que} \in T_{\text{add}}$ if there exists a tuple $(t_1, t_2, t_3) \in T_{\text{add}}$ such that $\{t_1, t_2\} = \{P_1, P_2\}$;
- If $\text{que} = (r; \text{Perm})$ is a permutation query, we say $\text{que} \in T_{\text{perm}}$ if there exists a tuple $(t_1, t_2) \in T_{\text{perm}}$ such that $t_1 = r$;
- If $\text{que} = (\text{str}; \text{Perm}^{\text{Inv}})$ is a permutation inverse query, we say $\text{que} \in T_{\text{perm}}$ if there exists a tuple $(t_1, t_2) \in T_{\text{perm}}$ such that $t_2 = \text{str}$.

Game 1. This game is identical to Game 0, with the exception of how the tables are maintained and how queries are responded to. Specifically,

Labeling query. Suppose $\text{que}_k = (x; \text{ec-}\mathcal{G}_N^{\text{label}})$. The system responds as follows:

- Case 1: If $x = 0$, the system responds with $\mathcal{O}$.
- Case 2: If $\text{que}_k \in T_{\text{label}}$, meaning there is a tuple $(t_1, t_2, t_3) \in T_{\text{label}}$ such that $t_1 = x$, the system responds with t_2 .
- Case 3: If $\text{que}_k \notin T_{\text{label}}$, the system first makes a query $(x; \mathcal{G}_{N,m}^{\text{label}})$ and receives a response str . Then:
 - Case 3.1: If there is a tuple $(t_1, t_2, t_3) \in T_{\text{label}}$ such that $t_3 = \text{str}$, the system responds with t_2 and replaces t_1 with x .
 - Case 3.2: Otherwise, the system responds with $P \leftarrow \text{ec-}\tilde{\mathcal{G}}_N^{\text{label}}(x)$, and inserts $(x, P, \text{str}), (N - x, -P, \text{str}^{N-1})$ into T_{label} . Furthermore, the system parses P into (u, b) , and inserts $(u||b||0^{\Delta_m}, \text{str}), (u||1-b||0^{\Delta_m}, \text{str}^{N-1})$ to T_{perm} .

Addition query. Suppose $\text{que}_k = (P_1, P_2; \text{ec-}\mathcal{G}_N^{\text{add}})$. The system responds as follows:

- Case 1: If $\text{que}_k \in T_{\text{add}}$, meaning there is a tuple $(t_1, t_2, t_3) \in T_{\text{add}}$ such that $\{t_1, t_2\} = \{P_1, P_2\}$, the system responds with t_3 .
- Case 2: If $P_1 = \mathcal{O}$ (or $P_2 = \mathcal{O}$), the system responds with P_2 (or P_1).
- Case 3: If there are two tuples $(t_1, t_2, t_3) \in T_{\text{label}}$ and $(t'_1, t'_2, t'_3) \in T_{\text{label}}$ such that $t_2 = P_1$ and $t'_2 = P_2$, the system makes a query $(t_3, t'_3; \mathcal{G}_{N,m}^{\text{add}})$ and receives a response str . Then:
 - Case 3.1: If there is a tuple $(t_1, t_2, t_3) \in T_{\text{label}}$ such that $t_3 = \text{str}$, the system responds with t_2 .
 - Case 3.2: Otherwise, the system responds with $P \leftarrow \text{ec-}\mathcal{G}_N^{\text{add}}(P_1, P_2)$, inserts the tuples $(\diamond, P, \text{str})$, $(\diamond, -P, \text{str}^{N-1})$ into T_{label} , and inserts the tuples (P_1, P_2, P) into T_{add} . Furthermore, the system parses P into (u, b) , and inserts $(u||b||0^{\Delta m}, \text{str})$, $(u||1-b||0^{\Delta m}, \text{str}^{N-1})$ to T_{Perm} .
- Case 4: Otherwise, if P_1 does not exist in T_{label} , the system parses P_1 into (u_1, b_1) , and makes the queries $(u_1||b_1||0^{\Delta m}; \text{Perm})$ and $(u_1||1-b_1||0^{\Delta m}; \text{Perm})$ with the responses str_1 and str'_1 , respectively. The system then inserts the tuples $(\diamond, P_1, \text{str}_1)$ and $(\diamond, -P_1, \text{str}'_1)$ into T_{label} , and inserts $(u_1||b_1||0^{\Delta m}, \text{str}_1)$ and $(u_1||1-b_1||0^{\Delta m}, \text{str}'_1)$ into T_{Perm} . Similarly, the system performs this for P_2 , and finally responds as in Case 2 or Case 3.

Permutation query. Suppose $\text{que}_k = (r; \text{Perm})$. The system responds as follows:

- Case 1: If $\text{que}_k \in T_{\text{Perm}}$, meaning there is a tuple $(t_1, t_2) \in T_{\text{Perm}}$ such that $t_1 = r$, the system responds with t_2 .
- Case 2: If $\text{que}_k \notin T_{\text{Perm}}$ and $r = u||b||0^{\Delta m}$ with $(u, b) \in E$, the system queries $(u||b||0^{\Delta m}; \text{Perm})$ and $(u||1-b||0^{\Delta m}; \text{Perm})$ with the responses str and str' , respectively. The system then responds with str and inserts $(u||b||0^{\Delta m}, \text{str})$ and $(u||1-b||0^{\Delta m}, \text{str}')$ into T_{Perm} . Additionally, the system inserts $(\diamond, P, \text{str})$ and $(\diamond, -P, \text{str}')$ into T_{label} .
- Case 3: Otherwise, the system responds with $\text{str} \leftarrow \widetilde{\text{Perm}}(r)$ and inserts (r, str) into T_{Perm} .

Permutation inverse query. Suppose $\text{que}_k = (\text{str}; \text{Perm}^{\text{Inv}})$. The system responds as follows:

- Case 1: If $\text{que}_k \in T_{\text{Perm}}$, meaning there is a tuple $(t_1, t_2) \in T_{\text{Perm}}$ such that $t_2 = \text{str}$, the system responds with t_1 .
- Case 2: If $\text{que}_k \notin T_{\text{Perm}}$ and $\tilde{\mathcal{G}}_{N,m}^{\text{add}}(\text{str}, \text{str}) \neq \perp$, the system responds with $r \leftarrow \widetilde{\text{Perm}}^{\text{Inv}}(\text{str})$, parses $r = u||b||0^{\Delta m}$, inserts $(u||b||0^{\Delta m}, \text{str})$, $(u||1-b||0^{\Delta m}, \text{str}^{N-1})$ into T_{Perm} , and inserts $(\diamond, P, \text{str})$, $(\diamond, -P, \text{str}^{N-1})$ into T_{label} .
- Case 3: Otherwise, the system responds with $r \leftarrow \widetilde{\text{Perm}}^{\text{Inv}}(\text{str})$ and inserts (r, str) into T_{Perm} .

Note that, for some adversarial queries in Game 1, the system responds solely by using the tables and accessing the honest interfaces. Each item stored in the tables is consistent with the real oracles. As a result, in both games, the responses to any query are identical, meaning that the view of the differentiator in Game 1 is the same as in Game 0. Therefore, we have

$$\Pr[\mathbf{D}_0] = \Pr[\mathbf{D}_1]$$

Now, the system still needs to answer the remaining adversarial queries using real oracles. In the subsequent hybrid games, we will gradually replace these responses with those generated from the tables and honest interfaces, ensuring that the adversary's view does not change significantly.

Game 2. This game is identical to Game 1, except in the way permutation queries are handled. Suppose $\text{que}_k = (r; \text{Perm})$. The system responds as follows:

- Case 1: If $\text{que}_k \in T_{\text{Perm}}$, same as in Game 1.
- Case 2: If $\text{que}_k \notin T_{\text{Perm}}$ and r can be parsed into $u||b||0$ with $(u, b) \in E$, the system randomly samples $x \in \mathbb{Z}_N$, responds with $\text{str} \leftarrow \tilde{\mathcal{G}}_{N,m}^{\text{label}}(x)$, inserts $(u||b||0^{\Delta m}, \text{str})$ and $(u||1-b||0^{\Delta m}, \text{str}^{N-1})$ into T_{Perm} , and inserts $(\diamond, P, \text{str})$ and $(\diamond, -P, \text{str}^{N-1})$ into T_{label} .
- Case 3: Otherwise, the system responds with a uniformly sampled $\text{str} \in \{0, 1\}^m$ such that $\tilde{\mathcal{G}}_{N,m}^{\text{add}}(\text{str}, \text{str}) = \perp$ via resampling (up to λ times), and inserts (r, str) into T_{Perm} .

Note that, the only difference between Game 1 and Game 2 occurs in Case 2 and Case 3, where r is previously unknown. In Case 2, $r = u||b||0^{\Delta m}$ with $(u, b) \in E$, meaning that $\text{Perm}^r(r)$ should be a valid group element of $\tilde{\mathcal{G}}_{N,m}$. In fact, the discrete log of each group element in $\tilde{\mathcal{G}}_{N,m}$ is independent of the differentiator's view. Thus, it is okay to replace $\text{Perm}^r(r)$ with $\tilde{\mathcal{G}}_{N,m}^{\text{label}}(x)$, where x is uniformly sampled from $\mathbb{Z}_N$, as long as x never appears in T_{label} . The probability of x -collision is bounded by

$$\Pr[\text{Bad}_1] \leq \frac{6q}{N}.$$

In Case 3 where P is invalid, by construction, $\text{Perm}^r(r)$ should be an invalid encoding in $\tilde{\mathcal{G}}_{N,m}$. Since all bits of $\text{Perm}^r(r)$ are independent of the differentiator's view, under the condition that no bad event occurs, the differentiator's view is identically in both games, i.e., $\text{Perm}^r(r)$ and the random string str . Hence it suffices to show that the bad event occurs only with negligible probability. There are two bad events: 1) str appears before que_k ; 2) str is a valid group element of $\tilde{\mathcal{G}}_{N,m}$. For str -collision event, since str is uniformly sampled from $\{0, 1\}^m$ and str is not a valid group element, we have

$$\Pr[\text{Bad}_2] \leq \frac{6q}{2^m - N} \leq \frac{6q}{2^m - 2^\lambda} \leq \frac{6q}{2^\lambda}.$$

For the second bad event, the probability is trivially bounded by

$$\Pr[\text{Bad}_3] \leq \left(\frac{N}{2^m}\right)^\lambda \leq \frac{1}{2^\lambda}.$$

Therefore, we have

$$|\Pr[\text{D}_1] - \Pr[\text{D}_2]| \leq q \cdot \sum_{i=1}^3 \Pr[\text{Bad}_i] \leq \frac{6q^2}{N} + \frac{6q^2}{2^\lambda} + \frac{q}{2^\lambda}.$$

Game 3. This game is identical to Game 2, except in the way permutation inverse queries are handled. Suppose $\text{que}_k = (\text{str}; \text{Perm}^{\text{Inv}})$. The system responds as follows:

- Case 1: If $\text{que}_k \in T_{\text{Perm}}$, same as in Game 2.
- Case 2: If $\text{que}_k \notin T_{\text{Perm}}$ and $\tilde{\mathcal{G}}_{N,m}^{\text{add}}(\text{str}, \text{str}) \neq \perp$, the system randomly samples $P := (u, b)$ from E , responds with $r \leftarrow u||b||0^{\Delta m}$, inserts $(u||b||0^{\Delta m}, \text{str})$ and $(u||1-b||0^{\Delta m}, \text{str}^{N-1})$ into T_{Perm} , and inserts $(\diamond, P, \text{str})$ and $(\diamond, -P, \text{str}^{N-1})$ into T_{label} .
- Case 3: Otherwise, the system uniformly samples r from $\{0, 1\}^m$, responds with r , and inserts (r, str) into T_{Perm} .

The differences between Game 2 and Game 3 occur in Case 2 and Case 3, where str is previously unknown. We now analyze each case individually.

In Case 2, str is a group element of $\mathcal{G}_{N,m}$. Therefore, its preimage under Perm should have the form $u||b||0^{\Delta m}$, where $P := (u, b)$ is a valid elliptic curve point in E and shares the same discrete logarithm as str , denoted as x . Since all bits of $\text{ec-}\tilde{\mathcal{G}}_{N,m}^{\text{label}}(x)$ are independent of the differentiator's view, it is okay to replace

$\text{ec-}\tilde{\mathcal{G}}_N^{\text{label}}(x)$ with a uniformly sampled P , as long as P has not appeared previously. The probability of this bad event is bounded by:

$$\Pr[\text{Bad}_1] \leq \frac{6q}{N}.$$

In Case 3, str is an invalid encoding. The system in Game 2 responds with $\widetilde{\text{Perm}}^{\text{Inv}}(\text{str})$, whereas in Game 3, the system responds with a random $r \in \{0, 1\}^m$. There are two bad events: 1) r takes the form of $u||b||0^{\Delta m}$ with $(u, b) \in E$; and 2) r appears in T_{Perm} . For the first bad event, the probability is

$$\Pr[\text{Bad}_2] \leq \frac{N}{2^m} \leq \frac{1}{2^{\omega(\log \lambda)}}.$$

For the second bad event, the probability is trivially bounded by

$$\Pr[\text{Bad}_3] \leq \frac{6q}{2^m - N} \leq \frac{6q}{2^\lambda}.$$

Therefore, we have

$$|\Pr[\text{D}_2] - \Pr[\text{D}_3]| \leq q \cdot \sum_{i=1}^3 \Pr[\text{Bad}_i] \leq \frac{6q^2}{N} + \frac{q}{2^{\omega(\log \lambda)}} + \frac{6q^2}{2^\lambda}.$$

Game 4. This game is identical to Game 3, except for the handling of labeling queries. Suppose $\text{que}_k = (x; \text{ec-}\mathcal{G}_N^{\text{label}})$. The system responds as follows:

- Case 1: If $x = 0$, same as in Game 3.
- Case 2: If $\text{que}_k \in T_{\text{label}}$, same as in Game 3.
- Case 3: If $\text{que}_k \notin T_{\text{label}}$, the system first makes a query $(x; \mathcal{G}_{N,m}^{\text{label}})$ and receives a response str . Then:
 - Case 3.1: If there is a tuple $(t_1, t_2, t_3) \in T_{\text{label}}$ such that $t_3 = \text{str}$, same as in Game 3.
 - Case 3.2: Otherwise, the system responds by uniformly sampling $P := (u, b)$ from E , inserts $(u||b||0^{\Delta m}, \text{str})$ and $(u||1-b||0^{\Delta m}, \text{str}^{N-1})$ into T_{Perm} , and inserts (x, P, str) and $(N-x, -P, \text{str}^{N-1})$ into T_{label} .

The only difference between Game 3 and Game 4 occurs in Case 3.2, where $\text{ec-}\tilde{\mathcal{G}}_N^{\text{label}}(x)$ has not appeared before que_k . Note that, the distribution of $\text{ec-}\tilde{\mathcal{G}}_N^{\text{label}}(x)$ is independent of the differentiator's view. Thus, under the condition that the bad event does not occur, the differentiator's view is distributed identically in both games. The bad event is that P appears before que_k , and this probability is trivially bounded by

$$\Pr[\text{Bad}] \leq \frac{6q}{N}.$$

Therefore, we have

$$|\Pr[\text{D}_3] - \Pr[\text{D}_4]| \leq q \cdot \Pr[\text{Bad}] \leq \frac{6q^2}{N}.$$

Game 5. This game is identical to Game 4, except for the handling of addition queries. Suppose $\text{que}_k = (P_1, P_2; \text{ec-}\mathcal{G}_N^{\text{add}})$. The system responds as follows:

- Case 1: If $\text{que}_k \in T_{\text{add}}$, same as in Game 4.
- Case 2: If $P_1 = \mathcal{O}$ (or $P_2 = \mathcal{O}$), same as in Game 4.

- Case 3: If there are two tuples $(t_1, t_2, t_3) \in T_{\text{label}}$ and $(t'_1, t'_2, t'_3) \in T_{\text{label}}$ such that $t_2 = P_1$ and $t'_2 = P_2$, the system makes a query $(t_3, t'_3; \mathcal{G}_{N,m}^{\text{add}})$ and receives a response str . Then:
 - Case 3.1: If there is a tuple $(t_1, t_2, t_3) \in T_{\text{label}}$ such that $t_3 = \text{str}$, same as in Game 4.
 - Case 3.2: Otherwise, the system responds by uniformly sampling $P := (u, b)$ from E , inserts $(u || b || 0^{\Delta m}, \text{str})$ and $(u || 1 - b || 0^{\Delta m}, \text{str}^{N-1})$ into T_{perm} , inserts $(\diamond, P, \text{str})$ and $(\diamond, -P, \text{str}^{N-1})$ into T_{label} , and inserts (P_1, P_2, P) into T_{add} .
- Case 4: Otherwise, if P_1 does not exist in T_{label} , the system uniformly samples $x_1 \in \mathbb{Z}_N$ as the preimage of P_1 , queries $(x_1; \mathcal{G}_{N,m}^{\text{label}})$ and $(N - x_1; \mathcal{G}_{N,m}^{\text{label}})$ with responses str_1 and str'_1 , respectively, inserts (x_1, P_1, str_1) , $(N - x_1, -P_1, \text{str}'_1)$ into T_{label} , and inserts $(P_1 || 0^{\Delta m}, \text{str}_1)$, $(-P_1 || 0^{\Delta m}, \text{str}'_1)$ into T_{perm} . The same procedure is followed for P_2 . Finally, the system responds as in Case 2 or Case 3.

The differences between Game 4 and Game 5 occur in Case 3.2 and Case 4. We now analyze each case individually.

In Case 3.2, where P_1 and P_2 are valid group elements, the system in Game 4 responds with $\text{ec-}\tilde{\mathcal{G}}_N^{\text{add}}(P_1, P_2)$; while in Game 5, the system responds with a uniformly sampled $P \in E$, and records the relationship between P and $\tilde{\mathcal{G}}_{N,m}^{\text{add}}(t_3, t'_3)$ in T_{perm} . As explained before, the differentiator's views of P and $\text{ec-}\tilde{\mathcal{G}}_N^{\text{add}}(P_1, P_2)$ are distributed identically, as long as the random string P has not appeared before. The probability of this bad event is bounded by

$$\Pr[\text{Bad}_1] \leq \frac{6q}{N}.$$

In Case 4, where at least one encoding of P_1 or P_2 has not appeared before que_k , we observe that, for any previously unknown $P_b \in \{P_1, P_2\}$, in both games, the system records its corresponding group element in $\tilde{\mathcal{G}}_{N,m}$ into T_{label} and T_{perm} . In Game 4, the system obtains the group element by calling $\widetilde{\text{Perm}}(P_b || 0^{\Delta m})$; whereas in Game 5, the system randomly samples $x_b \in \mathbb{Z}_N$ and then calls $\tilde{\mathcal{G}}_{N,m}^{\text{label}}(x_b)$. Analogously, the bad event in this situation is that x_b appears before que_k , which is bounded by

$$\Pr[\text{Bad}_2] \leq \frac{6q}{N}.$$

In addition, if the system finally responds to que_k in the same way as in Case 3.2, it introduces the bad event described above. Therefore, we have

$$\begin{aligned} |\Pr[\text{D}_4] - \Pr[\text{D}_5]| &\leq q \cdot (\Pr[\text{Bad}_1] + 2 \Pr[\text{Bad}_2]) \\ &\leq \frac{18q^2}{N}. \end{aligned}$$

Ideal Game. In Game 5, the queries to the adversarial interfaces are answered by the tables maintained by the system and by making queries to the honest interface. It is straightforward to demonstrate that we can replace $\tilde{\mathcal{G}}_{N,m}$ with a generic group $\mathcal{G}_{N,m}$, resulting in Ideal Game.

The difference between Game 5 and the Ideal Game is as follows: in Game 5, the system responds to all queries by calling $\tilde{\mathcal{G}}_{N,m}$; whereas, in the Ideal Game, the system makes queries to $\mathcal{G}_{N,m}$. In fact, since $\tilde{\mathcal{G}}_{N,m}^{\text{label}}(x) = \widetilde{\text{Perm}}(\text{ec-}\tilde{\mathcal{G}}_N^{\text{label}}(x) || 0^{\Delta m})$, where $\widetilde{\text{Perm}}$ is an ideal random permutation and $\text{ec-}\tilde{\mathcal{G}}_N$ is a generic group, all bits of $\tilde{\mathcal{G}}_{N,m}^{\text{label}}(x)$ are uniformly random. Analogously, if $\text{str}_1, \text{str}_2$ are valid group elements, all bits of $\tilde{\mathcal{G}}_{N,m}^{\text{add}}(\text{str}_1, \text{str}_2)$ are uniformly random. Thus, the distribution of $\tilde{\mathcal{G}}_{N,m}$ and $\mathcal{G}_{N,m}$ are identical, referring to

$$\Pr[\text{D}_5] = \Pr[\text{D}_{\text{Ideal}}].$$

We have shown that every pair of adjacent games are indistinguishable. Combining together, we establish the entire proof, yielding

$$\left| \Pr[\mathcal{D}_{\text{Real}}] - \Pr[\mathcal{D}_{\text{Ideal}}] \right| \leq \frac{36q^2}{N} + \frac{12q^2 + q}{2^\lambda} + \frac{q}{2^{\omega(\log \lambda)}}.$$

□
□

B.2 Sparse GGM does not computationally imply EC-GGM

Theorem B.2. *Let $\mathcal{G}_{N,m}$ be a sparse generic group model, and let $\text{ec-}\mathcal{G}_N$ be an elliptic curve generic group model over a point set E defined by the equation $y^2 = F(u)$ over $\mathbb{Z}_p$. Then, $\text{ec-}\mathcal{G}_N$ is computationally indifferentially separated from $\mathcal{G}_{N,m}$.*

Our strategy fundamentally builds on the analytical framework developed by Zhandry and Zhang [ZZ23]. To incorporate their approach into our proof, we formally define the Discrete Logarithm Identification (DLI) problem with respect to elliptic curve generic groups, as well as elliptic curve groups in the GGM.

Definition B.1 (DLI w.r.t. elliptic curve generic groups). *Let $\lambda \in \mathbb{N}$ be the security parameter, and let $\text{ec-}\mathcal{G}_N = (\text{ec-}\mathcal{G}_N^{\text{label}}, \text{ec-}\mathcal{G}_N^{\text{add}})$ be an elliptic curve generic group. An efficient algorithm with access to $\text{ec-}\mathcal{G}_N$, taking 1^λ and a group element $P \leftarrow \text{ec-}\mathcal{G}_N^{\text{label}}(x)$ as input, where $x \xleftarrow{\$} \mathbb{Z}_N$, outputs a “query-free” circuit C_{GEC} such that C_{GEC} identifies x with a good probability, specifically,*

- $\Pr[C_{\text{GEC}}(x) = 1] \geq \frac{1}{2}$;
- $\Pr[C_{\text{GEC}}(x') = 1 | x' \neq x] \leq \rho$, for any noticeable function ρ ,

where the probability is taken over the sampling of x' and the randomness of C_{GEC} . We say the DLI w.r.t. elliptic curve generic groups is hard if no query-efficient adversary can output such a circuit.

Definition B.2 (DLI w.r.t. elliptic curve groups in GGM). *Let $\lambda \in \mathbb{N}$ be the security parameter, and let $\mathcal{G}_{N,m}$ be a generic group model. Denote by $\Pi^{\mathcal{G}_{N,m}} = (\mathcal{L}^{\mathcal{G}_{N,m}}, \mathcal{A}^{\mathcal{G}_{N,m}})$ an elliptic curve group in $\mathcal{G}_{N,m}$. An efficient algorithm with access to $\mathcal{G}_{N,m}$, taking 1^λ and a group element $P \leftarrow \mathcal{L}^{\mathcal{G}_{N,m}}(x)$ as input, where $x \xleftarrow{\$} \mathbb{Z}_N$, outputs a “query-free” circuit $C_{\text{G-GEC}}$ such that $C_{\text{G-GEC}}$ identifies x with a good probability, specifically,*

- $\Pr[C_{\text{G-GEC}}(x) = 1] \geq \frac{2}{3}$;
- $\Pr[C_{\text{G-GEC}}(x') = 1 | x' \neq x] \leq \rho$, for any noticeable function ρ ,

where the probability is taken over the sampling of x' and the randomness of $C_{\text{G-GEC}}$. We say DLI w.r.t. elliptic curve groups in GGM is hard if no query-efficient adversary can output such a circuit.

Now, we are ready to establish the entire proof of Theorem B.2.

Proof. Let $\Pi^{\mathcal{G}_{N,m}} = (\mathcal{L}^{\mathcal{G}_{N,m}}, \mathcal{A}^{\mathcal{G}_{N,m}})$ be a construction of elliptic curve groups in a sparse GGM, with order N . Suppose $\Pi^{\mathcal{G}_{N,m}}$ is indifferentially separated from $\text{ec-}\mathcal{G}_N$. The argument goes in three steps:

1. DLI w.r.t. $\Pi^{\mathcal{G}_{N,m}}$ is easy.
2. If $\Pi^{\mathcal{G}_{N,m}}$ is indifferentially separated from $\text{ec-}\mathcal{G}_N$ and the DLI w.r.t. $\Pi^{\mathcal{G}_{N,m}}$ is easy, then the DLI w.r.t. $\text{ec-}\mathcal{G}_N$ is also easy.
3. Yet, DLI w.r.t. the elliptic curve generic group $\text{ec-}\mathcal{G}_N$ is hard.

These three steps lead to a contradiction, thereby disproving the initial statement that “ $\Pi^{\mathcal{G}_{N,m}}$ is indistinguishable from $\text{ec-}\mathcal{G}_N$ ”. This completes the proof. For clarity and readability, we will prove these steps in reverse order.

Lemma B.1. *DLI w.r.t. the elliptic curve generic group $\text{ec-}\mathcal{G}_N$ is hard.*

Proof. Assume that there exists an efficient adversary $\mathcal{A}$ making at most q queries to $\text{ec-}\mathcal{G}_N$ that breaks DLI. We will show that we can convert $\mathcal{A}$ into a query-efficient adversary $\mathcal{R}$, as shown in Figure 7, that breaks discrete logarithm problem in the EC-GGM.

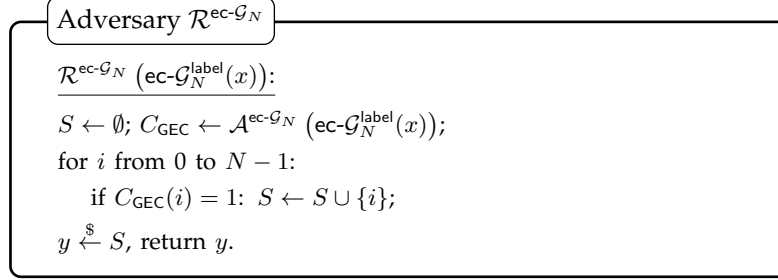


Figure 7: A query-efficient adversary breaks Discrete Logarithm in the EC-GGM

Note that $\mathcal{R}^{\text{ec-}\mathcal{G}_N}$ makes at most q queries to $\text{ec-}\mathcal{G}_N$. Define $t := \lambda \cdot q$. We set ρ to be $\frac{1}{t^{1/2}}$ and mbound to be $1 + \frac{N-1}{t^6}$. By applying Markov’s inequality, it is apparent that $\Pr[|S| > \text{mbound}] \leq \frac{1}{t^6}$. Therefore, we have

$$\begin{aligned} \Pr[\mathcal{R}^{\text{ec-}\mathcal{G}_N} \text{ outputs } x] &= \sum_{i=1}^N \Pr[|S| = i] \cdot \left(\frac{1}{i}\right) \cdot \Pr[x \in S \mid |S| = i] \\ &\geq \Pr[|S| \leq \text{mbound}] \cdot \left(\frac{1}{\text{mbound}}\right) \cdot \Pr[x \in S \mid |S| \leq \text{mbound}]. \end{aligned}$$

Note that $\Pr[x \in S] \geq 1/2$, which means that $\Pr[x \in S \mid |S| \leq \text{mbound}]$ should be close to $1/2$, because $|S| \leq \text{mbound}$ happens with high probability. Formally, we denote Event_1 and Event_2 to be the events “ $x \in S$ ” and “ $|S| \leq \text{mbound}$ ”, respectively, we have that

$$\begin{aligned} \Pr[\text{Event}_1 \mid \text{Event}_2] &= \frac{\Pr[\text{Event}_1] - \Pr[\text{Event}_1 \mid \neg \text{Event}_2] \Pr[\neg \text{Event}_2]}{\Pr[\text{Event}_2]} \\ &\geq \frac{\Pr[\text{Event}_1] - \Pr[\neg \text{Event}_2]}{\Pr[\text{Event}_2]} \\ &= 1 - \frac{1 - \Pr[\text{Event}_1]}{\Pr[\text{Event}_2]}. \end{aligned}$$

Substituting into the inequalities above, we have that

$$\begin{aligned} \Pr[\mathcal{R}^{\text{ec-}\mathcal{G}_N} \text{ outputs } x] &\geq \left(1 - \frac{1}{t^6}\right) \cdot \left(\frac{1}{1 + \frac{N-1}{t^6}}\right) \cdot \left(1 - \frac{1}{2} \cdot \left(1 - \frac{1}{t^6}\right)\right) \\ &\geq \frac{1}{2} \cdot \left(1 - \frac{1}{t^6}\right) \cdot \frac{t^6}{t^6 + N - 1} = \Theta\left(\frac{(\lambda q)^6}{N}\right) > O\left(\frac{(\lambda q)^2}{N}\right). \end{aligned}$$

This contradicts the established hardness of the discrete logarithm problem in the EC-GGM (see Lemma F.1 in Appendix F).

□
□

Lemma B.2. If $\Pi^{\mathcal{G}_{N,m}}$ is indifferntiable from $\text{ec-}\mathcal{G}_N$ and DLI w.r.t. $\Pi^{\mathcal{G}_{N,m}}$ is easy, then DLI w.r.t. $\text{ec-}\mathcal{G}_N$ is easy.

Proof. Let $\mathcal{A}^{\mathcal{G}_{N,m}}$ be an adversary that breaks the DLI with respect to $\Pi^{\mathcal{G}_{N,m}}$. Specifically, $\mathcal{A}^{\mathcal{G}_{N,m}}$ takes $\mathcal{L}^{\mathcal{G}_{N,m}}(x)$ as input, makes at most q queries to $\mathcal{G}_{N,m}$, and outputs a query-free circuit $C_{\text{G-GEC}}$ that satisfies the following two properties:

1. $C_{\text{G-GEC}}$ identifies x with good probability;
2. $C_{\text{G-GEC}}$ identifies x' ($x' \neq x$) with small probability.

Since $\Pi^{\mathcal{G}_{N,m}}$ is indifferntiable from $\text{ec-}\mathcal{G}_N$, there exists a simulator $\mathcal{S}$ that makes at most $q^* = \text{poly}(q)$ queries to $\text{ec-}\mathcal{G}_N$ and simulates the generic group $\mathcal{G}_{N,m}$ properly. Thus, we can leverage the simulator to transfer the adversary against DLI w.r.t. $\Pi^{\mathcal{G}_{N,m}}$ to an adversary against DLI w.r.t. $\text{ec-}\mathcal{G}_N$. That is, the adversary $\mathcal{A}^{(\mathcal{S}^{\text{ec-}\mathcal{G}_N})} = (\mathcal{A}^{\mathcal{S}})^{\text{ec-}\mathcal{G}_N}$ in the ideal world, taking $\text{ec-}\mathcal{G}_N^{\text{label}}(x)$ as input, outputs a query-free circuit C_{GEC} . By definition, we have that no efficient differentiator $\mathcal{D}$ can distinguish these two circuits, i.e., $C_{\text{G-GEC}}$ and C_{GEC} . Next, we claim that C_{GEC} also achieves the same two properties.

Property 1. In this part, we demonstrate that $\Pr[C_{\text{GEC}}(x) = 1] \geq \frac{1}{2}$. Assume toward a contradiction that $\Pr[C_{\text{GEC}}(x) = 1] < \frac{1}{2}$, then there exists an efficient differentiator, illustrated in Figure 8, that breaks indifferntiability.

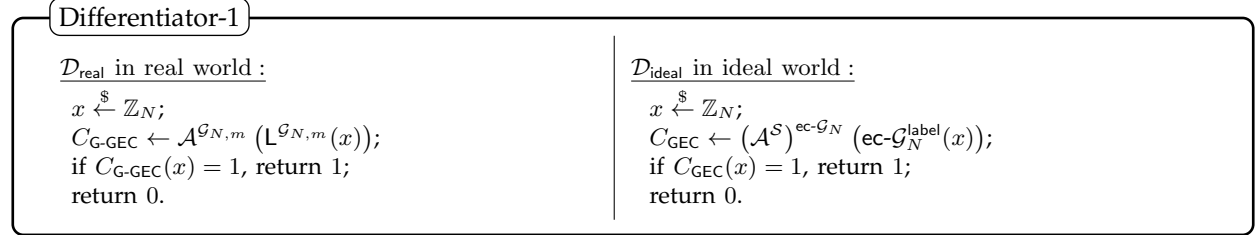


Figure 8: Differentiator-1 against Indifferntiability.

Note that $\Pr[\mathcal{D}_{\text{real}} = 1] \geq \frac{2}{3}$. However, if $\Pr[C_{\text{GEC}}(x) = 1] < \frac{1}{2}$, then $\Pr[\mathcal{D}_{\text{ideal}} = 1] < \frac{1}{2}$. This indicates that the differentiator in Figure 8 can break the indifferntiability with advantage at least $\frac{1}{6}$.

Property 2. In this part, we demonstrate that $\Pr_{x' \neq x}[C_{\text{GEC}}(x') = 1] \leq \rho$, for any noticeable function ρ . By contraposition, we assume that there exists a noticeable function ρ^* , such that $\Pr_{x' \neq x}[C_{\text{GEC}}(x') = 1] > \rho^*$. Then we proceed to construct an efficient differentiator that breaks indifferntiability. Let n be a polynomial such that $n \geq \frac{1}{\rho^*}$, we illustrate our differentiator in Figure 9.

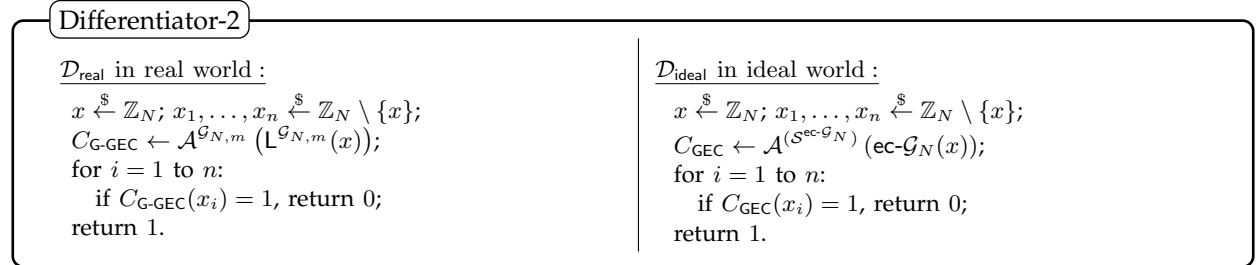


Figure 9: Differentiator-2 against Indifferntiability.

In the real world, by the statement of Lemma B.2, we have that for any noticeable function, say $\frac{0.1}{n}$, $\Pr_{x' \neq x}[C_{\text{G-GEC}}(x') = 1] \leq \frac{0.1}{n}$. Thus, by union bound, its apparent that

$$\Pr[\mathcal{D}_{\text{real}} = 1] \leq n \cdot \frac{0.1}{n} = 0.1.$$

On the other hand, if $\Pr_{x' \neq x}[C_{\text{GEC}}(x') = 1] > \rho^*$ in the ideal world, then

$$\begin{aligned} \Pr[\mathcal{D}_{\text{ideal}} = 1] &\geq 1 - (1 - \rho^*)^n \geq 1 - (1 - \rho^*)^{\frac{1}{\rho^*}} \\ &\geq 1 - \frac{1}{e} > 0.6. \end{aligned}$$

Therefore, the advantage of the differentiator in Figure 9 is at least $(0.6 - 0.1) \geq 0.5$.

Hence, if $\Pi^{\mathcal{G}_{N,m}}$ is indifferentiable from $\text{ec-}\mathcal{G}_N$ and DLI w.r.t. $\Pi^{\mathcal{G}_{N,m}}$ is easy, then DLI w.r.t. $\text{ec-}\mathcal{G}_N$ is easy. $\square$

$\square$

Adversary $\mathcal{A}^{\mathcal{G}_{N,m}}$

```

 $\mathcal{A}^{\mathcal{G}_{N,m}}(\mathbf{L}^{\mathcal{G}_{N,m}}(x))$ :
 $S_{\text{que-res}} \leftarrow \emptyset$ ;  $r, r_1, \dots, r_n \xleftarrow{\$} \mathbb{Z}_N$ ;
for  $i = 1$  to  $n$ : //collect frequent queries
     $\{(\text{que}_j, \text{res}_j)\}_{j=1}^q \xleftarrow{\text{query}} \mathbf{L}^{\mathcal{G}_{N,m}}(r_i)$ ,  $S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup \{(\text{que}_j, \text{res}_j)\}_{j=1}^q$ ;
     $\{(\text{que}_j, \text{res}_j)\}_{j=1}^q \xleftarrow{\text{query}} \mathbf{L}^{\mathcal{G}_{N,m}}(r)$ ,  $S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup \{(\text{que}_j, \text{res}_j)\}_{j=1}^q$ ;
     $\{(\text{que}_j, \text{res}_j)\}_{j=1}^q \xleftarrow{\text{query}} \mathbf{L}^{\mathcal{G}_{N,m}}(-r)$ ,  $S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup \{(\text{que}_j, \text{res}_j)\}_{j=1}^q$ ;
     $\{(\text{que}_j^{(x,-r)}, \text{res}_j^{(x,-r)})\}_{j=1}^q \xleftarrow{\text{query}} \mathbf{A}^{\mathcal{G}_{N,m}}(\mathbf{L}^{\mathcal{G}_{N,m}}(x), \mathbf{L}^{\mathcal{G}_{N,m}}(-r))$ ;
     $\{(\text{que}_j^{(x-r,r)}, \text{res}_j^{(x-r,r)})\}_{j=1}^q \xleftarrow{\text{query}} \mathbf{A}^{\mathcal{G}_{N,m}}(\mathbf{L}^{\mathcal{G}_{N,m}}(x-r), \mathbf{L}^{\mathcal{G}_{N,m}}(r))$ ;
for  $z \in \{(x, -r), (x-r, r)\}$  and  $j$  from 1 to  $q$ : //collect queries in group addition
    if  $\text{que}_j^z$  is a labeling query:  $S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup \{(\text{que}_j^z, \text{res}_j^z)\}$ ;
    if  $\text{que}_j^z = (\text{str}_1, \text{str}_2)$  is an addition query:
        if  $\exists (x_1, \text{str}_1), (x_2, \text{str}_2) \in S_{\text{que-res}}$ :  $S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup \{(x_1 + x_2, \text{res}_j^z)\}$ ;
return  $C_{\text{G-GEC}}(\cdot, S_{\text{que-res}}, \mathbf{L}^{\mathcal{G}_{N,m}}(x))$ .
 $C_{\text{G-GEC}}(\cdot, S_{\text{que-res}}, \mathbf{L}^{\mathcal{G}_{N,m}}(x))$ :
take  $\hat{x} \in \mathbb{Z}_N$  as input, run  $\text{str} \leftarrow \mathbf{L}^{S_{\text{que-res}}}(\hat{x})$ ;
when  $\mathbf{L}$  makes a labeling query  $\text{que}$ : //respond with  $S$ 
    if  $\exists (\text{que}, \text{str}) \in S_{\text{que-res}}$ : respond with  $\text{str}$ ;
    else: respond with a uniformly sampled  $\text{str}$ ;  $S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup \{(\text{que}, \text{str})\}$ ;
if  $\widehat{\text{str}} = \mathbf{L}^{\mathcal{G}_{N,m}}(x)$ : return 1; else: return 0.

```

Figure 10: Efficient Adversary $\mathcal{A}^{\mathcal{G}_{N,m}}$ and query-free circuit $C_{\text{G-GEC}}$ w.r.t. $\Pi^{\mathcal{G}_{N,m}}$.

Lemma B.3. *DLI w.r.t. $\Pi^{\mathcal{G}_{N,m}}$ is easy.*

Proof. By the definition of indifferentiability, the algorithms $\mathbf{L}^{\mathcal{G}_{N,m}}$ and $\mathbf{A}^{\mathcal{G}_{N,m}}$ are deterministic; and they shall support group operations correctly with high probability. More specifically, there exists a negligible function ϵ_1 , such that

$$\Pr[\mathbf{L}^{\mathcal{G}_{N,m}}(x) = \mathbf{L}^{\mathcal{G}_{N,m}}(y) \mid x \neq y \in \mathbb{Z}_N] \leq \epsilon_1, \quad (1)$$

$$\Pr[\mathbf{A}^{\mathcal{G}_{N,m}}(\mathbf{L}^{\mathcal{G}_{N,m}}(x), \mathbf{L}^{\mathcal{G}_{N,m}}(y)) = \mathbf{L}^{\mathcal{G}_{N,m}}(x+y)] \geq 1 - \epsilon_1, \quad (2)$$

where the probability is over the sampling of $x, y \in \mathbb{Z}_N$ and the generic group $\mathcal{G}_{N,m}$. As explained above, we stress that $L^{\mathcal{G}_{N,m}}$ only makes labeling queries. Let q be an integer in $\text{poly}(\lambda)$. We assume that both $L^{\mathcal{G}_{N,m}}$ and $A^{\mathcal{G}_{N,m}}$ make at most q queries to $\mathcal{G}_{N,m}$. Next, we prove that the DLI w.r.t. $\Pi^{\mathcal{G}_{N,m}}$ is easy by constructing an efficient adversary $\mathcal{A}$ and a query-free circuit $C_{\text{G-GEC}}$, illustrated in Figure 10.

We first clarify some undefined notions. Let n be a sufficiently large integer to be specified below. By $\{(\text{que}_j, \text{res}_j)\}_{j=1}^q \xrightarrow{\text{query}} L^{\mathcal{G}_{N,m}}(r)$, we denote that on input r , the algorithm $L^{\mathcal{G}_{N,m}}(r)$ makes q queries to $\mathcal{G}_{N,m}$, with the corresponding responses; similar for the algorithm $A^{\mathcal{G}_{N,m}}(L^{\mathcal{G}_{N,m}}(x), L^{\mathcal{G}_{N,m}}(y))$ ¹⁴. Given an input $\hat{x} \in \mathbb{Z}_N$, the query-free circuit $C_{\text{G-GEC}}$ runs algorithm $L^{\mathcal{G}_{N,m}}(\hat{x})$ except for replacing the querying oracle by looking up the table $S_{\text{que-res}}$ (and lazy sampling); we denote that as $L^{S_{\text{que-res}}}$.

The hope is that by the time $\mathcal{A}$ finishes, S will contain all “relevant” queries, such that $C_{\text{G-GEC}}$ satisfies the two desired properties:

1. $\Pr[C_{\text{G-GEC}}(x) = 1] \geq \frac{2}{3}$;
2. $\Pr[C_{\text{G-GEC}}(x') = 1 | x' \neq x] \leq \rho$, for any noticeable function ρ .

We next prove this one by one.

Property 1. Here, we prove that $\Pr[C_{\text{G-GEC}}(x) = 1] \geq \frac{2}{3}$. As shown in Figure 10, $C_{\text{G-GEC}}(x)$ outputs 1 if and only if $L^{S_{\text{que-res}}}(x) = L^{\mathcal{G}_{N,m}}(x)$. For any $x, y \in \mathbb{Z}_N$, let $S_{x,y}$ be the set of all group elements appeared in responses when invoking $A^{\mathcal{G}_{N,m}}(L^{\mathcal{G}_{N,m}}(x), L^{\mathcal{G}_{N,m}}(y))$. We denote

$$Q_x := \{(\text{que}_j, \text{res}_j)\}_{j=1}^q \xrightarrow{\text{query}} L^{\mathcal{G}_{N,m}}(x);$$

$$Q_{x,y} := \{(\text{que}, \text{res}) | \text{res} \in S_{x,y}, \mathcal{G}_{N,m}^{\text{label}}(\text{que}) = \text{res}\}.$$

Obviously, if $Q_x \subseteq S_{\text{que-res}}$, then $L^{S_{\text{que-res}}}(x) = L^{\mathcal{G}_{N,m}}(x)$, referring to $C_{\text{G-GEC}}(x) = 1$. Unfortunately, x is independent of the adversary’s view and some queries in Q_x may be completely ignored when running $L^{\mathcal{G}_{N,m}}(x)$. Thus we cannot guarantee $Q_x \subseteq S_{\text{que-res}}$. However, suppose that the label on x depends on res for some query-response (que, res) in Q_x . Since $A^{\mathcal{G}_{N,m}}$ takes as input the labels for $x - r$, r will also output the label on x , which indicates that $L^{\mathcal{G}_{N,m}}$ must somehow recover res . In other words, either $A^{\mathcal{G}_{N,m}}(L^{\mathcal{G}_{N,m}}(x - r), L^{\mathcal{G}_{N,m}}(r))$ or $L^{\mathcal{G}_{N,m}}(x - r)$ and $L^{\mathcal{G}_{N,m}}(r)$ must make query to get res . Thus, we argue that the only queries we care about are actually $Q_x \cap (Q_r \cup Q_{x-r} \cup Q_{x-r,r})$, denoted as Q_x^* , and it suffices to prove that $Q_x^* \subseteq S_{\text{que-res}}$ with a good probability. Next, we prove it by two steps as follows:

- $Q_x \cap Q_{x-r,r} \subseteq S_{\text{que-res}}$ with a good probability;
- $Q_x \cap (Q_r \cup Q_{x-r}) \subseteq S_{\text{que-res}}$ with a good probability.

$Q_x \cap Q_{x-r,r} \subseteq S_{\text{que-res}}$ with a good probability. In fact, for each $(\text{que}, \text{res}) \in Q_x \cap Q_{x-r,r}$, there are three cases during the run of $A^{\mathcal{G}_{N,m}}(L^{\mathcal{G}_{N,m}}(x - r), L^{\mathcal{G}_{N,m}}(r))$:

- Case 1: res is the response of a labeling query;
- Case 2: res is the response of an addition query, which can be converted into a labeling query;
- Case 3: res is the response of an addition query, which cannot be converted into a labeling query.

In the former two cases, as shown in Figure 10, we know that $(\text{que}, \text{res}) \in S_{\text{que-res}}$ absolutely. The Case 3 indicates that the algorithm $A^{\mathcal{G}_{N,m}}$ generates valid group elements res^* which does not exist in $S_{\text{que-res}}$. Since $S_{\text{que-res}}$ collects all the labeling queries and addition queries that can be converted into labeling queries during the run of $\mathcal{A}$, it follows that res^* must be a new group element. Therefore, let Bad denote the event in which Case 3 occurs during the run of $\mathcal{A}^{\mathcal{G}_{N,m}}$. We can bound the probability of Bad by constructing an extractor $\mathcal{E}$, as illustrated in Figure 11.

¹⁴Here, we abuse the notation $L^{\mathcal{G}_{N,m}}(x)$ as both the group element and the labeling operation on x .

Extractor $\mathcal{E}^{\mathcal{G}_{N,m}}(1^\lambda)$

```

 $\mathcal{L}^{\mathcal{G}_{N,m}}(x) \xleftarrow{\$} E;$ 

 $S_{\text{que-res}} \leftarrow \emptyset; r, r_1, \dots, r_n \xleftarrow{\$} \mathbb{Z}_N;$ 

for  $i$  from 1 to  $n$ : //collect frequent queries
     $\{(\text{que}_j, \text{res}_j)\}_{j=1}^q \xleftarrow{\text{query}} \mathcal{L}^{\mathcal{G}_{N,m}}(r_i), S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup \{(\text{que}_j, \text{res}_j)\}_{j=1}^q;$ 

 $\{(\text{que}_j, \text{res}_j)\}_{j=1}^q \xleftarrow{\text{query}} \mathcal{L}^{\mathcal{G}_{N,m}}(r), S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup \{(\text{que}_j, \text{res}_j)\}_{j=1}^q;$ 

 $\{(\text{que}_j, \text{res}_j)\}_{j=1}^q \xleftarrow{\text{query}} \mathcal{L}^{\mathcal{G}_{N,m}}(-r), S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup \{(\text{que}_j, \text{res}_j)\}_{j=1}^q;$ 

 $\{(\text{que}_j^{(x,-r)}, \text{res}_j^{(x,-r)})\}_{j=1}^q \xleftarrow{\text{query}} \mathcal{A}^{\mathcal{G}_{N,m}}(\mathcal{L}^{\mathcal{G}_{N,m}}(x), \mathcal{L}^{\mathcal{G}_{N,m}}(-r));$ 

 $\{(\text{que}_j^{(x-r,r)}, \text{res}_j^{(x-r,r)})\}_{j=1}^q \xleftarrow{\text{query}} \mathcal{A}^{\mathcal{G}_{N,m}}(\mathcal{L}^{\mathcal{G}_{N,m}}(x-r), \mathcal{L}^{\mathcal{G}_{N,m}}(r));$ 

for  $z \in \{(x, -r), (x-r, r)\}$  and  $j$  from 1 to  $q$ : //collect queries in group addition
    if  $\text{que}_j^z$  is a labeling query:  $S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup \{(\text{que}_j^z, \text{res}_j^z)\};$ 
    if  $\text{que}_j^z = (\text{str}_1, \text{str}_2)$  is an addition query:
        if  $\exists (x_1, \text{str}_1), (x_2, \text{str}_2) \in S_{\text{que-res}}: S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup \{(x_1 + x_2, \text{res}_j^z)\};$ 
        if  $\text{str}_1 \notin S_{\text{que-res}}$  and  $\mathcal{G}_{N,m}^{\text{add}}(\text{str}_1, \text{str}_1) \neq \perp$ , return  $\text{str}_1$ ;
        if  $\text{str}_2 \notin S_{\text{que-res}}$  and  $\mathcal{G}_{N,m}^{\text{add}}(\text{str}_2, \text{str}_2) \neq \perp$ , return  $\text{str}_2$ ;

return  $\perp$ .

```

Figure 11: The description of the extractor $\mathcal{E}^{\mathcal{G}_{N,m}}$ that outputs a new group element.

Note that if Bad occurs during the run of $\mathcal{A}^{\mathcal{G}_{N,m}}(\text{label})$, where label is uniform sampled from the set E of all elliptic curve points, $\mathcal{E}$ can successfully extract such a new group element. Based on Lemma G.1, we have

$$\Pr[\text{Bad occurs in } \mathcal{A}^{\mathcal{G}_{N,m}}(\text{label})] \leq \epsilon_2(\lambda),$$

where ϵ_2 is a negligible function, and the probability is taken over the sampling of the sparse GGM instance $\mathcal{G}_{N,m}$, the group encoding label $\in E$, and the internal randomness of the adversary $\mathcal{A}$. By the definition of indistinguishability, there exists a negligible function ϵ_3 , such that the distributions of $\{\mathcal{L}^{\mathcal{G}_{N,m}}(x)\}_{x \in \mathbb{Z}_N}$ and $\{\text{label}\}_{\text{label} \in E}$ are ϵ_3 -computationally indistinguishable. Therefore, for uniformly chosen GGM instance $\mathcal{G}_{N,m}$ and the value $x \in \mathbb{Z}_N$, we have

$$\Pr[\text{Bad occurs in } \mathcal{A}^{\mathcal{G}_{N,m}}(\mathcal{L}^{\mathcal{G}_{N,m}}(x))] \leq \epsilon_2(\lambda) + \epsilon_3(\lambda).$$

Consequently, we can conclude that

$$\Pr[Q_x \cap Q_{x-r,r} \subseteq S_{\text{que-res}}] \geq 1 - \epsilon_2(\lambda) - \epsilon_3(\lambda).$$

$Q_x \cap (Q_r \cup Q_{x-r}) \subseteq S_{\text{que-res}}$ with a good probability. Note that $Q_r \subseteq S_{\text{que-res}}$, therefore it is enough to show that $Q_x \cap Q_{x-r} \subseteq S_{\text{que-res}}$. However, that fact is that $x-r$ is unknown to $\mathcal{A}$. To resolve this barrier, we show

that with high probability the queries in $Q_x \cap Q_{x-r}$ are all frequent queries (defined below). Thus, we can draw sufficiently large number of samples, say $r_1, r_2, \dots, r_n$, record all the queries made in $\mathcal{L}^{\mathcal{G}_{N,m}}(r_i)$, and capture all the frequent queries. In other words, we show that

$$Q_x \cap Q_{x-r} \subseteq Q_{r_1} \cup \dots \cup Q_{r_n} \subseteq S_{\text{que-res}}$$

with high probability. Now we define the frequent group elements as follows: let $t := 200q$, we say a labeling query (que, res) and the group element res are frequent, if

$$\Pr[(\text{que}, \text{res}) \in Q_r : r \xleftarrow{\$} \mathbb{Z}_N] \geq \frac{1}{t}.$$

Since both x and r are uniformly sampled, $x - r$ and x are independent. For any labeling query (que, res) $\in Q_{x-r}$, if (que, res) is not frequent, then

$$\Pr[(\text{que}, \text{res}) \in Q_x] \leq \frac{1}{t}.$$

Thus, we have that

$$\Pr[Q_x \cap Q_{x-r} \text{ are all frequent queries}] \geq 1 - \frac{q}{t} = 1 - \frac{1}{200}.$$

Let $n := t^2 \cdot q$. Below, we prove that $Q_{r_1} \cup \dots \cup Q_{r_n}$ contains all frequent queries with a noticeable probability. In fact, there are at most $q_{\text{fre}} := q \cdot t$ frequent queries. Denote all frequency queries as $\{(\text{que}_{\text{fre-}i}, \text{res}_{\text{fre-}i})\}_{i \in [q_{\text{fre}}]}$. For each $(\text{que}_{\text{fre-}i}, \text{res}_{\text{fre-}i})$, we have

$$\Pr[(\text{que}_{\text{fre-}i}, \text{res}_{\text{fre-}i}) \notin Q_{r_1} \cup \dots \cup Q_{r_n}] \leq \left(1 - \frac{1}{t}\right)^n \leq e^{-q \cdot t},$$

which refers to

$$\Pr[(\text{que}_{\text{fre-}i}, \text{res}_{\text{fre-}i}) \in Q_{r_1} \cup \dots \cup Q_{r_n} : \forall i \in [q_{\text{fre}}]] \geq 1 - (q \cdot t)e^{-q \cdot t} \geq 1 - 200e^{-200}.$$

Therefore, we have that

$$\Pr[Q_x \cap (Q_r \cup Q_{x-r}) \subseteq S_{\text{que-res}}] \geq 1 - \frac{1}{200} - 200e^{-200}.$$

Combining together, we have that

$$\Pr[Q_x^* \subseteq S_{\text{que-res}}] \geq 1 - \frac{1}{200} - 200e^{-200} - \text{negl} \geq \frac{2}{3},$$

which refers to $\Pr[C_{\text{G-GEC}}(x) = 1] \geq \frac{2}{3}$.

Property 2. Here, we prove that $\Pr_{x' \neq x}[C_{\text{G-GEC}}(x') = 1] \leq \rho$, for any noticeable function ρ . Recall that, $C_{\text{G-GEC}}(x')$ outputs 1 if and only if $\mathcal{L}^{S_{\text{que-res}}}(x') = \mathcal{L}^{\mathcal{G}_{N,m}}(x)$. There are two cases as follows:

- Case 1: $Q_{x'} \subseteq S_{\text{que-res}}$ and $\mathcal{L}^{S_{\text{que-res}}}(x') = \mathcal{L}^{\mathcal{G}_{N,m}}(x)$;
- Case 2: $Q_{x'} \not\subseteq S_{\text{que-res}}$ and $\mathcal{L}^{S_{\text{que-res}}}(x') = \mathcal{L}^{\mathcal{G}_{N,m}}(x)$.

Immediately observe that if $Q_{x'} \subseteq S_{\text{que-res}}$, then $\mathcal{L}^{S_{\text{que-res}}}(x') = \mathcal{L}^{\mathcal{G}_{N,m}}(x')$. According to Equation 1, we have

$$\Pr[\text{Case 1}] \leq \epsilon_1.$$

For Case 2, since $Q_{x'} \not\subseteq S_{\text{que-res}}$, it is unnecessary that $\mathcal{L}^{S_{\text{que-res}}}(x') = \mathcal{L}^{\mathcal{G}_{N,m}}(x')$. Thus, we first define an event “ $S_{\text{que-res}}$ is good”, if $\Pr_{x' \neq x}[\mathcal{L}^{S_{\text{que-res}}}(x') = \mathcal{L}^{S_{\text{que-res}}}(x)] \leq \sqrt{\epsilon_1}$, where the probability is only over the sampling of x and x' ; and otherwise we say “ $S_{\text{que-res}}$ is bad”. When $S_{\text{que-res}}$ is good, for any $\text{str} \in \{0, 1\}^{m_1}$, we have that

$\Pr[\mathbf{L}^{S_{\text{que-res}}}(x') = \text{str}] \leq \sqrt[4]{\epsilon_1}$. Since S simulates a well-distributed GGM (via lazy sampling), by Equation 1, we have

$$\begin{aligned} & \Pr_{x' \neq x} [\mathbf{L}^{S_{\text{que-res}}}(x') = \mathbf{L}^{S_{\text{que-res}}}(x) | S_{\text{que-res}} \text{ is good}] \cdot \Pr[S_{\text{que-res}} \text{ is good}] \\ & + \Pr_{x' \neq x} [\mathbf{L}^{S_{\text{que-res}}}(x') = \mathbf{L}^{S_{\text{que-res}}}(x) | S_{\text{que-res}} \text{ is bad}] \cdot \Pr[S_{\text{que-res}} \text{ is bad}] \leq \epsilon_1 \end{aligned}$$

It's apparent that $\Pr[S_{\text{que-res}} \text{ is bad}] \leq \sqrt{\epsilon_1}$. Therefore,

$$\begin{aligned} & \Pr[\mathbf{L}^{S_{\text{que-res}}}(x') = \mathbf{L}^{\mathcal{G}_{N,m}}(x)] \\ & = \Pr[\mathbf{L}^{S_{\text{que-res}}}(x') = \mathbf{L}^{\mathcal{G}_{N,m}}(x) | S_{\text{que-res}} \text{ is good}] \cdot \Pr[S_{\text{que-res}} \text{ is good}] \\ & \quad + \Pr[\mathbf{L}^{S_{\text{que-res}}}(x') = \mathbf{L}^{\mathcal{G}_{N,m}}(x) | S_{\text{que-res}} \text{ is bad}] \cdot \Pr[S_{\text{que-res}} \text{ is bad}] \\ & \leq \sqrt[4]{\epsilon_1} + \sqrt{\epsilon_1}, \end{aligned}$$

which refers to $\Pr[\text{Case 2}] \leq \sqrt[4]{\epsilon_1} + \sqrt{\epsilon_1}$.

Combining together, we obtain

$$\Pr[C_{\text{G-GEC}}(x') = 1] \leq \epsilon_1 + \sqrt[4]{\epsilon_1} + \sqrt{\epsilon_1}$$

which means that for any noticeable function ρ , $\Pr_{x' \neq x}[C_{\text{G-GEC}}(x') = 1] \leq \rho$. □ □

Combining together, we establish the entire proof. □ □

C Groups with admissible encodings in EC-GGM

In this section, we demonstrate the existence of CDH-secure groups with nice admissible encodings in EC-GGM. Since EC-GGM inherently implies a generic elliptic curve group, which admit nice admissible encodings (e.g. Icart's map [Ica09]), it suffices to prove that the CDH problem in EC-GGM is hard.

Theorem C.1. *Let $\text{ec-}\mathcal{G}_N$ be an elliptic curve generic group model over a point set E defined by the equation $y^2 = F(u)$ over $\mathbb{Z}_p$. The computational Diffie-Hellman problem in $\text{ec-}\mathcal{G}_N$ is hard.*

Proof. We apply contraposition and let $\mathcal{A}$ be an adversary that breaks CDH in $\text{ec-}\mathcal{G}_N$. Specifically, $\mathcal{A}$ takes $\text{ec-}\mathcal{G}_N^{\text{label}}(x)$ and $\text{ec-}\mathcal{G}_N^{\text{label}}(y)$ as inputs for randomly chosen $x, y \in \mathbb{Z}_N$, makes at most q queries to $\text{ec-}\mathcal{G}_N$, and outputs the group element $\text{ec-}\mathcal{G}_N^{\text{label}}(x \cdot y)$ with a non-negligible probability ρ . We then leverage the indistinguishability to construct an adversary against CDH w.r.t. GGM, maintaining almost the same winning probability.

As shown in Appendix B.1, there exists a construction of sparse generic group from EC-GGM, denoted as $\Pi^{\text{Orac}} = (\mathbf{L}^{\text{Orac}}, \mathbf{A}^{\text{Orac}})$, which is indistinguishable from $\mathcal{G}_{N,m}$. Here, Orac is a tuple $(\text{ec-}\mathcal{G}_N, \text{Perm}, \text{Perm}^{\text{Inv}})$, with $(\text{Perm}, \text{Perm}^{\text{Inv}})$ being a random permutation. The construction is shown in Figure 6. Therefore, we can transform the adversary $\mathcal{A}$ into an adversary $\mathcal{R}$ against CDH w.r.t. Π^{Orac} . The adversary $\mathcal{R}$ is illustrated in Figure 12. Trivial to note that, $\mathcal{R}^{\text{Orac}}$ can win the CDH game w.r.t. Π^{Orac} with probability at least ρ .

Since Π^{Orac} is indistinguishable from $\mathcal{G}_{N,m}$, there exists a simulator $\mathcal{S}$ that makes at most $q^* = \text{poly}(q)$ queries to $\mathcal{G}_{N,m}$ and correctly simulates $\text{ec-}\mathcal{G}_N$. Thus, we can leverage the simulator to transform the adversary against CDH w.r.t. Π^{Orac} into an adversary against CDH w.r.t. $\mathcal{G}_{N,m}$. Specifically, the adversary $\mathcal{R}^{S^{\mathcal{G}_{N,m}}} = (\mathcal{R}^S)^{\mathcal{G}_{N,m}}$ in the ideal world, takes $\mathcal{G}_{N,m}^{\text{label}}(x)$ and $\mathcal{G}_{N,m}^{\text{label}}(y)$ as inputs for randomly chosen $x, y \in \mathbb{Z}_N$, and outputs a string $\text{str} \in \{0, 1\}^m$. We claim that $\mathcal{R}^S$ can break CDH problem against $\mathcal{G}_{N,m}$. By contraposition, we assume that the probability of $\mathcal{R}^{S^{\mathcal{G}_{N,m}}}(\mathcal{G}_{N,m}^{\text{label}}(x), \mathcal{G}_{N,m}^{\text{label}}(y))$ outputting the group encoding $\mathcal{G}_{N,m}^{\text{label}}(x \cdot y)$ is negligible. We then proceed to construct a differentiator, as illustrated in Figure 13.

Obviously note that $\Pr[\mathcal{D}_{\text{real}} = 1] \geq \rho$. If $\Pr[\mathcal{D}_{\text{ideal}} = 1] \leq \text{negl}(\lambda)$, then the differentiator in Figure 13 can differentiate between the real world and the ideal world with an advantage of at least $\rho - \text{negl}(\lambda)$, which contradicts Theorem B.1. Thus, it follows that $\mathcal{R}^S$ can break the CDH problem against $\mathcal{G}_{N,m}$, which contradicts the hardness of CDH problem in the GGM. Therefore, we conclude that CDH problem is hard in $\text{ec-}\mathcal{G}_N$. □ □

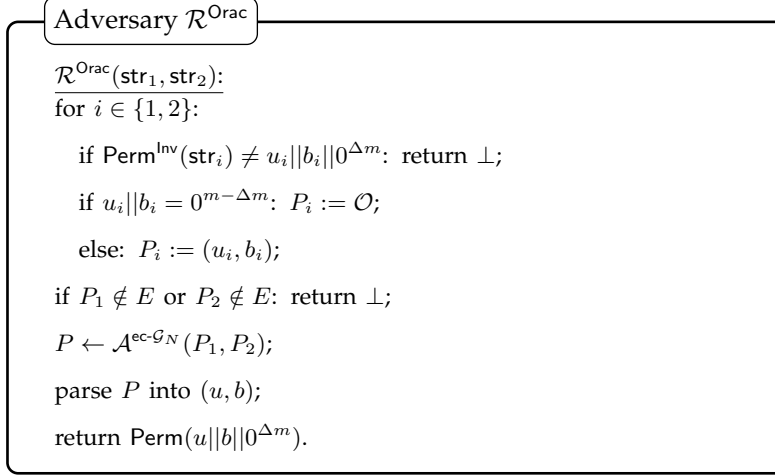


Figure 12: The description of the adversary $\mathcal{R}^{\text{Orac}}$ against CDH w.r.t. Π^{Orac} .

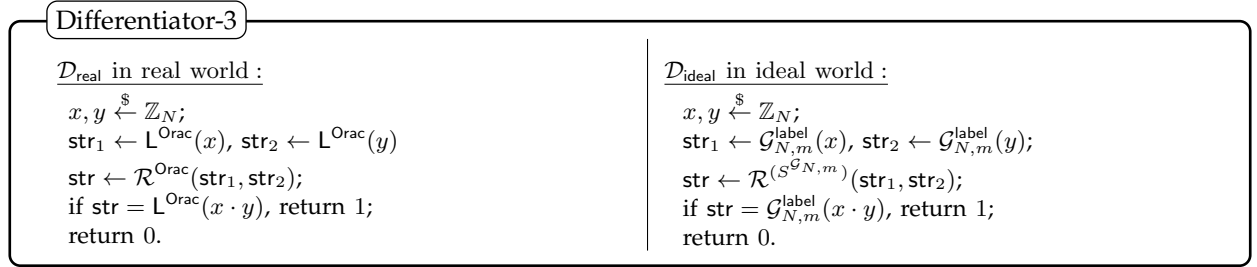


Figure 13: Differentiator-3 against Indifferentiability.

D Sparse GGM vs. Dense GGM

In this section, we explore the relationship between sparse GGM and dense GGM. As demonstrated by Ji et al. [JZW⁺24], we can construct an indifferentiable sparse GGM from a dense GGM with the addition of an independent random permutation. Building on this, we show that dense GGM implies CDH-secure dense groups, in conjunction with Appendix A, thereby establishing a statistical separation between sparse GGM and dense GGM. Considering computationally bounded adversaries, we further demonstrate that sparse GGM does not computationally imply dense GGM within the framework of indistinguishability. Here, we focus specifically on a particular type of dense GGM, where the GGM corresponds to a random injection from $\mathbb{Z}_N$ to $\mathbb{Z}_N$, denoted as $\mathcal{G}_N$.

D.1 Dense groups in dense GGM

In this section, we demonstrate the existence of CDH-secure dense groups in dense GGM. Since dense GGM inherently implies a generic group with dense group encoding, it suffices to prove that the CDH problem with respect to dense GGM is hard.

Theorem D.1. *Let $\mathcal{G}_{N,m_1}$ be a dense generic group model. The computational Diffie-Hellman problem is hard in $\mathcal{G}_{N,m_1}$.*

Proof. Assuming there exists an adversary $\mathcal{A}$ that breaks CDH in $\mathcal{G}_{N,m_1}$. That is, $\mathcal{A}$ takes $\mathcal{G}_{N,m_1}^{\text{label}}(x)$ and $\mathcal{G}_{N,m_1}^{\text{label}}(y)$ as inputs for randomly chosen $x, y \in \mathbb{Z}_N$, makes at most q queries to $\mathcal{G}_{N,m_1}$, and outputs the

group encoding $\mathcal{G}_{N,m_1}^{\text{label}}(x \cdot y)$ with a non-negligible probability ρ . We then leverage the indistinguishability to construct an adversary against CDH w.r.t. sparse GGM, which maintains almost the same winning probability.

Let $\mathcal{G}_{N,m_2}$ be a sparse generic group model, and Perm be a random permutation on $\{0, 1\}^{m_2}$ with its inverse Perm^{Inv} . For simplicity, we denote Π^{Orac} as the tuple $(\mathcal{G}_{N,m_1}, \text{Perm}, \text{Perm}^{\text{Inv}})$ and $\Delta m := m_2 - m_1$. In [JZW⁺24], Ji et al. give a construction $\Pi^{\text{Orac}} = (\mathcal{L}^{\text{Orac}}, \mathcal{A}^{\text{Orac}})$ of sparse generic group which is indistinguishable from $\mathcal{G}_{N,m_2}$. We give the description of Π^{Orac} in Figure 14.

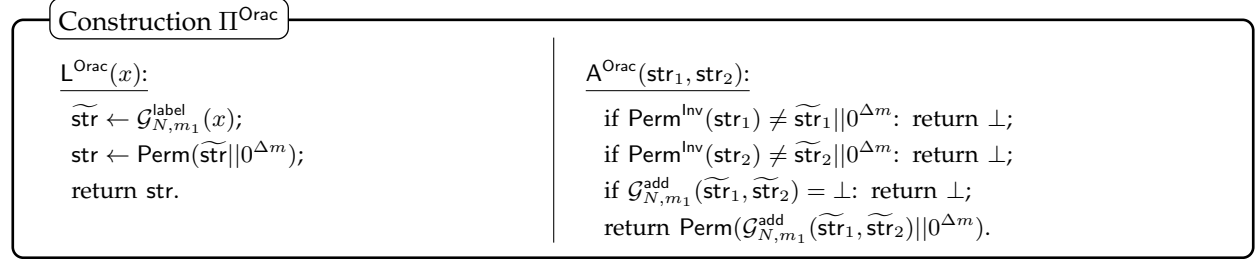


Figure 14: The construction Π^{Orac} in the $\mathcal{G}_{N,m_1}$ and Perm.

Obviously, we can transform the adversary $\mathcal{A}$ against CDH w.r.t. $\mathcal{G}_{N,m_1}$ into an adversary $\mathcal{R}$ against CDH w.r.t. Π^{Orac} . The adversary $\mathcal{R}^{\text{Orac}}$ is illustrated in Figure 15. Trivial to note that, $\mathcal{R}^{\text{Orac}}$ can win the CDH

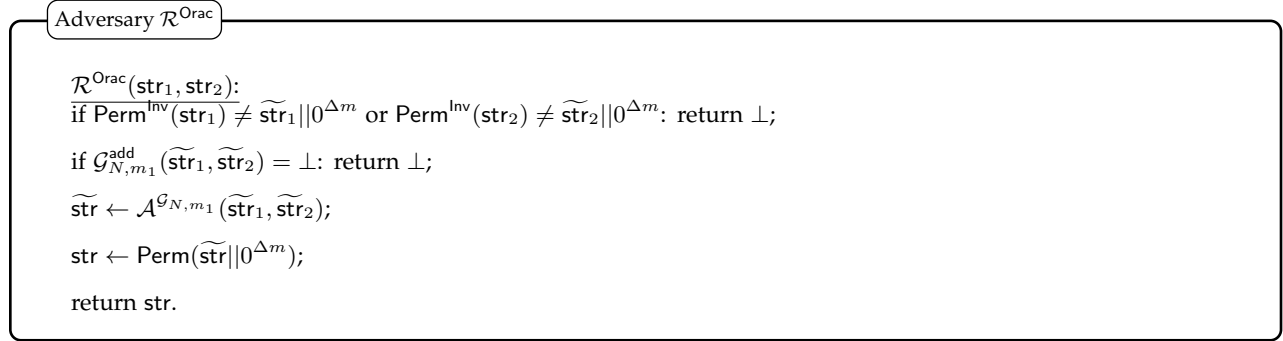


Figure 15: The description of the adversary $\mathcal{R}^{\text{Orac}}$ against CDH w.r.t. Π^{Orac} .

game w.r.t. Π^{Orac} with probability at least ρ .

Since Π^{Orac} is indistinguishable from $\mathcal{G}_{N,m_2}$, there exists a simulator S that makes at most $q^* = \text{poly}(q)$ queries to $\mathcal{G}_{N,m_2}$ and correctly simulates $\mathcal{G}_{N,m_1}$ and $(\text{Perm}, \text{Perm}^{\text{Inv}})$. Thus, we can leverage the simulator to transform the adversary against CDH w.r.t. Π^{Orac} into an adversary against CDH w.r.t. $\mathcal{G}_{N,m_2}$. Specifically, the adversary $\mathcal{R}^{S^{\mathcal{G}_{N,m_2}}} = (\mathcal{R}^S)^{\mathcal{G}_{N,m_2}}$ in the ideal world, takes $\mathcal{G}_{N,m_2}^{\text{label}}(x)$ and $\mathcal{G}_{N,m_2}^{\text{label}}(y)$ as inputs for randomly chosen $x, y \in \mathbb{Z}_N$, and outputs a string $\text{str} \in \{0, 1\}^{m_2}$. We claim that $\mathcal{R}^S$ can break CDH problem against $\mathcal{G}_{N,m_2}$. By contraposition, we assume that the probability of $\mathcal{R}^{S^{\mathcal{G}_{N,m_2}}}(\mathcal{G}_{N,m_2}^{\text{label}}(x), \mathcal{G}_{N,m_2}^{\text{label}}(y))$ outputting the group encoding $\mathcal{G}_{N,m_2}^{\text{label}}(x \cdot y)$ is negligible. We then proceed to construct a differentiator, as illustrated in Figure 16.

Note that $\Pr[\mathcal{D}_{\text{real}} = 1] \geq \rho$. If $\Pr[\mathcal{D}_{\text{ideal}} = 1] \leq \text{negl}(\lambda)$, then the differentiator in Figure 16 can differentiate between the real world and the ideal world with an advantage of at least $\rho - \text{negl}(\lambda)$, which contradicts the Theorem in [JZW⁺24]. Thus, it follows that $\mathcal{R}^S$ can break the CDH against $\mathcal{G}_{N,m_2}$, which contradicts the hardness of CDH in the sparse GGM. Therefore, we conclude that CDH is hard in $\mathcal{G}_{N,m_1}$. $\square$

Differentiator-4

$\mathcal{D}_{\text{real}}$ in real world :

$x, y \xleftarrow{\$} \mathbb{Z}_N;$
 $\text{str}_1 \leftarrow \mathcal{L}^{\text{Orac}}(x), \text{str}_2 \leftarrow \mathcal{L}^{\text{Orac}}(y);$
 $\text{str} \leftarrow \mathcal{R}^{\text{Orac}}(\text{str}_1, \text{str}_2);$
 if $\text{str} = \mathcal{L}^{\text{Orac}}(x \cdot y)$, return 1;
 return 0.

$\mathcal{D}_{\text{ideal}}$ in ideal world :

$x, y \xleftarrow{\$} \mathbb{Z}_N;$
 $\text{str}_1 \leftarrow \mathcal{G}_{N,m_2}^{\text{label}}(x), \text{str}_2 \leftarrow \mathcal{G}_{N,m_2}^{\text{label}}(y);$
 $\text{str} \leftarrow \mathcal{R}^{(\mathcal{G}_{N,m_2}^{\text{label}})}(\text{str}_1, \text{str}_2);$
 if $\text{str} = \mathcal{G}_{N,m_2}^{\text{label}}(x \cdot y)$, return 1;
 return 0.

Figure 16: Differentiator-4 against Indifferentiability.

D.2 Sparse GGM does not computationally imply dense GGM

Theorem D.2. Let $\mathcal{G}_N$ be a dense generic group model and $\mathcal{G}_{N,m}$ be a sparse generic group model. $\mathcal{G}_N$ is computationally indifferentially separated from $\mathcal{G}_{N,m}$.

Essentially, our strategy highly relies on the analysis in [ZZ23], and we first recap the discrete logarithm identification problem (DLI) with respect to generic groups.

Definition D.1 (DLI w.r.t. generic groups [ZZ23]). Let $\lambda \in \mathbb{N}$ be the security parameter, and $\mathcal{G}_{N,m} = (\mathcal{G}_{N,m}^{\text{label}}, \mathcal{G}_{N,m}^{\text{add}})$ be a generic group. An efficient algorithm with access to $\mathcal{G}_{N,m}$, taking 1^λ and a group element $\text{str} := \mathcal{G}_{N,m}^{\text{label}}(x)$ as input, where $x \xleftarrow{\$} \mathbb{Z}_N$, outputs a “query-free” circuit C_{GGM} such that C_{GGM} identifies x with a good probability, specifically,

- $\Pr[C_{\text{GGM}}(x) = 1] \geq \frac{1}{2};$
- $\Pr[C_{\text{GGM}}(x') = 1 | x' \neq x] \leq \rho$, for any noticeable function ρ ,

where the probability is over the sampling of x' and C_{GGM} ’s randomness. We say DLI w.r.t. generic groups is hard if no query-efficient adversary outputs such a circuit.

In order to integrate the analytical framework proposed by Zhandry and Zhang into our proof, we give the definition of the DLI with respect to dense groups in sparse GGM.

Definition D.2 (DLI w.r.t. dense groups in sparse GGM). Let $\lambda \in \mathbb{N}$ be the security parameter, and $\mathcal{G}_{N,m}$ be a sparse generic group model. Denote by $\Pi^{\mathcal{G}_{N,m}} = (\mathcal{L}^{\mathcal{G}_{N,m}}, \mathcal{A}^{\mathcal{G}_{N,m}})$ a dense group in $\mathcal{G}_{N,m}$, with group order N . An efficient algorithm with access to $\mathcal{G}_{N,m}$, taking 1^λ and a group element $\text{str} \leftarrow \mathcal{L}^{\mathcal{G}_{N,m}}(x)$ as input, where $x \xleftarrow{\$} \mathbb{Z}_N$, outputs a “query-free” circuit $C_{\text{G-GGM}}$ such that $C_{\text{G-GGM}}$ identifies x with a good probability, specifically,

- $\Pr[C_{\text{G-GGM}}(x) = 1] \geq \frac{2}{3};$
- $\Pr[C_{\text{G-GGM}}(x') = 1 | x' \neq x] \leq \rho$, for any noticeable function ρ ,

where the probability is over the sampling of x' and $C_{\text{G-GGM}}$ ’s randomness. We say DLI w.r.t. dense groups in sparse GGM is hard if no query-efficient adversary outputs such a circuit.

Now, we are ready to establish the entire proof of Theorem D.2.

Proof. Let $\Pi^{\mathcal{G}_{N,m}} = (\mathcal{L}^{\mathcal{G}_{N,m}}, \mathcal{A}^{\mathcal{G}_{N,m}})$ be a construction of dense groups in sparse GGM, with order N . Suppose $\Pi^{\mathcal{G}_{N,m}}$ is indifferentially separated from $\mathcal{G}_N$. The argument goes in three steps:

1. DLI w.r.t. $\Pi^{\mathcal{G}_{N,m}}$ is easy.
2. If $\Pi^{\mathcal{G}_{N,m}}$ is indifferentially separated from $\mathcal{G}_N$ and DLI w.r.t. $\Pi^{\mathcal{G}_{N,m}}$ is easy, then DLI w.r.t. $\mathcal{G}_N$ is also easy.

3. Yet, DLI w.r.t. the generic group $\mathcal{G}_N$ is hard.

The above three steps result in a contradiction, thereby disproving the initial statement “ $\Pi^{\mathcal{G}_N, m}$ is indistinguishable from $\mathcal{G}_N$ ”. This completes the proof. In what follows, we prove these steps in reverse order for readability.

Lemma D.1. *DLI w.r.t. the generic group $\mathcal{G}_N$ is hard.*

The proof of Lemma D.1 is taken verbatim from [ZZ23].

Proof. Assuming there is an efficient adversary $\mathcal{A}$ making at most q queries to $\mathcal{G}_N$ that breaks DLI, we can convert $\mathcal{A}$ into a query-efficient adversary $\mathcal{R}$, as shown in Figure 17, that breaks discrete logarithm problem in the GGM¹⁵.

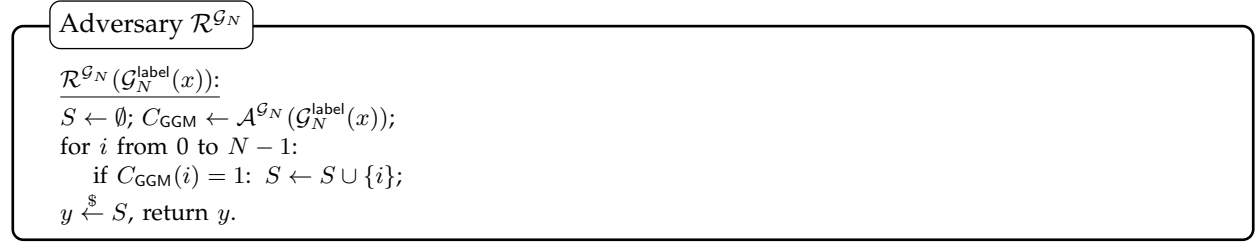


Figure 17: A query-efficient adversary breaks Discrete Logarithm in the GGM

Note that $\mathcal{R}^{\mathcal{G}_N}$ makes at most q queries to $\mathcal{G}_N$. We set ρ to be $\frac{1}{q^{1/2}}$ and mbound to be $1 + \frac{N-1}{q^6}$, respectively; applying Markov’s inequality, it is apparent that $\Pr[|S| > \text{mbound}] \leq \frac{1}{q^6}$. Therefore, we have that

$$\begin{aligned} \Pr[\mathcal{R}^{\mathcal{G}_N} \text{ outputs } x] &= \sum_{i=1}^N \Pr[|S| = i] \cdot \left(\frac{1}{i}\right) \cdot \Pr[x \in S \mid |S| = i] \\ &\geq \Pr[|S| \leq \text{mbound}] \cdot \left(\frac{1}{\text{mbound}}\right) \cdot \Pr[x \in S \mid |S| \leq \text{mbound}]. \end{aligned}$$

Note that $\Pr[x \in S] \geq 1/2$, which means that $\Pr[x \in S \mid |S| \leq \text{mbound}]$ should be close to $1/2$, because $|S| \leq \text{mbound}$ happens with high probability. Formally, we denote Event_1 and Event_2 to be the events “ $x \in S$ ” and “ $|S| \leq \text{mbound}$ ”, respectively, we have that

$$\begin{aligned} \Pr[\text{Event}_1 \mid \text{Event}_2] &= \frac{\Pr[\text{Event}_1] - \Pr[\text{Event}_1 \mid \neg \text{Event}_2] \Pr[\neg \text{Event}_2]}{\Pr[\text{Event}_2]} \\ &\geq \frac{\Pr[\text{Event}_1] - \Pr[\neg \text{Event}_2]}{\Pr[\text{Event}_2]} \\ &= 1 - \frac{1 - \Pr[\text{Event}_1]}{\Pr[\text{Event}_2]}. \end{aligned}$$

Substituting into the inequalities above, we have that

$$\begin{aligned} \Pr[\mathcal{R}^{\mathcal{G}_N} \text{ outputs } x] &\geq \left(1 - \frac{1}{q^6}\right) \cdot \left(\frac{1}{1 + \frac{N-1}{q^6}}\right) \cdot \left(1 - \frac{1}{2} \cdot \left(1 - \frac{1}{q^6}\right)\right) \\ &\geq \frac{1}{2} \cdot \left(1 - \frac{1}{q^6}\right) \cdot \frac{q^6}{q^6 + N - 1} = \Theta\left(\frac{q^6}{N}\right) > O\left(\frac{q^2}{N}\right), \end{aligned}$$

which contradicts the hardness of discrete logarithm in the GGM [Sho97]. □ □

¹⁵Note that $\mathcal{R}$ is time-inefficient if N is super-polynomial, which is not a problem in our setting since the lower bound of discrete log in the GGM [Sho97] only counts the number of oracle queries.

Lemma D.2. If $\Pi^{\mathcal{G}_{N,m}}$ is indifferentiable from $\mathcal{G}_N$ and DLI w.r.t. $\Pi^{\mathcal{G}_{N,m}}$ is easy, then DLI w.r.t. $\mathcal{G}_N$ is easy.

Proof. Let $\mathcal{A}^{\mathcal{G}_{N,m}}$ be an adversary that breaks the DLI with respect to $\Pi^{\mathcal{G}_{N,m}}$, that is, $\mathcal{A}^{\mathcal{G}_{N,m}}$ takes $\mathcal{L}^{\mathcal{G}_{N,m}}(x)$ as input, makes at most q queries to $\mathcal{G}_{N,m}$, and outputs a query-free circuit $C_{\text{G-GGM}}$ satisfying the following two properties:

1. $C_{\text{G-GGM}}$ identifies x with a good probability;
2. $C_{\text{G-GGM}}$ identifies $x' (x' \neq x)$ with a small probability.

Since $\Pi^{\mathcal{G}_{N,m}}$ is indifferentiable from $\mathcal{G}_N$, there exists a simulator $\mathcal{S}$ that makes at most $q^* = \text{poly}(q)$ queries to $\mathcal{G}_N$ and simulates the generic group $\mathcal{G}_{N,m}$ properly. Thus, we can leverage the simulator to transfer the adversary against DLI w.r.t. $\Pi^{\mathcal{G}_{N,m}}$ to an adversary against DLI w.r.t. $\mathcal{G}_N$. That is, the adversary $\mathcal{A}^{(\mathcal{S}^{\mathcal{G}_N})} = (\mathcal{A}^{\mathcal{S}})^{\mathcal{G}_N}$ in the ideal world, taking $\mathcal{G}_N^{\text{label}}(x)$ as input, outputs a query-free circuit C_{GGM} . By definition, we have that no efficient differentiator $\mathcal{D}$ can distinguish these two circuits, i.e., $C_{\text{G-GGM}}$ and C_{GGM} . Next, we claim that C_{GGM} also achieves the same two properties.

Property 1. In this part, we demonstrate that $\Pr[C_{\text{GGM}}(x) = 1] \geq \frac{1}{2}$. Assume toward a contradiction that $\Pr[C_{\text{GGM}}(x) = 1] < \frac{1}{2}$, then there exists an efficient differentiator, illustrated in Figure 18, that breaks indifferentiability.

Differentiator-5	
<u>$\mathcal{D}_{\text{real}}$ in real world :</u> $x \xleftarrow{\$} \mathbb{Z}_N$; $C_{\text{G-GGM}} \leftarrow \mathcal{A}^{\mathcal{G}_{N,m}}(\mathcal{L}^{\mathcal{G}_{N,m}}(x))$; if $C_{\text{G-GGM}}(x) = 1$, return 1; return 0.	<u>$\mathcal{D}_{\text{ideal}}$ in ideal world :</u> $x \xleftarrow{\$} \mathbb{Z}_N$; $C_{\text{GGM}} \leftarrow (\mathcal{A}^{\mathcal{S}})^{\mathcal{G}_N}(\mathcal{G}_N^{\text{label}}(x))$; if $C_{\text{GGM}}(x) = 1$, return 1; return 0.

Figure 18: Differentiator-5 against Indifferentiability.

Note that $\Pr[\mathcal{D}_{\text{real}} = 1] \geq \frac{2}{3}$. However, if $\Pr[C_{\text{GGM}}(x) = 1] < \frac{1}{2}$, then $\Pr[\mathcal{D}_{\text{ideal}} = 1] < \frac{1}{2}$. This indicates that the differentiator in Figure 18 can break the indifferentiability with advantage at least $\frac{1}{6}$.

Property 2. In this part, we demonstrate that $\Pr_{x' \neq x}[C_{\text{GGM}}(x') = 1] \leq \rho$, for any noticeable function ρ . By contraposition, we assume that there exists a noticeable function ρ^* , such that $\Pr_{x' \neq x}[C_{\text{GGM}}(x') = 1] > \rho^*$. Then we proceed to construct an efficient differentiator that breaks indifferentiability. Let n be a polynomial such that $n \geq \frac{1}{\rho^*}$, we illustrate our differentiator in Figure 19.

Differentiator-6	
<u>$\mathcal{D}_{\text{real}}$ in real world :</u> $x \xleftarrow{\$} \mathbb{Z}_N$; $x_1, \dots, x_n \xleftarrow{\$} \mathbb{Z}_N \setminus \{x\}$; $C_{\text{G-GGM}} \leftarrow \mathcal{A}^{\mathcal{G}_{N,m}}(\mathcal{L}^{\mathcal{G}_{N,m}}(x))$; for $i = 1$ to n : if $C_{\text{G-GGM}}(x_i) = 1$, return 0; return 1.	<u>$\mathcal{D}_{\text{ideal}}$ in ideal world :</u> $x \xleftarrow{\$} \mathbb{Z}_N$; $x_1, \dots, x_n \xleftarrow{\$} \mathbb{Z}_N \setminus \{x\}$; $C_{\text{GGM}} \leftarrow \mathcal{A}^{(\mathcal{S}^{\mathcal{G}_N})}(\mathcal{G}_N(x))$; for $i = 1$ to n : if $C_{\text{GGM}}(x_i) = 1$, return 0; return 1.

Figure 19: Differentiator-6 against Indifferentiability.

In the real world, by the statement of Lemma D.2, we have that for any noticeable function, say $\frac{0.1}{n}$, $\Pr_{x' \neq x}[C_{\text{G-GGM}}(x') = 1] \leq \frac{0.1}{n}$. Thus, by union bound, its apparent that

$$\Pr[\mathcal{D}_{\text{real}} = 1] \leq n \cdot \frac{0.1}{n} = 0.1.$$

On the other hand, if $\Pr_{x' \neq x}[C_{\text{GGM}}(x') = 1] > \rho^*$ in the ideal world, then

$$\begin{aligned} \Pr[\mathcal{D}_{\text{ideal}} = 1] &\geq 1 - (1 - \rho^*)^n \geq 1 - (1 - \rho^*)^{\frac{1}{\rho^*}} \\ &\geq 1 - \frac{1}{e} > 0.6. \end{aligned}$$

Therefore, the advantage of the differentiator in Figure 19 is at least $(0.6 - 0.1) \geq 0.5$.

Hence, if $\Pi^{\mathcal{G}_{N,m}}$ is indifferentiable from $\mathcal{G}_N$ and DLI w.r.t. $\Pi^{\mathcal{G}_{N,m}}$ is easy, then DLI w.r.t. $\mathcal{G}_N$ is easy. $\square$

$\square$

Lemma D.3. *DLI w.r.t. $\Pi^{\mathcal{G}_{N,m}}$ is easy.*

Adversary $\mathcal{A}^{\mathcal{G}_{N,m}}$

$\mathcal{A}^{\mathcal{G}_{N,m}}(\mathcal{L}^{\mathcal{G}_{N,m}}(x))$:

$S_{\text{que-res}} \leftarrow \emptyset; r, r_1, \dots, r_n \xleftarrow{\$} \mathbb{Z}_N;$

for $i = 1$ to n : *//collect frequent queries*

$\{(\text{que}_j, \text{res}_j)\}_{j=1}^q \xleftarrow{\text{query}} \mathcal{L}^{\mathcal{G}_{N,m}}(r_i), S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup \{(\text{que}_j, \text{res}_j)\}_{j=1}^q;$

$\{(\text{que}_j, \text{res}_j)\}_{j=1}^q \xleftarrow{\text{query}} \mathcal{L}^{\mathcal{G}_{N,m}}(r), S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup \{(\text{que}_j, \text{res}_j)\}_{j=1}^q;$

$\{(\text{que}_j, \text{res}_j)\}_{j=1}^q \xleftarrow{\text{query}} \mathcal{L}^{\mathcal{G}_{N,m}}(-r), S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup \{(\text{que}_j, \text{res}_j)\}_{j=1}^q;$

$\{(\text{que}_j^{(x,-r)}, \text{res}_j^{(x,-r)})\}_{j=1}^q \xleftarrow{\text{query}} \mathcal{A}^{\mathcal{G}_{N,m}}(\mathcal{L}^{\mathcal{G}_{N,m}}(x), \mathcal{L}^{\mathcal{G}_{N,m}}(-r));$

$\{(\text{que}_j^{(x-r,r)}, \text{res}_j^{(x-r,r)})\}_{j=1}^q \xleftarrow{\text{query}} \mathcal{A}^{\mathcal{G}_{N,m}}(\mathcal{L}^{\mathcal{G}_{N,m}}(x-r), \mathcal{L}^{\mathcal{G}_{N,m}}(r));$

for $z \in \{(x, -r), (x-r, r)\}$ and j from 1 to q : *//collect queries in group addition*

if que_j^z is a labeling query: $S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup \{(\text{que}_j^z, \text{res}_j^z)\};$

if $\text{que}_j^z = (\text{str}_1, \text{str}_2)$ is an addition query:

if $\exists(x_1, \text{str}_1), (x_2, \text{str}_2) \in S_{\text{que-res}}: S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup \{(x_1 + x_2, \text{res}_j^z)\};$

return $C_{\text{G-GGM}}(\cdot, S_{\text{que-res}}, \mathcal{L}^{\mathcal{G}_{N,m}}(x)).$

$C_{\text{G-GGM}}(\cdot, S_{\text{que-res}}, \mathcal{L}^{\mathcal{G}_{N,m}}(x))$:

take $\hat{x} \in \mathbb{Z}_N$ as input, run $\text{str} \leftarrow \mathcal{L}^{S_{\text{que-res}}}(\hat{x});$

when $\mathcal{L}$ makes a labeling query que : *//respond with S*

if $\exists(\text{que}, \text{str}) \in S_{\text{que-res}}: \text{respond with str};$

else: respond with a uniformly sampled $\text{str}; S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup \{(\text{que}, \text{str})\};$

if $\widehat{\text{str}} = \mathcal{L}^{\mathcal{G}_{N,m}}(x): \text{return 1}; \text{else: return 0}.$

Figure 20: Efficient Adversary $\mathcal{A}^{\mathcal{G}_{N,m}}$ and query-free circuit $C_{\text{G-GGM}}$ w.r.t. $\Pi^{\mathcal{G}_{N,m}}$.

Proof. By the definition of indifferentiability, the algorithms $\mathcal{L}^{\mathcal{G}_{N,m}}$ and $\mathcal{A}^{\mathcal{G}_{N,m}}$ are deterministic; and they shall support group operations correctly with high probability. More specifically, there exists a negligible function ϵ_1 , such that

$$\Pr[\mathcal{L}^{\mathcal{G}_{N,m}}(x) = \mathcal{L}^{\mathcal{G}_{N,m}}(y) \mid x \neq y \in \mathbb{Z}_N] \leq \epsilon_1, \quad (3)$$

$$\Pr[\mathcal{A}^{\mathcal{G}_{N,m}}(\mathcal{L}^{\mathcal{G}_{N,m}}(x), \mathcal{L}^{\mathcal{G}_{N,m}}(y)) = \mathcal{L}^{\mathcal{G}_{N,m}}(x + y)] \geq 1 - \epsilon_1, \quad (4)$$

where the probability is over the sampling of $x, y \in \mathbb{Z}_N$ and the generic group $\mathcal{G}_{N,m}$. As explained above, we stress that $L^{\mathcal{G}_{N,m}}$ only makes labeling queries. Let q be an integer in $\text{poly}(\lambda)$. We assume that both $L^{\mathcal{G}_{N,m}}$ and $A^{\mathcal{G}_{N,m}}$ make at most q queries to $\mathcal{G}_{N,m}$. Next, we prove that the DLI w.r.t. $\Pi^{\mathcal{G}_{N,m}}$ is easy by constructing an efficient adversary $\mathcal{A}$ and a query-free circuit $C_{\text{G-GGM}}$, illustrated in Figure 20.

We first clarify some undefined notions. Let n be a sufficiently large integer to be specified below. By $\{(\text{que}_j, \text{res}_j)\}_{j=1}^q \xleftarrow{\text{query}} L^{\mathcal{G}_{N,m}}(r)$, we denote that on input r , the algorithm $L^{\mathcal{G}_{N,m}}(r)$ makes q queries to $\mathcal{G}_{N,m}$, with the corresponding responses; similar for the algorithm $A^{\mathcal{G}_{N,m}}(L^{\mathcal{G}_{N,m}}(x), L^{\mathcal{G}_{N,m}}(y))$ ¹⁶. Given an input $\hat{x} \in \mathbb{Z}_N$, the query-free circuit $C_{\text{G-GGM}}$ runs algorithm $L^{\mathcal{G}_{N,m}}(\hat{x})$ except for replacing the querying oracle by looking up the table $S_{\text{que-res}}$ (and lazy sampling); we denote that as $L^{S_{\text{que-res}}}$.

The hope is that by the time $\mathcal{A}$ finishes, S will contain all “relevant” queries, such that $C_{\text{G-GGM}}$ satisfies the two desired properties:

1. $\Pr[C_{\text{G-GGM}}(x) = 1] \geq \frac{2}{3}$;
2. $\Pr[C_{\text{G-GGM}}(x') = 1 | x' \neq x] \leq \rho$, for any noticeable function ρ .

We next prove this one by one.

Property 1. Here, we prove that $\Pr[C_{\text{G-GGM}}(x) = 1] \geq \frac{2}{3}$. As shown in Figure 20, $C_{\text{G-GGM}}(x)$ outputs 1 if and only if $L^{S_{\text{que-res}}}(x) = L^{\mathcal{G}_{N,m}}(x)$. For any $x, y \in \mathbb{Z}_N$, let $S_{x,y}$ be the set of all group elements appeared in responses when invoking $A^{\mathcal{G}_{N,m}}(L^{\mathcal{G}_{N,m}}(x), L^{\mathcal{G}_{N,m}}(y))$. We denote

$$Q_x := \{(\text{que}_j, \text{res}_j)\}_{j=1}^q \xleftarrow{\text{query}} L^{\mathcal{G}_{N,m}}(x);$$

$$Q_{x,y} := \{(\text{que}, \text{res}) | \text{res} \in S_{x,y}, \mathcal{G}_{N,m}^{\text{label}}(\text{que}) = \text{res}\}.$$

Obviously, if $Q_x \subseteq S_{\text{que-res}}$, then $L^{S_{\text{que-res}}}(x) = L^{\mathcal{G}_{N,m}}(x)$, referring to $C_{\text{G-GGM}}(x) = 1$. Unfortunately, x is independent of the adversary’s view and some queries in Q_x may be completely ignored when running $L^{\mathcal{G}_{N,m}}(x)$. Thus we cannot guarantee $Q_x \subseteq S_{\text{que-res}}$. However, suppose that the label on x depends on res for some query-response (que, res) in Q_x . Since $A^{\mathcal{G}_{N,m}}$ takes as input the labels for $x - r$, r will also output the label on x , which indicates that $L^{\mathcal{G}_{N,m}}$ must somehow recover res . In other words, either $A^{\mathcal{G}_{N,m}}(L^{\mathcal{G}_{N,m}}(x - r), L^{\mathcal{G}_{N,m}}(r))$ or $L^{\mathcal{G}_{N,m}}(x - r)$ and $L^{\mathcal{G}_{N,m}}(r)$ must make query to get res . Thus, we argue that the only queries we care about are actually $Q_x \cap (Q_r \cup Q_{x-r} \cup Q_{x-r,r})$, denoted as Q_x^* , and it suffices to prove that $Q_x^* \subseteq S_{\text{que-res}}$ with a good probability. Next, we prove it by two steps as follows:

- $Q_x \cap Q_{x-r,r} \subseteq S_{\text{que-res}}$ with a good probability;
- $Q_x \cap (Q_r \cup Q_{x-r}) \subseteq S_{\text{que-res}}$ with a good probability.

$Q_x \cap Q_{x-r,r} \subseteq S_{\text{que-res}}$ with a good probability. In fact, for each $(\text{que}, \text{res}) \in Q_x \cap Q_{x-r,r}$, there are three cases during the run of $A^{\mathcal{G}_{N,m}}(L^{\mathcal{G}_{N,m}}(x - r), L^{\mathcal{G}_{N,m}}(r))$:

- Case 1: res is the response of a labeling query;
- Case 2: res is the response of an addition query, which can be converted into a labeling query;
- Case 3: res is the response of an addition query, which cannot be converted into a labeling query.

In the former two cases, as shown in Figure 20, we know that $(\text{que}, \text{res}) \in S_{\text{que-res}}$ absolutely. The Case 3 indicates that the algorithm $A^{\mathcal{G}_{N,m}}$ generates valid group elements res^* which does not exist in $S_{\text{que-res}}$. Since $S_{\text{que-res}}$ collects all the labeling queries and addition queries that can be converted into labeling queries during the run of $\mathcal{A}$, it follows that res^* must be a new group element. Therefore, let Bad denote the event in which Case 3 occurs during the run of $\mathcal{A}^{\mathcal{G}_{N,m}}$. We can bound the probability of Bad by constructing an extractor $\mathcal{E}$, as illustrated in Figure 21.

¹⁶Here, we abuse the notation $L^{\mathcal{G}_{N,m}}(x)$ as both the group element and the labeling operation on x .

Extractor $\mathcal{E}^{\mathcal{G}_{N,m}}(1^\lambda)$

```

 $\mathcal{L}^{\mathcal{G}_{N,m}}(x) \xleftarrow{\$} \mathbb{Z}_N;$ 

 $S_{\text{que-res}} \leftarrow \emptyset; r, r_1, \dots, r_n \xleftarrow{\$} \mathbb{Z}_N;$ 

for  $i$  from 1 to  $n$ : //collect frequent queries
     $\{(\text{que}_j, \text{res}_j)\}_{j=1}^q \xleftarrow{\text{query}} \mathcal{L}^{\mathcal{G}_{N,m}}(r_i), S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup \{(\text{que}_j, \text{res}_j)\}_{j=1}^q;$ 

 $\{(\text{que}_j, \text{res}_j)\}_{j=1}^q \xleftarrow{\text{query}} \mathcal{L}^{\mathcal{G}_{N,m}}(r), S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup \{(\text{que}_j, \text{res}_j)\}_{j=1}^q;$ 

 $\{(\text{que}_j, \text{res}_j)\}_{j=1}^q \xleftarrow{\text{query}} \mathcal{L}^{\mathcal{G}_{N,m}}(-r), S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup \{(\text{que}_j, \text{res}_j)\}_{j=1}^q;$ 

 $\{(\text{que}_j^{(x,-r)}, \text{res}_j^{(x,-r)})\}_{j=1}^q \xleftarrow{\text{query}} \mathcal{A}^{\mathcal{G}_{N,m}}(\mathcal{L}^{\mathcal{G}_{N,m}}(x), \mathcal{L}^{\mathcal{G}_{N,m}}(-r));$ 

 $\{(\text{que}_j^{(x-r,r)}, \text{res}_j^{(x-r,r)})\}_{j=1}^q \xleftarrow{\text{query}} \mathcal{A}^{\mathcal{G}_{N,m}}(\mathcal{L}^{\mathcal{G}_{N,m}}(x-r), \mathcal{L}^{\mathcal{G}_{N,m}}(r));$ 

for  $z \in \{(x, -r), (x-r, r)\}$  and  $j$  from 1 to  $q$ : //collect queries in group addition
    if  $\text{que}_j^z$  is a labeling query:  $S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup \{(\text{que}_j^z, \text{res}_j^z)\};$ 
    if  $\text{que}_j^z = (\text{str}_1, \text{str}_2)$  is an addition query:
        if  $\exists (x_1, \text{str}_1), (x_2, \text{str}_2) \in S_{\text{que-res}}: S_{\text{que-res}} \leftarrow S_{\text{que-res}} \cup \{(x_1 + x_2, \text{res}_j^z)\};$ 
        if  $\text{str}_1 \notin S_{\text{que-res}}$  and  $\mathcal{G}_{N,m}^{\text{add}}(\text{str}_1, \text{str}_1) \neq \perp$ , return  $\text{str}_1$ ;
        if  $\text{str}_2 \notin S_{\text{que-res}}$  and  $\mathcal{G}_{N,m}^{\text{add}}(\text{str}_2, \text{str}_2) \neq \perp$ , return  $\text{str}_2$ ;

return  $\perp$ .

```

Figure 21: The description of the extractor $\mathcal{E}^{\mathcal{G}_{N,m}}$ that outputs a new group element.

Note that if Bad occurs during the run of $\mathcal{A}^{\mathcal{G}_{N,m}}(\text{label})$, where label is uniform sampled from $\mathbb{Z}_N$, $\mathcal{E}$ can successfully extract such a new group element. Based on Lemma G.2, we have

$$\Pr[\text{Bad occurs in } \mathcal{A}^{\mathcal{G}_{N,m}}(\text{label})] \leq \epsilon_2(\lambda),$$

where ϵ_2 is a negligible function, and the probability is taken over the sampling of the GGM instance $\mathcal{G}_{N,m}$, the group encoding label $\in \mathbb{Z}_N$, and the internal randomness of the adversary $\mathcal{A}$. By the definition of indistinguishability, there exists a negligible function ϵ_3 , such that the distributions of $\{\mathcal{L}^{\mathcal{G}_{N,m}}(x)\}_{x \in \mathbb{Z}_N}$ and $\{\text{label}\}_{\text{label} \in \mathbb{Z}_N}$ are ϵ_3 -computationally indistinguishable. Therefore, for uniformly chosen GGM instance $\mathcal{G}_{N,m}$ and the value $x \in \mathbb{Z}_N$, we have

$$\Pr[\text{Bad occurs in } \mathcal{A}^{\mathcal{G}_{N,m}}(\mathcal{L}^{\mathcal{G}_{N,m}}(x))] \leq \epsilon_2(\lambda) + \epsilon_3(\lambda).$$

Consequently, we can conclude that

$$\Pr[Q_x \cap Q_{x-r,r} \subseteq S_{\text{que-res}}] \geq 1 - \epsilon_2(\lambda) - \epsilon_3(\lambda).$$

$Q_x \cap (Q_r \cup Q_{x-r}) \subseteq S_{\text{que-res}}$ with a good probability. Note that $Q_r \subseteq S_{\text{que-res}}$, therefore it is enough to show that $Q_x \cap Q_{x-r} \subseteq S_{\text{que-res}}$. However, that fact is that $x-r$ is unknown to $\mathcal{A}$. To resolve this barrier, we show

that with high probability the queries in $Q_x \cap Q_{x-r}$ are all frequent queries (defined below). Thus, we can draw sufficiently large number of samples, say $r_1, r_2, \dots, r_n$, record all the queries made in $\mathcal{L}^{\mathcal{G}_{N,m}}(r_i)$, and capture all the frequent queries. In other words, we show that

$$Q_x \cap Q_{x-r} \subseteq Q_{r_1} \cup \dots \cup Q_{r_n} \subseteq S_{\text{que-res}}$$

with high probability. Now we define the frequent group elements as follows: let $t := 200q$, we say a labeling query (que, res) and the group element res are frequent, if

$$\Pr[(\text{que}, \text{res}) \in Q_r : r \xleftarrow{\$} \mathbb{Z}_N] \geq \frac{1}{t}.$$

Since both x and r are uniformly sampled, $x - r$ and x are independent. For any labeling query (que, res) $\in Q_{x-r}$, if (que, res) is not frequent, then

$$\Pr[(\text{que}, \text{res}) \in Q_x] \leq \frac{1}{t}.$$

Thus, we have that

$$\Pr[Q_x \cap Q_{x-r} \text{ are all frequent queries}] \geq 1 - \frac{q}{t} = 1 - \frac{1}{200}.$$

Let $n := t^2 \cdot q$. Below, we prove that $Q_{r_1} \cup \dots \cup Q_{r_n}$ contains all frequent queries with a noticeable probability. In fact, there are at most $q_{\text{fre}} := q \cdot t$ frequent queries. Denote all frequency queries as $\{(\text{que}_{\text{fre-}i}, \text{res}_{\text{fre-}i})\}_{i \in [q_{\text{fre}}]}$. For each $(\text{que}_{\text{fre-}i}, \text{res}_{\text{fre-}i})$, we have

$$\Pr[(\text{que}_{\text{fre-}i}, \text{res}_{\text{fre-}i}) \notin Q_{r_1} \cup \dots \cup Q_{r_n}] \leq \left(1 - \frac{1}{t}\right)^n \leq e^{-q \cdot t},$$

which refers to

$$\Pr[(\text{que}_{\text{fre-}i}, \text{res}_{\text{fre-}i}) \in Q_{r_1} \cup \dots \cup Q_{r_n} : \forall i \in [q_{\text{fre}}]] \geq 1 - (q \cdot t)e^{-q \cdot t} \geq 1 - 200e^{-200}.$$

Therefore, we have that

$$\Pr[Q_x \cap (Q_r \cup Q_{x-r}) \subseteq S_{\text{que-res}}] \geq 1 - \frac{1}{200} - 200e^{-200}.$$

Combining together, we have that

$$\Pr[Q_x^* \subseteq S_{\text{que-res}}] \geq 1 - \frac{1}{200} - 200e^{-200} - \text{negl} \geq \frac{2}{3},$$

which refers to $\Pr[C_{\text{G-GGM}}(x) = 1] \geq \frac{2}{3}$.

Property 2. Here, we prove that $\Pr_{x' \neq x}[C_{\text{G-GGM}}(x') = 1] \leq \rho$, for any noticeable function ρ . Recall that, $C_{\text{G-GGM}}(x')$ outputs 1 if and only if $\mathcal{L}^{S_{\text{que-res}}}(x') = \mathcal{L}^{\mathcal{G}_{N,m}}(x)$. There are two cases as follows:

- Case 1: $Q_{x'} \subseteq S_{\text{que-res}}$ and $\mathcal{L}^{S_{\text{que-res}}}(x') = \mathcal{L}^{\mathcal{G}_{N,m}}(x)$;
- Case 2: $Q_{x'} \not\subseteq S_{\text{que-res}}$ and $\mathcal{L}^{S_{\text{que-res}}}(x') = \mathcal{L}^{\mathcal{G}_{N,m}}(x)$.

Immediately observe that if $Q_{x'} \subseteq S_{\text{que-res}}$, then $\mathcal{L}^{S_{\text{que-res}}}(x') = \mathcal{L}^{\mathcal{G}_{N,m}}(x')$. According to Equation 1, we have

$$\Pr[\text{Case 1}] \leq \epsilon_1.$$

For Case 2, since $Q_{x'} \not\subseteq S_{\text{que-res}}$, it is unnecessary that $\mathcal{L}^{S_{\text{que-res}}}(x') = \mathcal{L}^{\mathcal{G}_{N,m}}(x')$. Thus, we first define an event “ $S_{\text{que-res}}$ is good”, if $\Pr_{x' \neq x}[\mathcal{L}^{S_{\text{que-res}}}(x') = \mathcal{L}^{S_{\text{que-res}}}(x)] \leq \sqrt{\epsilon_1}$, where the probability is only over the sampling of x and x' ; and otherwise we say “ $S_{\text{que-res}}$ is bad”. When $S_{\text{que-res}}$ is good, for any $\text{str} \in \{0, 1\}^{m_1}$, we have that

$\Pr[\mathsf{L}^{S_{\text{que-res}}}(x') = \text{str}] \leq \sqrt[4]{\epsilon_1}$. Since S simulates a well-distributed GGM (via lazy sampling), by Equation 1, we have

$$\begin{aligned} & \Pr_{x' \neq x} [\mathsf{L}^{S_{\text{que-res}}}(x') = \mathsf{L}^{S_{\text{que-res}}}(x) | S_{\text{que-res}} \text{ is good}] \cdot \Pr[S_{\text{que-res}} \text{ is good}] \\ & + \Pr_{x' \neq x} [\mathsf{L}^{S_{\text{que-res}}}(x') = \mathsf{L}^{S_{\text{que-res}}}(x) | S_{\text{que-res}} \text{ is bad}] \cdot \Pr[S_{\text{que-res}} \text{ is bad}] \leq \epsilon_1 \end{aligned}$$

It's apparent that $\Pr[S_{\text{que-res}} \text{ is bad}] \leq \sqrt{\epsilon_1}$. Therefore,

$$\begin{aligned} & \Pr[\mathsf{L}^{S_{\text{que-res}}}(x') = \mathsf{L}^{\mathcal{G}_{N,m}}(x)] \\ & = \Pr[\mathsf{L}^{S_{\text{que-res}}}(x') = \mathsf{L}^{\mathcal{G}_{N,m}}(x) | S_{\text{que-res}} \text{ is good}] \cdot \Pr[S_{\text{que-res}} \text{ is good}] \\ & \quad + \Pr[\mathsf{L}^{S_{\text{que-res}}}(x') = \mathsf{L}^{\mathcal{G}_{N,m}}(x) | S_{\text{que-res}} \text{ is bad}] \cdot \Pr[S_{\text{que-res}} \text{ is bad}] \\ & \leq \sqrt[4]{\epsilon_1} + \sqrt{\epsilon_1}, \end{aligned}$$

which refers to $\Pr[\text{Case 2}] \leq \sqrt[4]{\epsilon_1} + \sqrt{\epsilon_1}$.

Combining together, we obtain

$$\Pr[C_{\text{G-GGM}}(x') = 1] \leq \epsilon_1 + \sqrt[4]{\epsilon_1} + \sqrt{\epsilon_1}$$

which means that for any noticeable function ρ , $\Pr_{x' \neq x}[C_{\text{G-GGM}}(x') = 1] \leq \rho$. □

Combining together, we establish the entire proof. □

E Impossibility of groups in sparse GBM

In this section, we show the impossibility results of CDH-secure groups with nice admissible encodings and CDH-secure dense groups in the sparse GBM.

E.1 Sparse GBM vs. groups with admissible encodings

In this section, we demonstrate that any the cryptographic group with nice admissible encodings that is CDH-secure, denoted as $\text{ae-}\mathcal{P}^{\text{CDH}}$, cannot exist within the sparse GBM. Following the same strategy in Section 3, it is sufficient to prove that non-interactive key exchange (NIKE) with nice admissible encodings that is secure against key-recovery attack (KRA-secure), denoted as $\text{ae-}\mathcal{P}^{\text{NIKE}}$, does not exist in the sparse GBM.

Theorem E.1. *Let $\mathcal{B}_{N,m_S,m_T}$ be a generic bilinear group model with sparse encoding. Let $\Pi^{\mathcal{B}_{N,m_S,m_T}} = (\text{KGen}^{\mathcal{B}_{N,m_S,m_T}}, \text{SHK}^{\mathcal{B}_{N,m_S,m_T}}, \text{AE}^{\mathcal{B}_{N,m_S,m_T}})$ be any non-interactive key exchange protocol with nice admissible encoding parameterized by a constant ϱ and a polynomial poly_{AE} . Then there exists an adversary $\mathcal{A}$ and two polynomials poly_1 and poly_2 such that:*

- $\mathcal{A}$ makes poly_1 queries to $\mathcal{B}_{N,m_S,m_T}$;
- $\mathcal{A}$ breaks the KRA-secure game with advantage $\frac{1}{\text{poly}_2}$.

Proof. For simplicity, in the following analysis we re-denote the sparse GBM as

$$\mathcal{B} = (\mathcal{B}^{\text{S-label}}, \mathcal{B}^{\text{S-add}}, \mathcal{B}^{\text{T-label}}, \mathcal{B}^{\text{T-add}}, \mathcal{B}^{\text{map}}).$$

Analogously, the non-interactive key exchange protocol with nice admissible encodings in sparse GBM is re-denoted as $\Pi^{\mathcal{B}} = (\text{KGen}^{\mathcal{B}}, \text{SHK}^{\mathcal{B}}, \text{AE}^{\mathcal{B}})$.

To establish the proof, we present a formal description of the adversary $\mathcal{A}$, as illustrated in Figure 22. Here, we specify that any algorithm in $\Pi^{\mathcal{B}}$ makes at most q queries, where q is a polynomial. We then clarify

some undefined notions: By $\{(\text{que}_j^{\text{sou}}, \text{res}_j^{\text{sou}})\}_{j=1}^{q_1}, \{(\text{que}_j^{\text{tar}}, \text{res}_j^{\text{tar}})\}_{j=1}^{q_2} \xleftarrow{\text{query}} \text{AE}^{\mathcal{B}}(\text{seed})$, we mean that when running $\text{AE}^{\mathcal{B}}(\text{seed})$, the algorithm makes q_1 labeling queries for the source group and q_2 labeling queries for the target group, with the corresponding responses, where $q_1 + q_2 \leq q$; similarly, by $\{(\text{que}_j, \text{res}_j)\}_{j=1}^q \xleftarrow{\text{query}} \text{inv-AE}^{\mathcal{B}}(\text{pk})$, we denote that the algorithm $\text{inv-AE}^{\mathcal{B}}(\text{pk})$ makes q queries with the corresponding responses¹⁷.

Trivial to note that the adversary $\mathcal{A}^{\mathcal{B}}$ is query-efficient. We next prove that the adversary $\mathcal{A}^{\mathcal{B}}$ can successfully guess the valid shared key with a noticeable probability. Let $S_{\text{Bob-L}}^{\text{sou}}$ denote the set of all valid group encodings for source group that appear when running the algorithms $\text{KGen}^{\mathcal{B}}(\text{sk}_2)$ and $\text{SHK}^{\mathcal{B}}(\text{pk}_1, \text{pk}_2)$, and $S_{\text{Bob-L}}^{\text{tar}}$ be those for target group; these group encodings are either the responses of queries or the valid inputs of the addition/bilinear map queries. Clearly, $|S_{\text{Bob-L}}^{\text{sou}}| + |S_{\text{Bob-L}}^{\text{tar}}| \leq 6q$, due to the fact that each algorithm makes at most q queries to $\mathcal{B}$. We then define:

$$S_{\text{Bob}}^{\text{sou}} := \{(x, \text{str}) | \text{str} \in S_{\text{Bob-L}}^{\text{sou}}, \mathcal{B}^{\text{S-label}}(x) = \text{str}\};$$

$$S_{\text{Bob}}^{\text{tar}} := \{(x, \text{str}) | \text{str} \in S_{\text{Bob-L}}^{\text{tar}}, \mathcal{B}^{\text{T-label}}(x) = \text{str}\}.$$

Note that in each iteration, if the encoding pairs in $\tilde{S}_{\mathcal{A}}^{\text{sou}}$ and $\tilde{S}_{\mathcal{A}}^{\text{tar}}$ are consistent with $S_{\text{Bob}}^{\text{sou}}$ and $S_{\text{Bob}}^{\text{tar}}$, respectively, the shared key computed in that iteration would be correct. Specifically, in such a context, there exists another instance of GBM that is consistent with both the simulation view of $\mathcal{A}$ and the real view of user Bob. The perfect correctness of $\text{ae-}\mathcal{P}^{\text{NIKE}}$ guarantees that the shared key computed in this iteration must be equal to the true key computed by Bob. However, without knowledge of sk_2 , $\tilde{S}_{\mathcal{A}}^{\text{sou}}$ and/or $\tilde{S}_{\mathcal{A}}^{\text{tar}}$ might be inconsistent with $S_{\text{Bob}}^{\text{sou}}$ and $S_{\text{Bob}}^{\text{tar}}$ with high probability. In fact, there are three events:

- Event 1: There exist (x, str) in $\tilde{S}_{\mathcal{A}}^{\text{sou}}$ (resp., $\tilde{S}_{\mathcal{A}}^{\text{tar}}$) and (x', str') in $S_{\text{Bob}}^{\text{sou}}$ (resp., $S_{\text{Bob}}^{\text{tar}}$) such that $x = x'$ but $\text{str} \neq \text{str}'$.
- Event 2: There exist (x, str) in $\tilde{S}_{\mathcal{A}}^{\text{sou}}$ (resp., $\tilde{S}_{\mathcal{A}}^{\text{tar}}$) and (x', str') in $S_{\text{Bob}}^{\text{sou}}$ (resp., $S_{\text{Bob}}^{\text{tar}}$) such that $x \neq x'$ but $\text{str} = \text{str}'$.
- Event 3: For any (x, str) in $\tilde{S}_{\mathcal{A}}^{\text{sou}}$ (resp., $\tilde{S}_{\mathcal{A}}^{\text{tar}}$) and (x', str') in $S_{\text{Bob}}^{\text{sou}}$ (resp., $S_{\text{Bob}}^{\text{tar}}$), we have that $x = x'$ if and only if $\text{str} = \text{str}'$.

We immediately observe that Event 1 occurs at most $6q$ times, since the updating phase would eliminate at least one pair in $S_{\text{Bob}}^{\text{sou}}$ or $S_{\text{Bob}}^{\text{tar}}$ with each occurrence. Next we show that, with a noticeable probability, Event 3 would deduce the valid shared key. According to the adversary, depicted in Figure 22, we note that the sets $S_{\text{que-res}}^{\text{sou}} \cup \tilde{S}_{\mathcal{A}}^{\text{sou}}$ and $S_{\text{que-res}}^{\text{tar}} \cup \tilde{S}_{\mathcal{A}}^{\text{tar}}$ responds to the labeling queries properly. For the addition/bilinear map queries, say que , there are four cases:

- Case 1: $\text{que} = (\text{str}_1, \text{str}_2)$ is an addition query for source or target group, and $S_{\text{que-res}}^{\text{sou}} \cup \tilde{S}_{\mathcal{A}}^{\text{sou}}$ or $S_{\text{que-res}}^{\text{tar}} \cup \tilde{S}_{\mathcal{A}}^{\text{tar}}$, respectively, covers (x_1, str_1) , (x_2, str_2) , and $(x_1 + x_2, \text{str}_3)$;
- Case 2: $\text{que} = (\text{str}_1, \text{str}_2)$ is an addition query for source or target group, but either str_1 or str_2 is not collected in $S_{\text{que-res}}^{\text{sou}} \cup \tilde{S}_{\mathcal{A}}^{\text{sou}}$, or $S_{\text{que-res}}^{\text{tar}} \cup \tilde{S}_{\mathcal{A}}^{\text{tar}}$ respectively;
- Case 3: $\text{que} = (\text{str}_1, \text{str}_2)$ is a bilinear map query, and $S_{\text{que-res}}^{\text{sou}} \cup \tilde{S}_{\mathcal{A}}^{\text{sou}}$ covers (x_1, str_1) , (x_2, str_2) , $S_{\text{que-res}}^{\text{tar}} \cup \tilde{S}_{\mathcal{A}}^{\text{tar}}$ covers $(x_1 \cdot x_2, \text{str}_3)$;
- Case 4: $\text{que} = (\text{str}_1, \text{str}_2)$ is a bilinear map query, but either str_1 or str_2 is not collected in $S_{\text{que-res}}^{\text{sou}} \cup \tilde{S}_{\mathcal{A}}^{\text{sou}}$.

For Case 1 and Case 3, $\tilde{S}_{\mathcal{A}}^{\text{sou}} \cup S_{\text{que-res}}^{\text{sou}}$ and $\tilde{S}_{\mathcal{A}}^{\text{tar}} \cup S_{\text{que-res}}^{\text{tar}}$ responds to the addition/bilinear map query properly; for Case 2 and Case 4, $\tilde{S}_{\mathcal{A}}^{\text{sou}} \cup S_{\text{que-res}}^{\text{sou}}$ or $\tilde{S}_{\mathcal{A}}^{\text{tar}} \cup S_{\text{que-res}}^{\text{tar}}$ always responds with $\perp$, which means that if both str_1 and str_2 are valid group elements then the response is invalid. Therefore, the only bad case that prevents Event 3 from guessing a proper shared key is that the adversary $\mathcal{A}$ generates valid group elements str of source or

¹⁷As explained above, we assume that the algorithms $\text{KGen}^{\mathcal{B}}$ and $\text{AE}^{\mathcal{B}}$ only make labeling queries.

Adversary $\mathcal{A}^B$

$\mathcal{A}^B(\text{pk}_1, \text{pk}_2)$:

$S_{\text{key}} \leftarrow \emptyset, S_{\text{que-res}}^{\text{sou}} \leftarrow \emptyset, S_{\text{que-res}}^{\text{tar}} \leftarrow \emptyset$;

if $\text{inv-AE}^B(\text{pk}_1) = \emptyset$ or $\text{inv-AE}^B(\text{pk}_2) = \emptyset$: return $\perp$;

Preprocessing phase: *//collect queries from admissible encoding*

for every seed $\in \text{inv-AE}^B(\text{pk}_1) \cup \text{inv-AE}^B(\text{pk}_2)$:

$\{(\text{que}_j^{\text{sou}}, \text{res}_j^{\text{sou}})\}_{j=1}^{q_1}, \{(\text{que}_j^{\text{tar}}, \text{res}_j^{\text{tar}})\}_{j=1}^{q_2} \xleftarrow{\text{query}} \text{AE}^B(\text{seed})$;

$S_{\text{que-res}}^{\text{sou}} \leftarrow S_{\text{que-res}}^{\text{sou}} \cup \{(\text{que}_j^{\text{sou}}, \text{res}_j^{\text{sou}})\}_{j=1}^{q_1}; S_{\text{que-res}}^{\text{tar}} \leftarrow S_{\text{que-res}}^{\text{tar}} \cup \{(\text{que}_j^{\text{tar}}, \text{res}_j^{\text{tar}})\}_{j=1}^{q_2}$;

for $i \in \{1, 2\}$:

$\{(\text{que}_j, \text{res}_j)\}_{j=1}^q \xleftarrow{\text{query}} \text{inv-AE}^B(\text{pk}_i)$;

for $j = 1$ to q :

if que_j is a labeling query for the source or target group, insert $(\text{que}_j, \text{res}_j)$ into $S_{\text{que-res}}^{\text{sou}}$ or $S_{\text{que-res}}^{\text{tar}}$, respectively;

if $\text{que}_j = (\text{str}_1, \text{str}_2)$ is an addition/bilinear map query with str_1 and str_2 in the corresponding $S_{\text{que-res}}^{\text{sou}}$ or $S_{\text{que-res}}^{\text{tar}}$, transform que_j into a labeling query and record it in $S_{\text{que-res}}^{\text{sou}}$ or $S_{\text{que-res}}^{\text{tar}}$, respectively;

Iteration phase:

for $i = 1$ to $12q + 1$:

search a private key $\tilde{\text{sk}}_1$ and two sets of encoding pairs $\tilde{S}_{\mathcal{A}}^{\text{sou}}$ and $\tilde{S}_{\mathcal{A}}^{\text{tar}}$ satisfying the following properties:

1. $\tilde{S}_{\mathcal{A}}^{\text{sou}}$ and $\tilde{S}_{\mathcal{A}}^{\text{tar}}$ are consistent with $S_{\text{que-res}}^{\text{sou}}$ and $S_{\text{que-res}}^{\text{tar}}$, respectively;
2. $|\tilde{S}_{\mathcal{A}}^{\text{sou}}| + |\tilde{S}_{\mathcal{A}}^{\text{tar}}| \leq 12q$;
3. $\text{pk}_1 \leftarrow \text{KGen}^{S_1, S_2}(\tilde{\text{sk}}_1)$, where $S_1 := S_{\text{que-res}}^{\text{sou}} \cup \tilde{S}_{\mathcal{A}}^{\text{sou}}$ and $S_2 := S_{\text{que-res}}^{\text{tar}} \cup \tilde{S}_{\mathcal{A}}^{\text{tar}}$;
4. $\tilde{S}_{\mathcal{A}}^{\text{sou}}$ and $\tilde{S}_{\mathcal{A}}^{\text{tar}}$ are sufficient for the algorithm $\tilde{\text{shk}}_i \leftarrow \text{SHK}^{S_1, S_2}(\text{pk}_2, \tilde{\text{sk}}_1)$:
 - if $\text{que} = x$ is a labeling query for the source or target group, then x should be covered in S_1 or S_2 , respectively;
 - if $\text{que} = (\text{str}_1, \text{str}_2)$ is an addition query for the source or target group, and there are (x_1, str_1) and (x_2, str_2) in the corresponding S_1 or S_2 , then $(x_1 + x_2, \text{str}_3)$ should also be covered in S_1 or S_2 , respectively;
 - if $\text{que} = (\text{str}_1, \text{str}_2)$ is a bilinear map query with (x_1, str_1) and (x_2, str_2) in S_1 , then $(x_1 + x_2, \text{str}_3)$ should also be covered in S_2 ;
 - otherwise, respond to the query with $\perp$.

$S_{\text{key}} \leftarrow S_{\text{key}} \cup \{\tilde{\text{shk}}_i\}$;

for each $(x, \text{str}) \in \tilde{S}_{\mathcal{A}}^{\text{sou}} \setminus S_{\text{que-res}}^{\text{sou}}$: $S_{\text{que-res}}^{\text{sou}} \leftarrow S_{\text{que-res}}^{\text{sou}} \cup (x, \mathcal{B}^{\text{S-label}}(x))$;

for each $(x, \text{str}) \in \tilde{S}_{\mathcal{A}}^{\text{tar}} \setminus S_{\text{que-res}}^{\text{tar}}$: $S_{\text{que-res}}^{\text{tar}} \leftarrow S_{\text{que-res}}^{\text{tar}} \cup (x, \mathcal{B}^{\text{T-label}}(x))$;

Final phase: *//output the guessing shared key*

return the majority value in S_{key} .

Figure 22: The description of $\mathcal{A}$ that breaks the KRA-secure game of $\text{ae-}\mathcal{P}^{\text{NIKE}}$ in $\mathcal{B}$.

target group without making labeling queries (i.e., without knowing the corresponding discrete logarithm). Besides, we observe that, when Event 2 occurs, the adversary also generates valid group elements without making labeling queries. Hence, if the probability of such a bad case is small, then the adversary $\mathcal{A}^B$ breaks the KRA-secure game and outputs the proper shared key with a good probability.

According to the description of the adversary $\mathcal{A}^B$, we immediately observe that, $\mathcal{A}^B$ aborts if either pk_1 or pk_2 has no preimage with respect to AE^B . To analyze the probability of such a bad event, we define a tuple $(\mathcal{B}, sk_1, sk_2)$ is invertible if the following conditions are satisfied:

1. $\text{inv-}AE^B(\text{KGen}^B(sk_1)) \neq \emptyset$;
2. $\text{inv-}AE^B(\text{KGen}^B(sk_2)) \neq \emptyset$.

Due to the fact that AE^B is a ϱ -regular admissible encoding from $\mathbb{Z}_p$ to the N valid public keys, there are at least $\left\lceil \frac{p}{\varrho} \right\rceil$ public keys that have preimages with respect to AE^B , and each of them corresponds to one private key. Therefore, we have that

$$\begin{aligned} \Pr[(\mathcal{B}, sk_1, sk_2) \text{ is invertible}] &\geq \frac{1}{\varrho^2} \cdot \left(\frac{p}{N}\right)^2 \\ &\geq \frac{1}{\varrho^2} \cdot \frac{1}{\text{poly}_{AE}(\lambda)^2}, \end{aligned}$$

where the probability is over the sampling of the instance of the GBM and private keys.

Moreover, we observe that once the randomness of the adversary is fixed, then the algorithm $\mathcal{A}^B(pk_1, pk_2)$ is deterministic. Let r be a nonce, we say the adversary performs good with respect to the tuple $(\mathcal{B}, sk_1, sk_2, r)$, if the adversary $\mathcal{A}^B(pk_1, pk_2)$, utilizing r as its source of randomness, is unable to generate a valid group element without knowing the discrete logarithm. Here, pk_i represents the public key generated from the private key sk_i .

Next, we introduce the concept of “good” for the tuple $(\mathcal{B}, sk_1, sk_2)$. More formally, we say a tuple $(\mathcal{B}, sk_1, sk_2)$ is good if

$$\Pr[\mathcal{A} \text{ performs good with respect to } (\mathcal{B}, sk_1, sk_2, r)] \geq \frac{1}{2},$$

where $pk_1 \leftarrow \text{KGen}^B(sk_1)$ and $pk_2 \leftarrow \text{KGen}^B(sk_2)$, and the probability is over the internal randomness of $\mathcal{A}$. Analogously, we say a tuple $(\mathcal{B}, sk_1, sk_2)$ is bad if

$$\Pr[\mathcal{A} \text{ performs good with respect to } (\mathcal{B}, sk_1, sk_2, r)] < \frac{1}{2}.$$

For clarity, we denote these three events in which the tuple $(\mathcal{B}, sk_1, sk_2)$ is invertible, good or bad as Invertible, Good and Bad, respectively. Due to the perfect correctness of Π^B , it is apparent that the probability $\mathcal{A}$ wins the KRA-secure game, conditioned on $\text{Good} \wedge \text{Invertible}$, is $\geq \frac{1}{2}$. Thus, the probability of $\mathcal{A}$ wins can be bounded by

$$\begin{aligned} \Pr[\mathcal{A} \text{ wins}] &\geq \Pr[\mathcal{A}^B \text{ wins} | \text{Good} \wedge \text{Invertible}] \cdot \Pr[\text{Good} \wedge \text{Invertible}] \\ &\geq \frac{1}{2} \cdot \Pr[\text{Good} \wedge \text{Invertible}] \\ &= \frac{1}{2} \cdot \Pr[\text{Invertible}] \cdot \Pr[\text{Good} | \text{Invertible}] \\ &\geq \frac{1}{2\varrho^2} \cdot \Pr[\text{Good} | \text{Invertible}]. \end{aligned}$$

Therefore, it suffices to prove that $\Pr[\text{Good} | \text{Invertible}]$ is noticeable. To finalize the proof, we utilize the sparsity property of the GBM. As explained in Section 3, it is hard for any algorithm, taking only the security

Extractor $\mathcal{E}^{\mathcal{B}}(1^\lambda)$

$\text{seed}_1, \text{seed}_2 \xleftarrow{\$} \mathbb{Z}_p; \text{pk}_1 \leftarrow \text{AE}^{\mathcal{B}}(\text{seed}_1); \text{pk}_2 \leftarrow \text{AE}^{\mathcal{B}}(\text{seed}_2);$

$S_{\text{que-res}}^{\text{sou}} \leftarrow \emptyset, S_{\text{que-res}}^{\text{tar}} \leftarrow \emptyset;$

Preprocessing phase: //collect queries from admissible encoding

for every $\text{seed} \in \text{inv-AE}^{\mathcal{B}}(\text{pk}_1) \cup \text{inv-AE}^{\mathcal{B}}(\text{pk}_2)$:

$\{(\text{que}_j^{\text{sou}}, \text{res}_j^{\text{sou}})\}_{j=1}^{q_1}, \{(\text{que}_j^{\text{tar}}, \text{res}_j^{\text{tar}})\}_{j=1}^{q_2} \xleftarrow{\text{query}} \text{AE}^{\mathcal{B}}(\text{seed});$

$S_{\text{que-res}}^{\text{sou}} \leftarrow S_{\text{que-res}}^{\text{sou}} \cup \{(\text{que}_j^{\text{sou}}, \text{res}_j^{\text{sou}})\}_{j=1}^{q_1}, S_{\text{que-res}}^{\text{tar}} \leftarrow S_{\text{que-res}}^{\text{tar}} \cup \{(\text{que}_j^{\text{tar}}, \text{res}_j^{\text{tar}})\}_{j=1}^{q_2};$

for $i \in \{1, 2\}$:

$\{(\text{que}_j, \text{res}_j)\}_{j=1}^q \xleftarrow{\text{query}} \text{inv-AE}^{\mathcal{B}}(\text{pk}_i);$

for $j = 1$ to q :

if que_j is a labeling query or addition/bilinear map query that can be converted into a labeling query, record in $S_{\text{que-res}}^{\text{sou}}$ or $S_{\text{que-res}}^{\text{tar}}$, respectively;

if $\text{que}_j = (\text{str}_1, \text{str}_2)$ is a bilinear map/addition query for source group, with str_1 or str_2 not in $S_{\text{que-res}}^{\text{sou}} \cup \tilde{S}_{\mathcal{A}}^{\text{sou}}$:

if $\text{str}_1 \notin S_{\text{que-res}}^{\text{sou}} \cup \tilde{S}_{\mathcal{A}}^{\text{sou}}$ and $\mathcal{B}^{\text{S-add}}(\text{str}_1, \text{str}_1) \neq \perp$, return (S, str_1) ;

if $\text{str}_2 \notin S_{\text{que-res}}^{\text{sou}} \cup \tilde{S}_{\mathcal{A}}^{\text{sou}}$ and $\mathcal{B}^{\text{S-add}}(\text{str}_2, \text{str}_2) \neq \perp$, return (S, str_2) ;

if $\text{que}_j = (\text{str}_1, \text{str}_2)$ is an addition query for target group, with str_1 or str_2 not in $S_{\text{que-res}}^{\text{tar}} \cup \tilde{S}_{\mathcal{A}}^{\text{tar}}$:

if $\text{str}_1 \notin S_{\text{que-res}}^{\text{tar}} \cup \tilde{S}_{\mathcal{A}}^{\text{tar}}$ and $\mathcal{B}^{\text{T-add}}(\text{str}_1, \text{str}_1) \neq \perp$, return (T, str_1) ;

if $\text{str}_2 \notin S_{\text{que-res}}^{\text{tar}} \cup \tilde{S}_{\mathcal{A}}^{\text{tar}}$ and $\mathcal{B}^{\text{T-add}}(\text{str}_2, \text{str}_2) \neq \perp$, return (T, str_2) ;

Iteration phase:

for $i = 1$ to $12q + 1$:

search a private key $\tilde{\text{sk}}_1$ and two sets of encoding pairs $\tilde{S}_{\mathcal{A}}^{\text{sou}}$ and $\tilde{S}_{\mathcal{A}}^{\text{tar}}$ satisfying the following properties:

1. $\tilde{S}_{\mathcal{A}}^{\text{sou}}$ and $\tilde{S}_{\mathcal{A}}^{\text{tar}}$ are consistent with $S_{\text{que-res}}^{\text{sou}}$ and $S_{\text{que-res}}^{\text{tar}}$, respectively;
2. $|\tilde{S}_{\mathcal{A}}^{\text{sou}}| + |\tilde{S}_{\mathcal{A}}^{\text{tar}}| \leq 12q$;
3. $\text{pk}_1 \leftarrow \text{KGen}^{S_1, S_2}(\tilde{\text{sk}}_1)$, where $S_1 := S_{\text{que-res}}^{\text{sou}} \cup \tilde{S}_{\mathcal{A}}^{\text{sou}}$ and $S_2 := S_{\text{que-res}}^{\text{tar}} \cup \tilde{S}_{\mathcal{A}}^{\text{tar}}$;
4. $\tilde{S}_{\mathcal{A}}^{\text{sou}}$ and $\tilde{S}_{\mathcal{A}}^{\text{tar}}$ are sufficient for the algorithm $\tilde{\text{shk}}_i \leftarrow \text{SHK}^{S_1, S_2}(\text{pk}_2, \tilde{\text{sk}}_1)$;

Re-run the algorithm $\tilde{\text{shk}}_i \leftarrow \text{SHK}^{S_1, S_2}(\text{pk}_2, \tilde{\text{sk}}_1)$ and during the execution:

if $\text{que} = (\text{str}_1, \text{str}_2)$ is a bilinear map/addition query for source group:

if $\text{str}_1 \notin S_{\text{que-res}}^{\text{sou}} \cup \tilde{S}_{\mathcal{A}}^{\text{sou}}$ and $\mathcal{B}^{\text{S-add}}(\text{str}_1, \text{str}_1) \neq \perp$, return (S, str_1) ;

if $\text{str}_2 \notin S_{\text{que-res}}^{\text{sou}} \cup \tilde{S}_{\mathcal{A}}^{\text{sou}}$ and $\mathcal{B}^{\text{S-add}}(\text{str}_2, \text{str}_2) \neq \perp$, return (S, str_2) ;

if $\text{que} = (\text{str}_1, \text{str}_2)$ is an addition query for target group:

if $\text{str}_1 \notin S_{\text{que-res}}^{\text{tar}} \cup \tilde{S}_{\mathcal{A}}^{\text{tar}}$ and $\mathcal{B}^{\text{T-add}}(\text{str}_1, \text{str}_1) \neq \perp$, return (T, str_1) ;

if $\text{str}_2 \notin S_{\text{que-res}}^{\text{tar}} \cup \tilde{S}_{\mathcal{A}}^{\text{tar}}$ and $\mathcal{B}^{\text{T-add}}(\text{str}_2, \text{str}_2) \neq \perp$, return (T, str_2) ;

for each $(x, \text{str}) \in \tilde{S}_{\mathcal{A}}^{\text{sou}} \setminus S_{\text{que-res}}^{\text{sou}}$: $S_{\text{que-res}}^{\text{sou}} \leftarrow S_{\text{que-res}}^{\text{sou}} \cup (x, \mathcal{B}^{\text{S-label}}(x))$;

for each $(x, \text{str}) \in \tilde{S}_{\mathcal{A}}^{\text{tar}} \setminus S_{\text{que-res}}^{\text{tar}}$: $S_{\text{que-res}}^{\text{tar}} \leftarrow S_{\text{que-res}}^{\text{tar}} \cup (x, \mathcal{B}^{\text{T-label}}(x))$;

return $\perp$.

Figure 23: The description of $\mathcal{E}$ w.r.t. $\text{ae-}\mathcal{P}^{\text{NIKE}}$ that outputs a new group element of $\mathcal{B}$.

parameter as input and with access to sparse GGM, to output a valid group element without knowledge of the corresponding discrete logarithm. We stress that this statement still holds even when the model is extended to the GBM, where both the source generic group and target generic group are sparse¹⁸. For ease of exposition, we refer to this event as the algorithm outputs a new group element. More precisely, we say an algorithm $\mathcal{E}$ outputs a new group element of source group, say (S, str) ¹⁹, if (1) str has been used as an input to some bilinear map queries or addition queries for source group made by $\mathcal{E}$, and (2) str has not appeared as the output of any previous queries for source group made by $\mathcal{E}$. Analogously, we say $\mathcal{E}$ outputs a new group element of target group, say (T, str) , if (1) str has been used as an input to some addition queries for target group made by $\mathcal{E}$, and (2) str has not appeared as the output of any previous queries for target group made by $\mathcal{E}$.

We note that, if $\Pr[\text{Good}|\text{Invertible}]$ is small, then, conditioned on Invertible , the adversary $\mathcal{A}$ is likely to generate a new group element with a good probability. Consequently, our proof strategy proceeds in two steps:

1. Construct an extractor $\mathcal{E}$ that takes only the security parameter as input;
2. Demonstrate that, if $\Pr[\text{Good}|\text{Invertible}]$ is small, then $\mathcal{E}$ outputs a new group element with a good probability.

We now present the formal description of the extractor $\mathcal{E}$, as depicted in Figure 23. The extractor $\mathcal{E}$ takes only the security parameter as input, randomly selects two seeds (i.e., seed_1 and seed_2), and computes the corresponding public keys (i.e., pk_1 and pk_2) using admissible encodings. Following this, the extractor proceeds in a manner closely resembling the adversary in the KRA-security game, as shown in Figure 22. Specifically, the extractor proceeds the preprocessing phase and the iteration phase. During the execution, if an addition or bilinear map query, denoted by $\text{que} = (\text{str}_1, \text{str}_2)$, involves either str_1 or str_2 being new and valid group elements, then $\mathcal{E}$ outputs the corresponding group element (highlighted in red). Observe that the randomness of $\mathcal{E}$ is composed of three components: seed_1 , seed_2 , and r , where r represents the randomness used during the iteration phase. Furthermore, once the GBM instance and the two seeds are fixed, the extractor $\mathcal{E}$ proceeds identically to the adversary $\mathcal{A}$ in the KRA-security game, continuing until $\mathcal{E}$ successfully outputs a new and valid group element.

Next, we analyze the probability of the extractor wins, which occurs when $\mathcal{E}$ outputs a new and valid group element. Let $\mathcal{B}$ denote the GBM to which the extractor $\mathcal{E}$ has access. Let seed_1 and seed_2 denote the seeds sampled by the extractor. We define the seed pair, $(\text{seed}_1, \text{seed}_2)$, to be bad with respect to $\mathcal{B}$, if the tuple $(\mathcal{B}, \text{sk}_1, \text{sk}_2)$, deduced from $(\text{seed}_1, \text{seed}_2)$, is a Bad and Invertible tuple. More precisely, when we say a tuple $(\mathcal{B}, \text{sk}_1, \text{sk}_2)$ deduced from $(\text{seed}_1, \text{seed}_2)$, we mean that

$$\text{KGen}^{\mathcal{B}}(\text{sk}_1) = \text{AE}^{\mathcal{B}}(\text{seed}_1) \text{ and } \text{KGen}^{\mathcal{B}}(\text{sk}_2) = \text{AE}^{\mathcal{B}}(\text{seed}_2).$$

For clarity, we refer to this event as BadSeed in the following analysis. According to the definition of Bad, we note that if $\mathcal{E}$ fortunately samples a bad seed pair, i.e., BadSeed occurs, then $\mathcal{E}$ wins with a good probability. Concretely,

$$\Pr[\mathcal{E}^{\mathcal{B}} \text{ wins} | \text{BadSeed}] \geq \frac{1}{2}.$$

Next, we analyze the probability of BadSeed with respect to any specific GBM instance. To facilitate the analysis, we introduce several concepts relevant to the generic bilinear group model. For any instance of the GBM, denoted as $\mathcal{B}$, we define $Q_{\mathcal{B}}$ as the set of all private keys such that for any private key $\text{sk} \in Q_{\mathcal{B}}$, the corresponding public key (denoted $\text{pk} = \text{KGen}^{\mathcal{B}}(\text{sk})$) has valid preimages under the admissible encodings. Moreover, due to that the admissible encodings is ϱ -regular, it is apparent that

$$\left\lceil \frac{p}{\varrho} \right\rceil \leq |Q_{\mathcal{B}}| \leq N.$$

¹⁸We establish the concrete upper bound for this event in Lemma G.2; due to space limit, we put the proof in Appendix G.2.

¹⁹Here we use additional notation S/T to distinguish between source group and target group.

Next, we categorize all instances of the GBM into two types: good GBM instances and bad GBM instances. Specifically, let $\mathcal{B}$ be a GBM instance, and $Q_{\mathcal{B}} := \{sk_1, \dots, sk_t\}$. Let (i, j) be a pair where $i, j \in [1, t]$ ²⁰. We classify $\mathcal{B}$ as a bad instance if there are more than $\frac{1}{2\varrho^2} \cdot t^2$ pairs such that each induces a Bad and Invertible tuple. More precisely, when we say a pair such as (i^*, j^*) induces a Bad and Invertible tuple, we mean that the tuple $(\mathcal{B}, sk_{i^*}, sk_{j^*})$ is bad and invertible. Analogously, we classify $\mathcal{B}$ as a good instance if there are at most $\frac{1}{2\varrho^2} \cdot t^2$ pairs, each of which induces a Bad and Invertible tuple. Therefore, for any good GBM instance, there are at least $(1 - \frac{1}{2\varrho^2}) \cdot t^2$ tuples that are both Good and Invertible.

Furthermore, for any GBM instance $\mathcal{B}$, if a tuple $(\mathcal{B}, sk_1, sk_2)$ is invertible, then there are at least one seed pair, such as $(seed_1, seed_2)$, from which $(\mathcal{B}, sk_1, sk_2)$ can be deduced from. Consequently, for any fixed GBM instance, we have that,

$$\text{Num of bad and invertible tuples} \leq \text{Num of bad seed pairs}.$$

We are now prepared to establish the lower bound on the probability of the extractor $\mathcal{E}$ wins when accessing $\mathcal{B}$, conditioned on $\mathcal{B}$ being a bad GBM instance. Concretely, let t be the size of $Q_{\mathcal{B}}$, we have that,

$$\begin{aligned} \Pr[\mathcal{E}^{\mathcal{B}} \text{ wins} | \mathcal{B} \text{ is bad}] &\geq \Pr[\mathcal{E}^{\mathcal{B}} \text{ wins} | \text{BadSeed}] \cdot \frac{\text{Num of bad seed pairs}}{\text{Num of all seed pairs}} \\ &\geq \frac{1}{2} \cdot \frac{\text{Num of bad and invertible tuples}}{\text{Num of all seed pairs}} \\ &\geq \frac{1}{2} \cdot \frac{\frac{1}{2\varrho^2} \cdot t^2}{p^2} \\ &\geq \frac{1}{2} \cdot \frac{\frac{1}{2\varrho^2} \cdot \left\lceil \frac{p}{\varrho} \right\rceil^2}{p^2} \geq \frac{1}{4\varrho^4}. \end{aligned}$$

Next, we analyze the probability of $\mathcal{E}$ wins, where the probability is also considered over the uniform sampling of the GBM instance. Specifically,

$$\begin{aligned} \Pr[\mathcal{E} \text{ wins}] &\geq \Pr[\mathcal{E}^{\mathcal{B}} \text{ wins} | \mathcal{B} \text{ is bad}] \cdot \frac{\text{Num of bad GBM instances}}{\text{Num of all GBM instances}} \\ &\geq \frac{1}{4\varrho^4} \cdot \frac{\text{Num of bad GBM instances}}{\text{Num of all GBM instances}}. \end{aligned}$$

For clarity in exposition, we define TotalGBM as the set of all GBM instances, BadGBM as the subset of bad GBM instances, and GoodGBM as the subset of good GBM instances. Moreover, we define

$$\text{BadRatio} := \frac{|\text{BadGBM}|}{|\text{TotalGBM}|}; \text{GoodRatio} := \frac{|\text{GoodGBM}|}{|\text{TotalGBM}|}.$$

Thus, the probability of the extractor $\mathcal{E}$ wins is bounded by

$$\Pr[\mathcal{E} \text{ wins}] \geq \frac{1}{4\varrho^4} \cdot \text{BadRatio}.$$

According to Lemma [G.2](#), we know that the probability of $\mathcal{E}$ outputs a new group element is negligible, indicating that BadRatio is negligible. Therefore, it suffices to prove that, if $\Pr[\text{Good} | \text{Invertible}]$ is not noticeable, then BadRatio is not negligible.

²⁰ i and j may be equal.

Claim. If $\Pr[\text{Good}|\text{Invertible}] \leq \frac{1}{4\varrho^2} \cdot \frac{1}{\text{poly}_{\text{AE}}(\lambda)^2}$, then $\text{BadRatio} \geq \frac{1}{2}$.

We proceed our analysis by contraposition. Specifically, we prove that if $\text{BadRatio} < \frac{1}{2}$, then $\Pr[\text{Good}|\text{Invertible}] > \frac{1}{4\varrho^2} \cdot \frac{1}{\text{poly}_{\text{AE}}(\lambda)^2}$. By definition, we have that,

$$\begin{aligned}\Pr[\text{Good}|\text{Invertible}] &= \frac{\Pr[\text{Good} \wedge \text{Invertible}]}{\Pr[\text{Invertible}]} \\ &= \frac{\sum_{\mathcal{B}} \text{Num of Good and Invertible tuples w.r.t. } \mathcal{B}}{\sum_{\mathcal{B}} \text{Num of Invertible tuples w.r.t. } \mathcal{B}},\end{aligned}$$

where the summation $\sum_{\mathcal{B}}$ is taken over all GBM instances. Note that for any GBM instance $\mathcal{B}$, since the size of the private-key space is N , there are at most N^2 relevant tuples associated with $\mathcal{B}$. This implies that

$$\sum_{\mathcal{B}} \text{Num of Invertible tuples w.r.t. } \mathcal{B} \leq |\text{TotalGBM}| \cdot N^2.$$

Furthermore, by the definition of good GBM instance, for any good GBM instance $\mathcal{B}$, there are at least $(1 - \frac{1}{2\varrho^2}) \cdot t^2$ tuples that are both Good and Invertible tuples associated with $\mathcal{B}$. Therefore, we know that,

$$\sum_{\mathcal{B}} \text{Num of Good and Invertible tuples w.r.t. } \mathcal{B} \geq |\text{GoodGBM}| \cdot (1 - \frac{1}{2\varrho^2}) \cdot t^2,$$

where $t = |Q_{\mathcal{B}}| \geq \frac{p}{\varrho}$. Therefore, we have that

$$\begin{aligned}\Pr[\text{Good}|\text{Invertible}] &\geq \frac{|\text{GoodGBM}| \cdot (1 - \frac{1}{2\varrho^2}) \cdot t^2}{|\text{TotalGBM}| \cdot N^2} \\ &\geq (1 - \text{BadRatio}) \cdot \frac{1}{2\varrho^2} \cdot \frac{1}{\text{poly}_{\text{AE}}(\lambda)^2}.\end{aligned}$$

Therefore, we conclude that, if $\text{BadRatio} < \frac{1}{2}$, then

$$\Pr[\text{Good}|\text{Invertible}] \geq \frac{1}{4\varrho^2} \cdot \frac{1}{\text{poly}_{\text{AE}}(\lambda)^2}.$$

Combining together, we establish the entire proof. □

□

E.2 Sparse GBM vs. dense groups

In this section, we demonstrate that any the cryptographic group with dense group encodings that is CDH-secure, denoted as $\text{d-}\mathcal{P}^{\text{CDH}}$, cannot exist within the sparse GBM. Following the same strategy in Appendix A, it is sufficient to prove that non-interactive key exchange (NIKE) with dense public-key space that is secure against key-recovery attack (KRA-secure), denoted as $\text{d-}\mathcal{P}^{\text{NIKE}}$, does not exist in the sparse GBM.

Theorem E.2. Let $\mathcal{B}_{N, m_S, m_T}$ be a generic bilinear group model with sparse encoding. Let $\Pi^{\mathcal{B}_{N, m_S, m_T}} = (\text{KGen}^{\mathcal{B}_{N, m_S, m_T}}, \text{SHK}^{\mathcal{B}_{N, m_S, m_T}})$ be any non-interactive key exchange protocol with dense public-key space. Then there exists an adversary $\mathcal{A}$ and two polynomials poly_1 and poly_2 such that:

- $\mathcal{A}$ makes poly_1 queries to $\mathcal{B}$;
- $\mathcal{A}$ breaks the KRA-secure game with advantage $\frac{1}{\text{poly}_2}$.

Adversary $\mathcal{A}^{\mathcal{B}}$

$\mathcal{A}^{\mathcal{B}}(\text{pk}_1, \text{pk}_2)$:

$S_{\text{key}} \leftarrow \emptyset; S_{\text{que-res}}^{\text{sou}} \leftarrow \emptyset; S_{\text{que-res}}^{\text{tar}} \leftarrow \emptyset$

Iteration phase:

for $i = 1$ to $12q + 1$:

search a private key $\tilde{\text{sk}}_1$ and two sets of encoding pairs, $\tilde{S}_{\mathcal{A}}^{\text{sou}}$ w.r.t. source group and $\tilde{S}_{\mathcal{A}}^{\text{tar}}$ w.r.t. target group, satisfying the following properties:

1. $\tilde{S}_{\mathcal{A}}^{\text{sou}}$ and $\tilde{S}_{\mathcal{A}}^{\text{tar}}$ are consistent with $S_{\text{que-res}}^{\text{sou}}$ and $S_{\text{que-res}}^{\text{tar}}$, respectively;
2. $|\tilde{S}_{\mathcal{A}}^{\text{sou}}| + |\tilde{S}_{\mathcal{A}}^{\text{tar}}| \leq 12q$;
3. $\text{pk}_1 \leftarrow \text{KGen}^{S_1, S_2}(\tilde{\text{sk}}_1)$, where $S_1 := S_{\text{que-res}}^{\text{sou}} \cup \tilde{S}_{\mathcal{A}}^{\text{sou}}$ and $S_2 := S_{\text{que-res}}^{\text{tar}} \cup \tilde{S}_{\mathcal{A}}^{\text{tar}}$;
4. $\tilde{S}_{\mathcal{A}}^{\text{sou}}$ and $\tilde{S}_{\mathcal{A}}^{\text{tar}}$ are sufficient for the algorithm $\widetilde{\text{shk}}_i \leftarrow \text{SHK}^{S_1, S_2}(\text{pk}_2, \tilde{\text{sk}}_1)$:
 - if $\text{que} = x$ is a labeling query for the source or target group, then x should be covered in S_1 or S_2 , respectively;
 - if $\text{que} = (\text{str}_1, \text{str}_2)$ is an addition query for the source or target group, and there are (x_1, str_1) and (x_2, str_2) in the corresponding S_1 or S_2 , then $(x_1 + x_2, \text{str}_3)$ should also be covered in S_1 or S_2 , respectively;
 - if $\text{que} = (\text{str}_1, \text{str}_2)$ is a bilinear map query with (x_1, str_1) and (x_2, str_2) in S_1 , then $(x_1 + x_2, \text{str}_3)$ should also be covered in S_2 ;
 - otherwise, respond to the query with $\perp$.

$S_{\text{key}} \leftarrow S_{\text{key}} \cup \{\widetilde{\text{shk}}_i\}$;

for each $(x, \text{str}) \in \tilde{S}_{\mathcal{A}}^{\text{sou}} \setminus S_{\text{que-res}}^{\text{sou}}$: $S_{\text{que-res}}^{\text{sou}} \leftarrow S_{\text{que-res}}^{\text{sou}} \cup (x, \mathcal{B}^{\text{S-label}}(x))$;

for each $(x, \text{str}) \in \tilde{S}_{\mathcal{A}}^{\text{tar}} \setminus S_{\text{que-res}}^{\text{tar}}$: $S_{\text{que-res}}^{\text{tar}} \leftarrow S_{\text{que-res}}^{\text{tar}} \cup (x, \mathcal{B}^{\text{T-label}}(x))$;

Final phase: //output the guessing shared key

Return the majority value in S_{key} .

Figure 24: The description of $\mathcal{A}$ that breaks the KRA-secure game of $\text{d-}\mathcal{P}^{\text{NIKE}}$ in $\mathcal{B}$.

Proof. For simplicity, in the following analysis we re-denote the sparse GBM as

$$\mathcal{B} = (\mathcal{B}^{\text{S-label}}, \mathcal{B}^{\text{S-add}}, \mathcal{B}^{\text{T-label}}, \mathcal{B}^{\text{T-add}}, \mathcal{B}^{\text{map}}).$$

Analogously, the non-interactive key exchange protocol with dense public-key space in sparse GBM is denoted as $\Pi^{\mathcal{B}} = (\text{KGen}^{\mathcal{B}}, \text{SHK}^{\mathcal{B}})$.

To establish the proof, we present a formal description of the adversary $\mathcal{A}$, as illustrated in Figure 24. Here, we specify that any algorithm in $\Pi^{\mathcal{B}}$ makes at most q queries to $\mathcal{B}$, where q is a polynomial.

Trivial to note that the adversary $\mathcal{A}^{\mathcal{B}}$ is query-efficient. We next prove that the adversary $\mathcal{A}^{\mathcal{B}}$ can successfully guess the valid shared key with a noticeable probability. Let $S_{\text{Bob-L}}^{\text{sou}}$ denote the set of all valid group encodings for source group that appear when running the algorithms $\text{KGen}^{\mathcal{B}}(\text{sk}_2)$ and $\text{SHK}^{\mathcal{B}}(\text{pk}_1, \text{pk}_2)$, and $S_{\text{Bob-L}}^{\text{tar}}$ be those for target group; those group encodings are either the responses of queries or the valid inputs of the addition/bilinear map queries. Clearly, $|S_{\text{Bob-L}}^{\text{sou}}| + |S_{\text{Bob-L}}^{\text{tar}}| \leq 6q$, due to the fact that each algorithm makes at most q queries to $\mathcal{B}$. We then define:

$$\begin{aligned} S_{\text{Bob}}^{\text{sou}} &:= \{(x, \text{str}) \mid \text{str} \in S_{\text{Bob-L}}^{\text{sou}}, \mathcal{B}^{\text{S-label}}(x) = \text{str}\}; \\ S_{\text{Bob}}^{\text{tar}} &:= \{(x, \text{str}) \mid \text{str} \in S_{\text{Bob-L}}^{\text{tar}}, \mathcal{B}^{\text{T-label}}(x) = \text{str}\}. \end{aligned}$$

Note that in each iteration, if the encoding pairs $\tilde{S}_{\mathcal{A}}^{\text{sou}}$ and $\tilde{S}_{\mathcal{A}}^{\text{tar}}$ are consistent with $S_{\text{Bob}}^{\text{sou}}$ and $S_{\text{Bob}}^{\text{tar}}$, respectively, the shared key computed in that iteration would be correct. Specifically, in such a context, another GBM instance exists that is consistent with both the simulation view of $\mathcal{A}$ and the real view of Bob, and the perfect correctness of $\text{d-}\mathcal{P}^{\text{NIKE}}$ ensures that the shared key computed in this iteration is correct. However, without knowledge of sk_2 , $\tilde{S}_{\mathcal{A}}^{\text{sou}}$ and/or $\tilde{S}_{\mathcal{A}}^{\text{tar}}$ might be inconsistent with $S_{\text{Bob}}^{\text{sou}}$ and $S_{\text{Bob}}^{\text{tar}}$ with high probability. Specifically, as analyzed in Appendix E.1, there are essentially three bad events that prevent $\mathcal{A}$ from guessing the correct shared key:

- Event 1: There exist (x, str) in $\tilde{S}_{\mathcal{A}}^{\text{sou}}$ (resp., $\tilde{S}_{\mathcal{A}}^{\text{tar}}$) and (x', str') in $S_{\text{Bob}}^{\text{sou}}$ (resp., $S_{\text{Bob}}^{\text{tar}}$) such that $x = x'$ but $\text{str} \neq \text{str}'$.
- Event 2: There exist (x, str) in $\tilde{S}_{\mathcal{A}}^{\text{sou}}$ (resp., $\tilde{S}_{\mathcal{A}}^{\text{tar}}$) and (x', str') in $S_{\text{Bob}}^{\text{sou}}$ (resp., $S_{\text{Bob}}^{\text{tar}}$) such that $x \neq x'$ but $\text{str} = \text{str}'$.
- Event 3: When running $\text{SHK}^{S_1, S_2}(\text{pk}_2, \tilde{\text{sk}}_1)$, there exists an addition or bilinear map query $\text{que} = (\text{str}_1, \text{str}_2)$ such that either str_1 or str_2 is not collected in $S_{\text{que-res}}^{\text{sou}} \cup \tilde{S}_{\mathcal{A}}^{\text{sou}}$ (or $S_{\text{que-res}}^{\text{tar}} \cup \tilde{S}_{\mathcal{A}}^{\text{tar}}$, respectively), yet both are present in $S_{\text{Bob}}^{\text{sou}}$ (or $S_{\text{Bob}}^{\text{tar}}$).

The updating phase of $\mathcal{A}$ causes Event 1 to occur at most $6q$ times. Meanwhile, the occurrence of either Event 2 or Event 3 results in a bad case that the adversary generates valid group elements str without making labeling queries (i.e., without knowing the corresponding discrete logarithm). Therefore, if the probability of such a bad case is small, $\mathcal{A}^{\mathcal{B}}$ can break the KRA-secure game and output the valid shared key with good probability.

Consider choosing a GBM $\mathcal{B}$ along with two private keys sk_1 and sk_2 , and fixing them. We observe that once the randomness of the adversary is set, the algorithm $\mathcal{A}^{\mathcal{B}}(\text{pk}_1, \text{pk}_2)$ becomes deterministic. Let r be a nonce; we say the adversary performs good with respect to the tuple $(\mathcal{B}, \text{sk}_1, \text{sk}_2, r)$ if $\mathcal{A}^{\mathcal{B}}(\text{pk}_1, \text{pk}_2)$, using r as its source of randomness, cannot generate a valid group element without knowing the corresponding discrete logarithm. Here, pk_i represents the public key derived from the private key sk_i .

Next, we introduce the concept of “good” for the tuple $(\mathcal{B}, \text{sk}_1, \text{sk}_2)$. More formally, we say a tuple $(\mathcal{B}, \text{sk}_1, \text{sk}_2)$ is good if

$$\Pr[\mathcal{A} \text{ performs good with respect to } (\mathcal{B}, \text{sk}_1, \text{sk}_2, r)] \geq \frac{1}{2},$$

where $\text{pk}_1 \leftarrow \text{KGen}^{\mathcal{B}}(\text{sk}_1)$ and $\text{pk}_2 \leftarrow \text{KGen}^{\mathcal{B}}(\text{sk}_2)$, and the probability is over the internal randomness of $\mathcal{A}$. Analogously, we say a tuple $(\mathcal{B}, \text{sk}_1, \text{sk}_2)$ is bad if

$$\Pr[\mathcal{A} \text{ performs good with respect to } (\mathcal{B}, \text{sk}_1, \text{sk}_2, r)] < \frac{1}{2}.$$

For clarify, we denote these two events in which the tuple $(\mathcal{B}, sk_1, sk_2)$ is good or bad as Good and Bad, respectively. Given the perfect correctness of $\Pi^{\mathcal{B}}$, it follows that the probability of $\mathcal{A}$ winning the KRA-secure game, conditioned on Good, is $\geq \frac{1}{2}$. Thus,

$$\begin{aligned}\Pr[\mathcal{A} \text{ wins}] &\geq \Pr[\mathcal{A}^{\mathcal{B}} \text{ wins} | \text{Good}] \cdot \Pr[\text{Good}] \\ &\geq \frac{1}{2} \cdot \Pr[\text{Good}].\end{aligned}$$

Therefore, it suffices to prove that $\Pr[\text{Good}]$ is noticeable. To finalize the proof, we adopt the same strategy as in Appendix E.1, leveraging the sparsity property of the GBM. Specifically, note that if $\Pr[\text{Good}]$ is small, the adversary $\mathcal{A}$ is likely to generate a valid group element without knowing the corresponding discrete logarithm (which we refer to as a new group element). Consequently, our proof proceeds in two steps:

1. Construct an extractor $\mathcal{E}$ that takes only the security parameter as input;
2. Demonstrate that, if $\Pr[\text{Good}]$ is small, then $\mathcal{E}$ outputs a new group element with a good probability.

These two steps lead to a contradiction of Lemma G.2. We now present the formal description of the extractor $\mathcal{E}$, as illustrated in Figure 25. The extractor $\mathcal{E}$ takes only the security parameter as input, and randomly selects two public keys (i.e., pk_1 and pk_2) from the public-key space. It then operates in a manner closely resembling the adversary in the KRA-security game, as shown in Figure 24. Specifically, the extractor proceeds the iteration phase. During the execution, if an addition query, denoted by $que = (str_1, str_2)$, involves either str_1 or str_2 being a new and valid group element, then $\mathcal{E}$ outputs the corresponding group element (highlighted in red). Observe that the randomness of $\mathcal{E}$ is composed of three components: pk_1 , pk_2 , and r , where r represents the randomness used during the iteration phase. Furthermore, once the GBM instance and the two public keys are fixed, and assuming both pk_1 and pk_2 are valid, the extractor $\mathcal{E}$ proceeds identically to the adversary $\mathcal{A}$ in the KRA-security game, continuing until $\mathcal{E}$ successfully outputs a new and valid group element. Here, we define a public key pk as valid if there exists a corresponding sk such that $\text{KGen}^{\mathcal{B}}(sk) = pk$.

Next, we analyze the probability of the extractor wins, which occurs when $\mathcal{E}$ outputs a new and valid group element. Let $\mathcal{B}$ denote the GBM to which the extractor $\mathcal{E}$ has access. Let pk_1 and pk_2 denote the public keys sampled by the extractor. We define a public-key pair, say (pk_1, pk_2) , to be bad with respect to $\mathcal{B}$ if there exists a tuple $(\mathcal{B}, sk_1, sk_2)$, that corresponds to (pk_1, pk_2) and is a Bad tuple. More precisely, when we say a tuple $(\mathcal{B}, sk_1, sk_2)$ corresponds to (pk_1, pk_2) w.r.t. $\mathcal{B}$, we mean that

$$\text{KGen}^{\mathcal{B}}(sk_1) = pk_1 \text{ and } \text{KGen}^{\mathcal{B}}(sk_2) = pk_2.$$

For simplicity, we refer to this event as BadPKs in the following analysis. According to the definition of Bad, we note that if $\mathcal{E}$ fortunately samples a bad public-key pair, i.e., BadPKs occurs, then $\mathcal{E}$ wins with a good probability. Concretely,

$$\Pr[\mathcal{E}^{\mathcal{B}} \text{ wins} | \text{BadPKs}] \geq \frac{1}{2}.$$

Next, we analyze the probability of BadPKs with respect to any specific GBM instance. To facility the analysis, we categorize all instances of the GBM into two types: good GBM instances and bad GBM instances. Specifically, let $\mathcal{B}$ be a GBM instance. Let (sk_i, sk_j) be a private-key pair where $sk_i, sk_j \in \mathbb{Z}_N$ ²¹. We classify $\mathcal{B}$ as a bad instance if there are more than $\frac{1}{4} \cdot N^2$ private-key pairs such that each induces a Bad tuple. More precisely, when we say a private-key pair (sk_i, sk_j) w.r.t. $\mathcal{B}$ induces a Bad tuple, we mean that the tuple $(\mathcal{B}, sk_i, sk_j)$ is bad. Analogously, we classify $\mathcal{B}$ as a good instance if there are at most $\frac{1}{4} \cdot N^2$ private-key pairs, each of which induces a Bad tuple. Therefore, for any good GBM instance, there are at least $(1 - \frac{1}{4}) \cdot N^2$ tuples that are Good.

²¹Note that sk_i and sk_j may be equal.

Extractor $\mathcal{E}^{\mathcal{B}}(1^\lambda)$

$\text{pk}_1 \xleftarrow{\$} \mathcal{PK}; \text{pk}_2 \xleftarrow{\$} \mathcal{PK};$

$S_{\text{que-res}}^{\text{sou}} \leftarrow \emptyset; S_{\text{que-res}}^{\text{tar}} \leftarrow \emptyset;$

Iteration phase:

for $i = 1$ to $12q + 1$:

search a private key $\tilde{\text{sk}}_1$ and two sets of encoding pairs, $\tilde{S}_{\mathcal{A}}^{\text{sou}}$ w.r.t. source group and $\tilde{S}_{\mathcal{A}}^{\text{tar}}$ w.r.t. target group, satisfying the following properties:

1. $\tilde{S}_{\mathcal{A}}^{\text{sou}}$ and $\tilde{S}_{\mathcal{A}}^{\text{tar}}$ are consistent with $S_{\text{que-res}}^{\text{sou}}$ and $S_{\text{que-res}}^{\text{tar}}$, respectively;
2. $|\tilde{S}_{\mathcal{A}}^{\text{sou}}| + |\tilde{S}_{\mathcal{A}}^{\text{tar}}| \leq 12q$;
3. $\text{pk}_1 \leftarrow \text{KGen}^{S_1, S_2}(\tilde{\text{sk}}_1)$, where $S_1 := S_{\text{que-res}}^{\text{sou}} \cup \tilde{S}_{\mathcal{A}}^{\text{sou}}$ and $S_2 := S_{\text{que-res}}^{\text{tar}} \cup \tilde{S}_{\mathcal{A}}^{\text{tar}}$;
4. $\tilde{S}_{\mathcal{A}}^{\text{sou}}$ and $\tilde{S}_{\mathcal{A}}^{\text{tar}}$ are sufficient for the algorithm $\tilde{\text{shk}}_i \leftarrow \text{SHK}^{S_1, S_2}(\text{pk}_2, \tilde{\text{sk}}_1)$;

Re-run the algorithm $\tilde{\text{shk}}_i \leftarrow \text{SHK}^{S_1, S_2}(\text{pk}_2, \tilde{\text{sk}}_1)$ and during the execution:

if $\text{que} = (\text{str}_1, \text{str}_2)$ is a bilinear map/addition query for source group:

if $\text{str}_1 \notin S_{\text{que-res}}^{\text{sou}} \cup \tilde{S}_{\mathcal{A}}^{\text{sou}}$ and $\mathcal{B}^{\text{S-add}}(\text{str}_1, \text{str}_1) \neq \perp$, return (S, str_1) ;

if $\text{str}_2 \notin S_{\text{que-res}}^{\text{sou}} \cup \tilde{S}_{\mathcal{A}}^{\text{sou}}$ and $\mathcal{B}^{\text{S-add}}(\text{str}_2, \text{str}_2) \neq \perp$, return (S, str_2) ;

if $\text{que} = (\text{str}_1, \text{str}_2)$ is an addition query for target group:

if $\text{str}_1 \notin S_{\text{que-res}}^{\text{tar}} \cup \tilde{S}_{\mathcal{A}}^{\text{tar}}$ and $\mathcal{B}^{\text{T-add}}(\text{str}_1, \text{str}_1) \neq \perp$, return (T, str_1) ;

if $\text{str}_2 \notin S_{\text{que-res}}^{\text{tar}} \cup \tilde{S}_{\mathcal{A}}^{\text{tar}}$ and $\mathcal{B}^{\text{T-add}}(\text{str}_2, \text{str}_2) \neq \perp$, return (T, str_2) ;

for each $(x, \text{str}) \in \tilde{S}_{\mathcal{A}}^{\text{sou}} \setminus S_{\text{que-res}}^{\text{sou}}$: $S_{\text{que-res}}^{\text{sou}} \leftarrow S_{\text{que-res}}^{\text{sou}} \cup (x, \mathcal{B}^{\text{S-label}}(x))$;

for each $(x, \text{str}) \in \tilde{S}_{\mathcal{A}}^{\text{tar}} \setminus S_{\text{que-res}}^{\text{tar}}$: $S_{\text{que-res}}^{\text{tar}} \leftarrow S_{\text{que-res}}^{\text{tar}} \cup (x, \mathcal{B}^{\text{T-label}}(x))$;

return $\perp$.

Figure 25: The description of $\mathcal{E}$ w.r.t. $\text{d-}\mathcal{P}^{\text{NIKE}}$ that outputs a new group element of $\mathcal{B}$.

Furthermore, for any GBM instance $\mathcal{B}$, there is exactly one tuple $(\mathcal{B}, \text{sk}_1, \text{sk}_2)$ corresponds to a particular $(\text{pk}_1, \text{pk}_2)$ with respect to $\mathcal{B}$, where both pk_1 and pk_2 are valid. Consequently, for any fix GBM instance, we have that

$$\text{Num of Bad tuples} = \text{Num of BadPKs}.$$

We are now prepared to establish the lower bound on the probability of the extractor $\mathcal{E}$ wins when accessing $\mathcal{B}$, conditioned on $\mathcal{B}$ being a bad GBM instance. Concretely, we have that

$$\begin{aligned} \Pr[\mathcal{E}^{\mathcal{B}} \text{ wins} \mid \mathcal{B} \text{ is bad}] &\geq \Pr[\mathcal{E}^{\mathcal{B}} \text{ wins} \mid \text{BadPKs}] \cdot \frac{\text{Num of BadPKs}}{\text{Num of all pk pairs}} \\ &\geq \frac{1}{2} \cdot \frac{\text{Num of Bad tuples}}{\text{Num of all pk pairs}} \\ &\geq \frac{1}{2} \cdot \frac{\frac{1}{4} \cdot N^2}{|\mathcal{PK}|^2} \\ &= \frac{1}{8} \cdot \left(\frac{|\mathcal{SK}|}{|\mathcal{PK}|} \right)^2 = \frac{1}{8} \cdot \frac{1}{\Theta(\lambda)^2} \end{aligned}$$

Next, we analyze the probability of $\mathcal{E}$ wins, where the probability is also considered over the uniform sampling of the GBM instance. Specifically,

$$\begin{aligned} \Pr[\mathcal{E} \text{ wins}] &\geq \Pr[\mathcal{E}^{\mathcal{B}} \text{ wins} \mid \mathcal{B} \text{ is bad}] \cdot \frac{\text{Num of bad GBM instances}}{\text{Num of all GBM instances}} \\ &\geq \frac{1}{8} \cdot \frac{1}{\Theta(\lambda)^2} \cdot \frac{\text{Num of bad GBM instances}}{\text{Num of all GBM instances}}. \end{aligned}$$

For clarity in exposition, we define TotalGBM as the set of all GBM instances, BadGBM as the subset of bad GBM instances, and GoodGBM as the subset of good GBM instances. Moreover, we define

$$\text{BadRatio} := \frac{|\text{BadGBM}|}{|\text{TotalGBM}|}; \text{GoodRatio} := \frac{|\text{GoodGBM}|}{|\text{TotalGBM}|}.$$

Thus, the probability of the extractor $\mathcal{E}$ wins is bounded by

$$\Pr[\mathcal{E} \text{ wins}] \geq \frac{1}{8} \cdot \frac{1}{\Theta(\lambda)^2} \cdot \text{BadRatio}.$$

According to the analysis in Lemma G.2, we know that the probability of $\mathcal{E}$ outputs a new group element is negligible, indicating that BadRatio is negligible. Therefore, it suffices to prove that, if $\Pr[\text{Good}]$ is not noticeable, then BadRatio is not negligible.

Claim. If $\Pr[\text{Good}] \leq \frac{1}{2}$, then $\text{BadRatio} \geq \frac{1}{4}$.

We proceed our analysis by contraposition. Specifically, we prove that if $\text{BadRatio} < \frac{1}{4}$, then $\Pr[\text{Good}] > \frac{1}{2}$. Note that for any GBM instance $\mathcal{B}$, since the size of the private-key space is N , there are at most N^2 relevant tuples associated with $\mathcal{B}$. Thus, we have that

$$\Pr[\text{Good}] = \frac{\sum_{\mathcal{B}} \text{Num of Good w.r.t. } \mathcal{B}}{|\text{TotalGBM}| \cdot N^2}$$

where the summation $\sum_{\mathcal{B}}$ is taken over all GBM instances.

Furthermore, by the definition of good GBM instance, for any good GBM instance $\mathcal{B}$, there are at least $(1 - \frac{1}{4}) \cdot N^2 = \frac{3}{4} \cdot N^2$ tuples that are Good associated with $\mathcal{B}$. Therefore, we know that

$$\sum_{\mathcal{B}} \text{Num of Good w.r.t. } \mathcal{B} \geq |\text{GoodGBM}| \cdot \frac{3}{4} \cdot N^2,$$

Therefore, we have that

$$\begin{aligned}\Pr[\text{Good}] &\geq \frac{|\text{GoodGBM}| \cdot \frac{3}{4} \cdot N^2}{|\text{TotalGBM}| \cdot N^2} \\ &\geq (1 - \text{BadRatio}) \cdot \frac{3}{4}.\end{aligned}$$

Therefore, we conclude that, if $\text{BadRatio} < \frac{1}{4}$, then

$$\Pr[\text{Good}] \geq (1 - \frac{1}{4}) \cdot \frac{3}{4} = \frac{9}{16} > \frac{1}{2}.$$

Combining together, we establish the entire proof. □

□

F Discrete logarithm is hard in EC-GGM

In this part, we show that no query-efficient algorithm can solve the discrete logarithm (DL) problem in the elliptic curve generic group model except with negligible probability.

Definition F.1 (Discrete Logarithm of EC-GGM). *Let $\text{ec-}\mathcal{G}_N$ be an elliptic curve generic group model over a point set E defined by the equation $y^2 = F(u)$ over $\mathbb{Z}_p$. Given an adversary $\mathcal{A}$, the game proceeds as follows:*

- The challenger computes $x \xleftarrow{\$} \mathbb{Z}_N$, $P \leftarrow \text{ec-}\mathcal{G}_N^{\text{label}}(x)$, and sends P to the adversary.
- The adversary has access to $\text{ec-}\mathcal{G}_N$, and eventually outputs some $\tilde{x} \in \mathbb{Z}_N$.

We say that an adversary $\mathcal{A}^{\text{ec-}\mathcal{G}_N}$ on input $\text{ec-}\mathcal{G}_N^{\text{label}}(x)$ wins the game if its output $\tilde{x}$ satisfies $\tilde{x} = x$. The advantage of $\mathcal{A}$ with respect to $\text{ec-}\mathcal{G}_N$, denoted $\text{DLAdv}[\mathcal{A}, \text{ec-}\mathcal{G}_N]$, is defined as the probability that $\mathcal{A}$ wins the DLog game in EC-GGM.

Lemma F.1. *Let $\text{ec-}\mathcal{G}_N$ be an elliptic curve generic group model. For any adversary $\mathcal{A}$ that makes at most q number of queries to $\text{ec-}\mathcal{G}_N$, it holds that*

$$\text{DLAdv}[\mathcal{A}, \text{ec-}\mathcal{G}_N] = O\left(\frac{\lambda^2 \cdot q^2}{N}\right).$$

Proof. Shoup [Sho97] established that, in the generic group model $\mathcal{G}_{N,m}$, any adversary making $\tilde{q}$ queries succeeds in the DL game with probability at most $O(\tilde{q}^2/N)$. We now give a reduction showing how any adversary $\mathcal{A}$ for DL w.r.t. EC-GGM can be turned into an adversary $\mathcal{R}$ w.r.t. GGM.

For clarity of exposition, we introduce the notation $\text{str}^a := \mathcal{G}_{N,m}^{\text{add}}(\underbrace{\text{str}, \dots, \text{str}}_a)$ in the generic group model,

where str denotes a valid group element and $a \in \mathbb{Z}_N$. In detail, the adversary $\mathcal{R}^{\mathcal{G}_{N,m}}$ receives as input $\text{str} \leftarrow \mathcal{G}_{N,m}^{\text{label}}(x)$ where $x \xleftarrow{\$} \mathbb{Z}_N$. $\mathcal{R}$ initializes an empty table T which consists of tuples with form of $(\text{str}, P) \in \{0, 1\}^m \times E$. Then the adversary $\mathcal{R}$ proceeds as follows:

- If $\text{str} = \mathcal{G}_{N,m}^{\text{label}}(0)$, output 0. Otherwise, sample $P \xleftarrow{\$} E$, and insert both (str, P) and $(\text{str}^{N-1}, -P)$ into T .
- Run $\mathcal{A}(P)$, answering each of its EC-GGM queries que as follows:
 - If $\text{que} = x'$ is a labeling query, $\mathcal{R}$ queries $\text{str}' \leftarrow \mathcal{G}_{N,m}^{\text{label}}(x')$. If there exists $(\text{str}', P') \in T$, respond with P . Otherwise, sample $P' \xleftarrow{\$} E$, insert (str', P') and $(\text{str}'^{N-1}, -P')$ into T , and respond with P' .

- If $\text{que} = (P_1, P_2)$ is an addition query. For each point P_i that does not exist in T , sample $x_i \xleftarrow{\$} \mathbb{Z}_N$, compute $\text{str}_i \leftarrow \mathcal{G}_{N,m}^{\text{label}}(x_i)$, and insert both (str_i, P_i) and $(\text{str}_i^{N-1}, -P_i)$ into T . Then, for $(\text{str}_1, P_1), (\text{str}_2, P_2) \in T$, compute $\text{str}' \leftarrow \mathcal{G}_{N,m}^{\text{add}}(\text{str}_1, \text{str}_2)$. If there exists (str', P') in T , respond with P' . Otherwise, sample $P' \xleftarrow{\$} E$, insert both (str', P') and $(\text{str}'^{N-1}, -P')$ into T , and respond with P' .

- When $\mathcal{A}$ outputs a candidate discrete logarithm $\tilde{x}$, adversary $\mathcal{R}$ simply returns $\tilde{x}$ as its own output.

Note that the adversary $\mathcal{A}$ makes at most $O(\lambda \cdot q)$ number of queries to $\mathcal{G}_{N,m}$, and that the table T contains at most $6q$ tuples. Denote by Bad_1 the bad event that the uniformly sampled point P already appears in T . The probability of Bad_1 occurs is bounded by

$$\Pr[\text{Bad}_1] \leq \frac{6q^2}{N}.$$

Denote by Bad_2 the bad event that the uniformly sampled group element $\text{str} \leftarrow \mathcal{G}_{N,m}^{\text{label}}(x)$ already exists in the table T . The probability of Bad_2 occurs is bounded by

$$\Pr[\text{Bad}_2] \leq \frac{12q^2}{N}.$$

Apparently, under the condition that both Bad_1 and Bad_2 do not occur, the adversary $\mathcal{R}$ simulates a proper view of the DL game w.r.t. EC-GGM for *adversary*. Hence

$$\begin{aligned} \Pr[\mathcal{R} \text{ wins}] &\geq \Pr[\neg \text{Bad}_1 \wedge \neg \text{Bad}_2] \cdot \text{DLAdv}[\mathcal{A}, \text{ec-}\mathcal{G}_N] \\ &\geq \left(1 - \frac{18q^2}{N}\right) \cdot \text{DLAdv}[\mathcal{A}, \text{ec-}\mathcal{G}_N]. \end{aligned}$$

On the other hand, by Shoup [Sho97], we know $\Pr[\mathcal{R} \text{ wins}] = O\left(\frac{\lambda^2 \cdot q^2}{N}\right)$. Therefore, we have

$$\text{DLAdv}[\mathcal{A}, \text{ec-}\mathcal{G}_N] = O\left(\frac{\lambda^2 \cdot q^2}{N}\right).$$

□

G Extracting new label is hard in sparse GGM/GBM

G.1 Analysis in sparse GGM

In this part, we formalize the process of extracting a new group encoding in GGM as a game, and then analyze the probability that any query-efficient adversary can win this game.

Definition G.1 (Extraction Game of GGM). *Given an adversary $\mathcal{A}$, the game proceeds as follows:*

- The challenger initializes the GGM $\mathcal{G}_{N,m}$ with two oracles, $\mathcal{G}_{N,m}^{\text{label}}$ and $\mathcal{G}_{N,m}^{\text{add}}$, and a table T .
- The adversary then makes a sequence of queries to the challenger. Each query can be one of two types:
 - Labeling query: On input $x \in \mathbb{Z}_N$, the challenger computes $\text{str} = \mathcal{G}_{N,m}^{\text{label}}(x)$, inserts str into the table T , and returns str .
 - Addition query: On input $(\text{str}_0, \text{str}_1)$, where $\text{str}_0, \text{str}_1 \in \{0, 1\}^m$, the challenger computes $\text{res} = \mathcal{G}_{N,m}^{\text{add}}(\text{str}_0, \text{str}_1)$. If $\text{res} \in \{0, 1\}^m$, it is inserted into T . The challenger then returns res .

- Eventually, the adversary outputs a string $\text{str}^* \in \{0, 1\}^m \setminus T$.

We say that $\mathcal{A}$ wins the game if str^* is a valid group encoding in $\mathcal{G}_{N,m}$. The advantage of $\mathcal{A}$ with respect to $\mathcal{G}_{N,m}$, denoted $\text{ExtAdv}[\mathcal{A}, \mathcal{G}_{N,m}]$, is defined as the probability that $\mathcal{A}$ wins the extraction game in GGM.

Lemma G.1. *Let $\mathcal{G}_{N,m}$ be a generic group model. For any adversary $\mathcal{A}$ with access to $\mathcal{G}_{N,m}$, where q_1, q_2 denote the maximum number of queries made to $\mathcal{G}_{N,m}^{\text{label}}$ and $\mathcal{G}_{N,m}^{\text{add}}$, respectively, it holds that*

$$\text{ExtAdv}[\mathcal{A}, \mathcal{G}_{N,m}] \leq \frac{(6q_2 + 1) \cdot N}{2^m - q_1}.$$

Proof. Since any probabilistic adversary can be viewed as a distribution over deterministic adversaries, and the average-case performance of any probabilistic adversary cannot beat that of the best deterministic adversary, it suffices to consider deterministic adversaries in this proof.

For a specific GGM instance, which corresponds to a specific injection $\sigma \in \mathcal{I}_{\mathbb{Z}_N, S}$, once the adversary is deterministic, the result is completely determined. Therefore, it is sufficient to calculate the number of injections that result in $\mathcal{A}$ winning. Let $\mathcal{A}$ be a deterministic adversary in the extraction game. We define $\mathcal{W}_{\text{ext}}(\mathcal{A}, \mathcal{G}_{N,m})$ as the event where $\mathcal{A}$ wins. The advantage of $\mathcal{A}$ in the extraction game is then given by:

$$\text{ExtAdv}[\mathcal{A}, \mathcal{I}_{\mathbb{Z}_N, S}] = \Pr[\mathcal{W}_{\text{ext}}(\mathcal{A}, \mathcal{G}_{N,m})].$$

For any injection $\sigma \in \mathcal{I}_{\mathbb{Z}_N, S}$, let $\mathcal{W}_{\text{ext}}^\sigma(\mathcal{A})$ denote the event where $\mathcal{A}$ wins, conditioned on the initial state of the game being fixed to σ . Since $\mathcal{A}$ is deterministic, for a given σ , the event $\mathcal{W}_{\text{ext}}^\sigma(\mathcal{A})$ will either occur (denoted $\mathcal{W}_{\text{ext}}^\sigma(\mathcal{A}) = 1$) or not occur (denoted $\mathcal{W}_{\text{ext}}^\sigma(\mathcal{A}) = 0$). We define $\mathcal{U}$ as the set of injections σ such that $\mathcal{W}_{\text{ext}}^\sigma(\mathcal{A})$ occurs, i.e.,

$$\mathcal{U} = \{\sigma \in \mathcal{I}_{\mathbb{Z}_N, S} \mid \mathcal{W}_{\text{ext}}^\sigma(\mathcal{A}) = 1\}.$$

Since σ is sampled uniformly at random from $\mathcal{I}_{\mathbb{Z}_N, S}$, we have:

$$\text{ExtAdv}[\mathcal{A}, \mathcal{G}_{N,m}] = \frac{1}{|\mathcal{I}_{\mathbb{Z}_N, S}|} \sum_{\sigma} \mathcal{W}_{\text{ext}}^\sigma(\mathcal{A}) = \frac{|\mathcal{U}|}{|\mathcal{I}_{\mathbb{Z}_N, S}|}.$$

Clearly, $|\mathcal{I}_{\mathbb{Z}_N, S}| = (2^m)! / (2^m - N)!$. Next, we analyze the size of the set $\mathcal{U}$.

For a deterministic adversary, let res_i be the response to the i -th query que_i . There are three possible cases for each query:

- Case 1: $\text{que}_i = x_i \in \mathbb{Z}_N$ is a labeling query, and $\text{res}_i \in S$;
- Case 2: $\text{que}_i = (\text{str}_{i1}, \text{str}_{i2}) \in S^2$ is an addition query, and $\text{res}_i \in S$;
- Case 3: $\text{que}_i = (\text{str}_{i1}, \text{str}_{i2}) \in S^2$ is an addition query, and $\text{res}_i = \perp$;

After querying each oracle the maximum number of times, the adversary outputs a string $\text{str}^* \in S$. We assume that no two queries are identical and that str^* is not among the responses to the previous queries.

Let $\text{res} = \{\text{res}_i\}_i$ denote the responses to all queries from adversary. Since the adversary is deterministic, the sequence of its queries, as well as the output str^* , are entirely determined by the responses res . It follows that for a given initial state σ , the responses are fixed and denoted as res^σ . Hence, we can separately analyze the number of initial states that lead to the adversary winning, given each specific response sequence res . We define the set of all initial states for which the adversary's queries yield the response sequence res as:

$$\mathcal{V}_{\text{res}} = \{\sigma \in \mathcal{I}_{\mathbb{Z}_N, S} \mid \text{res}^\sigma = \text{res}\}$$

Next, we focus on the subset of responses where all elements corresponding to addition queries are $\perp$, i.e.,

$$S_\perp = \{\text{res} \mid \text{res}_i = \perp \text{ for all addition queries } \text{que}_i\}.$$

For any $\text{res} \in S_\perp$, the adversary outputs a string $\text{str}^* \in S$. Thus, $\mathcal{A}$ can win with respect to an injection $\sigma \in \mathcal{V}_{\text{res}}$ if and only if there exists an $x^* \in \mathbb{Z}_N$ such that $\sigma(x^*) = \text{str}^*$. Moreover, since there are q_1 fixed locations in the injection, and t labels must be invalid, where $1 \leq t \leq 2q_2$, the number of injections that make $\mathcal{A}$ win is for a particular $\text{res} \in S_\perp$ is bounded by

$$\begin{aligned} |\mathcal{U} \cap \mathcal{V}_{\text{res}}| &= (N - q_1) \cdot \prod_{i=1}^{\ell} (2^m - (q_1 + t + i)) \\ &\leq (N - q_1) \cdot \prod_{i=1}^{\ell} (2^m - (q_1 + 1 + i)) \end{aligned}$$

where $\ell = N - q_1 - 1$.

Since $|S_\perp| = 2^m \cdots (2^m - q_1 + 1)$, we derive that the total number of injections that make the adversary win across all $\text{res} \in S_\perp$ is bounded by:

$$|\cup_{\text{res} \in S_\perp} (\mathcal{U} \cap \mathcal{V}_{\text{res}})| \leq 2^m \cdots (2^m - q_1 + 1) \cdot (N - q_1) \cdot \prod_{i=1}^{\ell} (2^m - (q_1 + 1 + i)).$$

Let the set of initial states that have not been considered be denoted as $\mathcal{R} = \mathcal{I}_{\mathbb{Z}_N, S} \setminus \cup_{\text{res} \in S_\perp} \mathcal{V}_{\text{res}}$. The size of $\mathcal{R}$ is:

$$|\mathcal{R}| = |\mathcal{I}_{\mathbb{Z}_N, S}| - |\cup_{\text{res} \in S_\perp} \mathcal{V}_{\text{res}}|$$

For any $\text{res} \in S_\perp$, there are q_1 fixed locations in the injection $\sigma \in \mathcal{V}_{\text{res}}$ and t invalid labels. Thus, for each $\text{res} \in S_\perp$, the number of injections

$$|\mathcal{V}_{\text{res}}| = \prod_{i=0}^{\ell} (2^m - (q_1 + t + i)) \geq \prod_{i=0}^{\ell} (2^m - (q_1 + 2q_2 + i)).$$

Consequently, the size of $\mathcal{R}$ is bounded by:

$$|\mathcal{R}| \leq (2^m)! / (2^m - N)! - 2^m \cdots (2^m - q_1 + 1) \cdot \prod_{i=0}^{\ell} (2^m - (q_1 + 2q_2 + i))$$

Asssuming the adversary wins on all the initial states in $\mathcal{R}$, we have

$$\begin{aligned} \frac{|\mathcal{U}|}{|\mathcal{I}_{\mathbb{Z}_N, S}|} &\leq \frac{|\cup_{\text{res} \in S_{\text{no-col}}} (\mathcal{V}_{\text{res}} \cap \mathcal{U})| + |\mathcal{R}|}{|\mathcal{I}_{\mathbb{Z}_N, S}|} \\ &\leq \frac{(N - q_1) \cdot \prod_{i=1}^{\ell} (2^m - (q_1 + 1 + i))}{(2^m - q_1) \cdot \prod_{i=1}^{\ell} (2^m - (q_1 + i))} + 1 - \frac{\prod_{i=0}^{\ell} (2^m - (q_1 + 2q_2 + i))}{\prod_{i=0}^{\ell} (2^m - (q_1 + i))} \\ &\leq \frac{N - q_1}{2^m - q_1} + 1 - \left(1 - \frac{2q_2}{2^m - q_1}\right)^{N - q_1} \\ &\leq \frac{(6q_2 + 1) \cdot N}{2^m - q_1}. \end{aligned}$$

This concludes the proof. □

□

G.2 Analysis in sparse GBM

In this part, we formalize the extraction of a new group encoding in GBM as a game, and then analyze the probability for any query-efficient adversary to win such a game.

Definition G.2 (Extraction Game of GBM). *For a given adversary $\mathcal{A}$, the game runs as follows:*

- The challenger initialises $\mathcal{B}_{N,m_S,m_T} = (\mathcal{B}_{N,m_S,m_T}^{S\text{-label}}, \mathcal{B}_{N,m_S,m_T}^{S\text{-add}}, \mathcal{B}_{N,m_S,m_T}^{T\text{-label}}, \mathcal{B}_{N,m_S,m_T}^{T\text{-add}}, \mathcal{B}_{N,m_S,m_T}^{\text{map}})$ and two tables T_S and T_T .
- $\mathcal{A}$ then makes a series of queries to the challenger, each can be one of three types:
 - Labeling query to $\mathcal{B}_{N,m_S,m_T}^{i\text{-label}}$: On input $x \in \mathbb{Z}_N$, the challenger computes $\text{str} = \mathcal{B}_{N,m_S,m_T}^{i\text{-label}}(x)$, inserts str into T_i and returns str .
 - Addition query to $\mathcal{B}_{N,m_S,m_T}^{i\text{-add}}$: On input $(\text{str}_1, \text{str}_2)$ for $\text{str}_1, \text{str}_2 \in \{0, 1\}^{m_i}$, the challenger computes $\text{res} = \mathcal{B}_{N,m_S,m_T}^{i\text{-add}}(\text{str}_1, \text{str}_2)$. If $\text{res} \in \{0, 1\}^{m_i}$, insert res into T_i . Finally, the challenger returns res .
 - Bilinear map query to $\mathcal{B}_{N,m_S,m_T}^{\text{Map}}$: On input $(\text{str}_1, \text{str}_2)$ for $\text{str}_1, \text{str}_2 \in \{0, 1\}^{m_S}$, the challenger computes $\text{res} = \mathcal{B}_{N,m_S,m_T}^{\text{Map}}(\text{str}_1, \text{str}_2)$. If $\text{res} \in \{0, 1\}^{m_T}$, insert res into T_T . Finally, the challenger returns res .
- Eventually $\mathcal{A}$ outputs a tuple (i, str^*) , where $i \in \{S, T\}$ and $\text{str}^* \in \{0, 1\}^{m_i}$, such that $\text{str}^* \notin T_i$.

We say that $\mathcal{A}$ wins the above game if str^* is a valid encoding with respect to i (namely there exists $x^* \in \mathbb{Z}_N$ such that $\mathcal{B}^{i\text{-label}}(x^*) = \text{str}^*$). We define the advantage of $\mathcal{A}$ with respect to $\mathcal{B}_{N,m_S,m_T}$, denoted by $\text{ExtAdv}[\mathcal{A}, \mathcal{B}_{N,m_S,m_T}]$, as the winning probability of $\mathcal{A}$ in extraction game of GBM.

Lemma G.2. *Let $\mathcal{B}_{N,m_S,m_T}$ be a generic bilinear group model. For any adversary $\mathcal{A}$ with access to $\mathcal{B}_{N,m_S,m_T}$, where $q_{S_1}, q_{S_2}, q_{T_1}, q_{T_2}$ and q_M are the maximum number of queries made to $\mathcal{B}_{N,m_S,m_T}^{S\text{-label}}, \mathcal{B}_{N,m_S,m_T}^{S\text{-add}}, \mathcal{B}_{N,m_S,m_T}^{T\text{-label}}, \mathcal{B}_{N,m_S,m_T}^{T\text{-add}}$ and $\mathcal{B}_{N,m_S,m_T}^{\text{Map}}$, respectively, it holds that*

$$\text{ExtAdv}[\mathcal{A}, \mathcal{B}_{N,m_S,m_T}] \leq \frac{7N \cdot (q_{S_2} + q_{T_2} + q_M)}{\min(2^{m_S} - q_{S_1}, 2^{m_T} - q_{T_1})}.$$

Proof. As explained in Appendix G.1, we consider only the deterministic adversaries in this proof. Since an instance of GBM uniquely determines a pair of injections σ_S and σ_T , once σ_S and σ_T are determined, the result for a deterministic $\mathcal{A}$ is completely certain. Thus we only need to calculate the number of injection pairs that make $\mathcal{A}$ win.

We denote by $\mathcal{W}_{\text{ext}}(\mathcal{A}, \mathcal{B}_{N,m_S,m_T})$ the event that $\mathcal{A}$ win, then we have

$$\text{ExtAdv}[\mathcal{A}, \mathcal{B}_{N,m_S,m_T}] = \Pr[\mathcal{W}_{\text{ext}}(\mathcal{A}, \mathcal{B}_{N,m_S,m_T})]$$

Let $S_S := \{0, 1\}^{m_S}$ and $S_T := \{0, 1\}^{m_T}$. For any injection pair $\sigma = (\sigma_S, \sigma_T)$, where $\sigma_S \in \mathcal{I}_{\mathbb{Z}_N, S_S}$ and $\sigma_T \in \mathcal{I}_{\mathbb{Z}_N, S_T}$, we reuse $\mathcal{W}_{\text{ext}}^\sigma(\mathcal{A})$ to denote the event same as $\mathcal{W}_{\text{ext}}(\mathcal{A}, \mathcal{B}_{N,m_S,m_T})$ except that the initial state of the game is fixed to be σ . For a specific σ , since $\mathcal{A}$ is deterministic, the event $\mathcal{W}_{\text{ext}}^\sigma(\mathcal{A})$ must either occur (denoted by $\mathcal{W}_{\text{ext}}^\sigma(\mathcal{A}) = 1$) or not occur (denoted by $\mathcal{W}_{\text{ext}}^\sigma(\mathcal{A}) = 0$). We denote by $\mathcal{U}$ the set of pairs σ such that $\mathcal{W}_{\text{ext}}^\sigma(\mathcal{A})$ occur, i.e.

$$\mathcal{U} = \{\sigma \in \mathcal{I}_{\mathbb{Z}_N, S_S} \times \mathcal{I}_{\mathbb{Z}_N, S_T} \mid \mathcal{W}_{\text{ext}}^\sigma(\mathcal{A}) = 1\}$$

Since σ is sampled uniformly at random from $\mathcal{I}_{\mathbb{Z}_N, S_S} \times \mathcal{I}_{\mathbb{Z}_N, S_T}$, we have

$$\text{ExtAdv}[\mathcal{A}, \mathcal{B}_{N,m_S,m_T}] = \frac{1}{|\mathcal{I}_{\mathbb{Z}_N, S_S} \times \mathcal{I}_{\mathbb{Z}_N, S_T}|} \sum_{\sigma} \mathcal{W}_{\text{ext}}^\sigma(\mathcal{A}) = \frac{|\mathcal{U}|}{|\mathcal{I}_{\mathbb{Z}_N, S_S} \times \mathcal{I}_{\mathbb{Z}_N, S_T}|}.$$

Clearly $|\mathcal{I}_{\mathbb{Z}_N, S_S} \times \mathcal{I}_{\mathbb{Z}_N, S_T}| = ((2^{m_S})! \cdot (2^{m_T})!) / ((2^{m_S} - N)! \cdot (2^{m_T} - N)!)$, we now analyze the size of the set $\mathcal{U}$. Let res_i be the response of the i -th query que_i of $\mathcal{A}$, for $j \in \{S, T\}$, there are five cases:

- Case 1: que_i is a labeling query to $\mathcal{B}_{N,m_S,m_T}^{j\text{-label}}$ and $\text{res}_i \in S_j$;
- Case 2: que_i is an addition query to $\mathcal{B}_{N,m_S,m_T}^{j\text{-add}}$ and $\text{res}_i \in S_j$;

- Case 3: que_i is an addition query to $\mathcal{B}_{N, m_S, m_T}^{j\text{-add}}$ and $\text{res}_i = \perp$;
- Case 4: que_i is a bilinear map query to $\mathcal{B}_{N, m_S, m_T}^{\text{Map}}$ and $\text{res}_i \in S_T$;
- Case 5: que_i is a bilinear map query to $\mathcal{B}_{N, m_S, m_T}^{\text{Map}}$ and $\text{res}_i = \perp$.

After querying each oracle for maximum number of times, $\mathcal{A}$ outputs a tuple (j, str^*) , where $j \in \{S, T\}$ and $\text{str}^* \in \{0, 1\}^{m_i}$. Naturally, we assume that there are not two identical queries.

Let $\text{res} = \{\text{res}_i\}_i$ denote the responses of all queries from $\mathcal{A}$. We still denote by res^σ the responses determined by a specific initial state σ . Then we analyze separately the number of initial states that make $\mathcal{A}$ win corresponding to each particular series of responses. We now define the set of all initial states on which the responses of the queries from $\mathcal{A}$ are res ,

$$\mathcal{V}_{\text{res}} = \{\sigma \in \mathcal{I}_{\mathbb{Z}_N, S_S} \times \mathcal{I}_{\mathbb{Z}_N, S_T} \mid \text{res}^\sigma = \text{res}\}.$$

Denote by $S_\perp$ the set of responses where all elements w.r.t addition queries or bilinear map queries are $\perp$, namely

$$S_\perp = \{\text{res} \mid \text{res}_i \neq \perp \text{ iff } \text{que}_i \text{ is an labeling query}\}$$

Then we can divide the set $\mathcal{U}$ into two classes:

- Class 1: The set of initial states σ that make $\mathcal{A}$ win and $\text{res}^\sigma \in S_\perp$, denoted as $\mathcal{U}_1$;
- Class 2: The set of initial states σ that make $\mathcal{A}$ win but $\text{res}^\sigma \notin S_\perp$, denoted as $\mathcal{U}_2$;

Below we consider the size of each of the two classes.

Class 1. For any $\text{res} \in S_\perp$, the deterministic $\mathcal{A}$ finally outputs a tuple (j, str^*) . Thus $\mathcal{A}$ can win, with respect to an initial state $\sigma = (\sigma_S, \sigma_T)$, if and only if there exists $x^* \in \mathbb{Z}_N$ such that $\sigma_j(x^*) = \text{str}^*$. At the same time, there are q_{S_1} locations of σ_S and q_{T_1} locations of σ_T are fixed, and t_S labels in $\{0, 1\}^{m_S}$ and t_T labels in $\{0, 1\}^{m_T}$ must be invalid, where $1 \leq t_S \leq 2(q_{S_2} + q_M)$ and $1 \leq t_T \leq q_{T_2}$. For a particular $\text{res} \in S_\perp$, if the tuple output by $\mathcal{A}$ is (S, str^*) , with $\text{str}^* \in \{0, 1\}^{m_S}$, then the number of the injections that make $\mathcal{A}$ win is

$$(N - q_{S_1}) \cdot \prod_{i=1}^{\ell_S} (2^{m_S} - (q_{S_1} + t_S + i)) \cdot \prod_{i=0}^{\ell_T} (2^{m_T} - (q_{T_1} + t_T + i))$$

where $\ell_S = N - q_{S_1} - 1$ and $\ell_T = N - q_{T_1} - 1$. And if the tuple output by $\mathcal{A}$ is (T, str^*) , with $\text{str}^* \in \{0, 1\}^{m_T}$, then the number of the injections that make $\mathcal{A}$ win is

$$(N - q_{T_1}) \cdot \prod_{i=0}^{\ell_S} (2^{m_S} - (q_{S_1} + t_S + i)) \cdot \prod_{i=1}^{\ell_T} (2^{m_T} - (q_{T_1} + t_T + i))$$

Thus we have

$$\begin{aligned} |\mathcal{U} \cap \mathcal{V}_{\text{res}}| &\leq N \cdot \max(2^{m_S} - (q_{S_1} + t_S), 2^{m_T} - (q_{T_1} + t_T)) \cdot \prod_{i=1}^{\ell_S} (2^{m_S} - (q_{S_1} + t_S + i)) \\ &\quad \cdot \prod_{i=1}^{\ell_T} (2^{m_T} - (q_{T_1} + t_T + i)) \\ &\leq N \cdot \max(2^{m_S} - (q_{S_1} + 1), 2^{m_T} - (q_{T_1} + 1)) \cdot \prod_{i=1}^{\ell_S} (2^{m_S} - (q_{S_1} + 1 + i)) \\ &\quad \cdot \prod_{i=1}^{\ell_T} (2^{m_T} - (q_{T_1} + 1 + i)) \end{aligned}$$

Since $\mathcal{U}_1 = \cup_{\text{res} \in S_\perp} (\mathcal{U} \cap \mathcal{V}_{\text{res}})$, we derive that

$$|\mathcal{U}_1| = |\cup_{\text{res} \in S_\perp} (\mathcal{U} \cap \mathcal{V}_{\text{res}})| = |S_\perp| \cdot |\mathcal{U} \cap \mathcal{V}_{\text{res}}|$$

Class 2. We denote the rest of initial states that have not considered as $\mathcal{R} = \mathcal{I}_{\mathbb{Z}_N, S_S} \times \mathcal{I}_{\mathbb{Z}_N, S_T} \setminus \cup_{\text{res} \in S_\perp} \mathcal{V}_{\text{res}}$. We have

$$|\mathcal{R}| = |\mathcal{I}_{\mathbb{Z}_N, S_S} \times \mathcal{I}_{\mathbb{Z}_N, S_T}| - |\cup_{\text{res} \in S_\perp} \mathcal{V}_{\text{res}}|$$

For a specific $\text{res} \in S_\perp$, as explained in Class 1, the number of the injections

$$\begin{aligned} |\mathcal{V}_{\text{res}}| &= \prod_{i=0}^{\ell_S} (2^{m_S} - (q_{S_1} + t_S + i)) \cdot \prod_{i=0}^{\ell_T} (2^{m_T} - (q_{T_1} + t_T + i)) \\ &\geq \prod_{i=0}^{\ell_S} (2^{m_S} - (q_{S_1} + 2q_{S_2} + 2q_M + i)) \cdot \prod_{i=0}^{\ell_T} (2^{m_T} - (q_{T_1} + 2q_{T_2} + i)). \end{aligned}$$

Thus,

$$|\mathcal{R}| \leq |\mathcal{I}_{\mathbb{Z}_N, S_S} \times \mathcal{I}_{\mathbb{Z}_N, S_T}| - |S_\perp| \cdot \prod_{i=0}^{\ell_S} (2^{m_S} - (q_{S_1} + 2q_{S_2} + 2q_M + i)) \cdot \prod_{i=0}^{\ell_T} (2^{m_T} - (q_{T_1} + 2q_{T_2} + i)).$$

We assume the adversary wins on all the initial states in $\mathcal{R}$, therefore

$$\begin{aligned} \frac{|\mathcal{U}|}{|\mathcal{I}_{\mathbb{Z}_N, S}|} &= \frac{|\mathcal{U}_1| + |\mathcal{U}_2|}{|\mathcal{I}_{\mathbb{Z}_N, S_S} \times \mathcal{I}_{\mathbb{Z}_N, S_T}|} \leq \frac{|\mathcal{U}_1| + |\mathcal{R}|}{|\mathcal{I}_{\mathbb{Z}_N, S_S} \times \mathcal{I}_{\mathbb{Z}_N, S_T}|} \\ &\leq \frac{N \cdot \max(2^{m_S} - (q_{S_1} + 1), 2^{m_T} - (q_{T_1} + 1))}{2^{m_S} - q_{S_1} \cdot 2^{m_T} - q_{T_1}} + 1 \\ &\quad - (1 - \frac{2q_{S_2} + 2q_M}{2^{m_S} - q_{S_1}})^{N - q_{S_1}} \cdot (1 - \frac{2q_{T_2}}{2^{m_T} - q_{T_1}})^{N - q_{T_1}} \\ &\leq \frac{7N \cdot (q_{S_2} + q_{T_2} + q_M)}{\min(2^{m_S} - q_{S_1}, 2^{m_T} - q_{T_1})}. \end{aligned}$$

□